

Manual do Laboratório de IA Edge 1

Prática com Arduino UNO Q, usando Arduino App Lab, Edge Impulse e Arduino Cloud. Laboratórios 0 a 15. Edição revisada e validada em hardware (App Lab 0.10.0, Bricks 0.12.0).

Sumário

Introdução.....	1
Laboratório 0: Preparando-se	9
Laboratório 1: Seu primeiro app e os fundamentos do App Lab	20
Laboratório 2: Bricks do App Lab e IA em uma foto	29
Laboratório 3: Os dois mundos, Linux e o MCU	37
Laboratório 4: Comunicação entre processadores	45
Laboratório 5: Laço de controle entre domínios.....	53
Laboratório 6: Sensores Modulino e painel na nuvem	60
Laboratório 7: Noções básicas de câmera e captura de imagem	70
Laboratório 8: O custo do modelo: tamanho, resolução e latência	79
Laboratório 9: Aplicação de visão em tempo real	88
Laboratório 10: Entendendo o tamanho e as restrições do modelo	99
Laboratório 11: Treine o seu modelo.....	105
Laboratório 12: Quantização, encolhendo o modelo.....	117
Laboratório 13: Implantando e medindo seu sistema	123
Laboratório 14: Construindo um agente de borda.....	130
Laboratório 15: Benchmarking e confiabilidade.....	137
Apêndice A: Referência rápida	144

Introdução

Sobre este manual

Este manual guia você por dezesseis laboratórios práticos, da configuração da placa até a implantação e o benchmark de um sistema de Edge AI. Se você já fez algum curso de programação ou de circuitos, tem base suficiente para começar. Aprendizado de máquina e sistemas embarcados são introduzidos do zero. Ao final,

you will build systems that run AI models in real hardware, not in simulation or in the cloud.

What is Edge AI and why it matters

Every time you unlock your phone with your face, a neural network runs on the processor of the device itself, not in a data center. This is Edge AI, or AI at the edge: inference of machine learning executed on the device that collected the data.

For engineers, this matters for three reasons: latency, privacy and availability.

- A drone that detects obstacles needs to react in tens of milliseconds. A round trip to the cloud takes 300 to 500 ms, and this when the network is available.
- A medical wearable should not send raw health data over a public network. Processing on the device reduces the risk of privacy.
- An industrial inspection camera cannot stop because the connection failed. With local inference, the network becomes a “nice-to-have” for reports, not a requirement to operate.

The Arduino UNO Q runs AI models directly on the board. Along the labs you will measure in practice the cost of this: some models respond in less than 30 ms; others, larger, take hundreds of milliseconds. Knowing how to choose is part of the job.

About the Arduino UNO Q 4GB

The Arduino UNO Q is not a typical Arduino board. It combines a Linux computer complete with a microcontroller in real time, on the same board, each doing what it does best.

- **MPU (Microprocessor Unit):** a Qualcomm QRB2210 quad-core Cortex-A53 at 2.0 GHz running Debian Linux, with 4 GB of RAM LPDDR4 (the system has about 3.6 GiB), GPU Adreno 702 and DSP Hexagon. It is where their Python code, AI models and the network run.
- **MCU (Microcontroller Unit):** an STM32U585 Cortex-M33 at 160 MHz running Zephyr RTOS, an operating system in real time, made for timing precision and direct access to pins. It is the “traditional Arduino” side of the board.

The two communicate via the **Arduino Bridge**: Python on the MPU calls functions written in the sketch on the MCU by name, and the sketch can send data back. You do not need to deal with a serial protocol between the chips.

Four terms appear throughout the manual:

Termo	Significado
MPU , ou lado Linux	O processador Qualcomm e tudo o que roda nele. “MPU”, “lado Linux” e “processador Linux” são a mesma coisa.
Sketch	O programa em C++ que roda no microcontrolador. É o nome que a Arduino dá a um programa de MCU.
Brick	Um bloco pronto do App Lab que você adiciona ao seu app: uma interface web, um modelo de IA, a conexão com a nuvem.
Container	Um ambiente Linux isolado, com as bibliotecas já instaladas, onde o seu Python e os modelos de IA rodam dentro da placa.

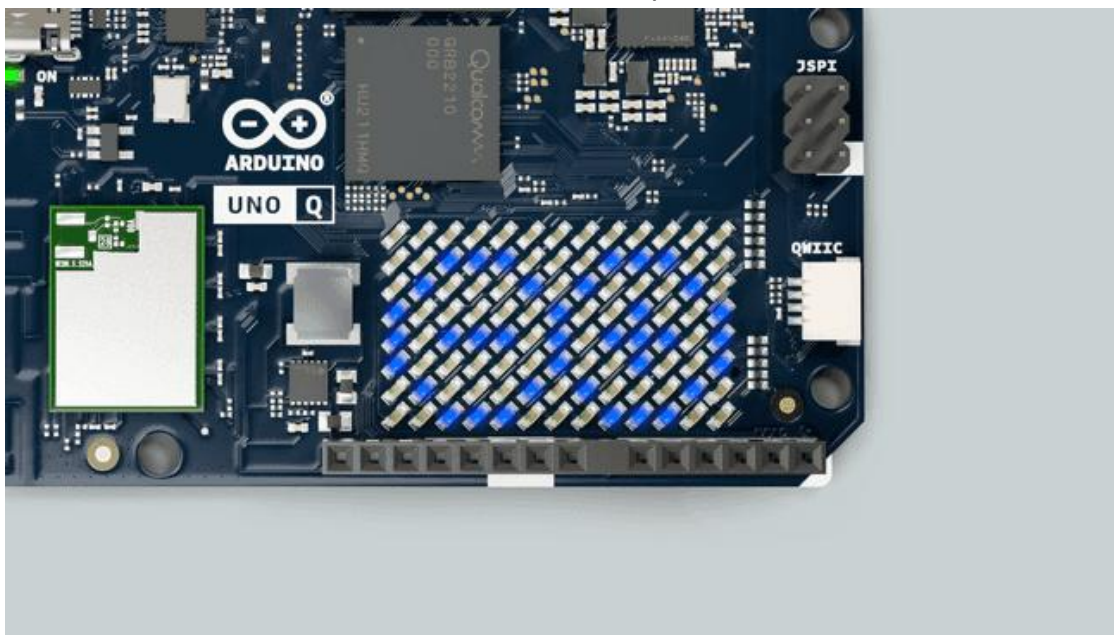


Figura I.1: A placa Arduino UNO Q 4GB

Aviso: a placa tem **uma única porta USB-C**. Ela serve para alimentação, para a conexão com o computador e para periféricos USB, mas não para tudo ao mesmo tempo. Nos laboratórios com webcam você vai usar um hub USB-C com Power Delivery e acessar a placa por Wi-Fi. Isso está explicado no Laboratório 0 e na tabela de equipamento abaixo.

O padrão Sentir-Pensar-Agir

Este manual apresenta um padrão de projeto que aparece em todo sistema embarcado inteligente:

- **SENTIR:** capturar dados do mundo real com sensores (câmera, microfone, temperatura, distância, aceleração).
- **PENSAR:** processar esses dados com um modelo de aprendizado de máquina para extrair significado (detectar objetos, classificar sons, prever anomalias).
- **AGIR:** fazer algo com o resultado (acender um LED, tocar um alarme, mover um motor, enviar uma mensagem).

Os laboratórios constroem esse ciclo progressivamente: primeiro cada etapa isolada, depois integradas, depois otimizadas para as restrições reais do hardware.

Visão geral do curso

O manual contém 16 laboratórios (Lab 0 a Lab 15), organizados por tema:

Laboratórios	Tema
0	Preparação: desembalar, instalar o software, verificar a configuração
1-2	Fundamentos: App Lab, Bricks e o primeiro modelo de IA, rodando sobre uma foto
3-6	Arquitetura e controle: os dois processadores, a comunicação entre eles, laços de controle e sensores com painel na nuvem
7-9	Visão em tempo real: câmera, custo dos modelos de visão e uma aplicação de detecção contínua
10-13	Modelos e implantação: tamanho de modelos, treinamento do seu modelo, quantização e medição do sistema completo
14-15	Agentes e confiabilidade: agente de borda com memória, benchmark e testes de falha

Os Laboratórios 0 a 6 não usam câmera. Do Laboratório 7 em diante é preciso uma webcam USB ligada por um hub; no Laboratório 9 um celular pode fazer o papel da câmera, e o Laboratório 10 não usa câmera.

Segurança e boas práticas

Aviso: o Arduino UNO Q é um computador real, com consumo real de energia. Use uma fonte USB-C de qualidade, com pelo menos 3 A. Não use cabos USB “só carga” para conectar ao computador: eles alimentam a

placa, mas não transmitem dados. Não cubra a placa durante cargas pesadas de ML. Ela aquece; nos testes deste manual a CPU ficou em torno de 40 °C em repouso e 50 °C rodando um modelo de detecção de objetos. Se a temperatura subir demais, o processador reduz o desempenho sozinho.

Dica: mantenha a placa e o computador na mesma rede Wi-Fi. Use uma senha forte para o usuário Linux. Não rode `sudo apt upgrade` genérico na placa sem orientação: a atualização de kernel e drivers pode afetar câmera e Bridge. Para atualizar os componentes Arduino, use o App Lab (Settings → System). Também existe um comando para isso no terminal da placa, listado no Apêndice A.

Como usar este manual

Cada laboratório mapeia capítulos do livro *Fundamentos de Edge AI* e reforça os conceitos com exercícios práticos no UNO Q 4GB.

Estrutura dos laboratórios

Todos os laboratórios usam a mesma estrutura:

Seção	Propósito
Objetivo	O que o lab entrega ao final
O que você vai aprender	3 a 5 habilidades específicas
Materiais necessários	Hardware, software e contas
Visão geral do conceito	Resumo ligado aos capítulos do livro
Procedimento	Passos numerados, agrupados em partes
Perguntas de checkpoint	Verificações de compreensão
Principais lições	Resumo dos conceitos
Extensão opcional	Desafios extras, em geral com o app IoT Remote
O que entregar	Entregáveis esperados
Solução de problemas	Problemas comuns e como resolver
Notas	Espaço para anotações

Ferramentas que você vai usar

Ferramenta	Propósito	Onde conseguir
Arduino App Lab	Ambiente de desenvolvimento principal (Python na MPU, sketch no MCU, Bricks)	arduino.cc/en/software ; também vem instalado na própria placa, para quem a usa como computador, com monitor e teclado (modo SBC)

Ferramenta	Propósito	Onde conseguir
Terminal (SSH ou ADB)	Acesso à linha de comando Linux da placa, pela rede (SSH) ou pelo cabo USB (ADB). O Laboratório 0 ensina os dois.	O SSH já vem no Windows 10+, macOS e Linux; o ADB é instalado no Laboratório 0
Edge Impulse Studio	Coleta de dados, treinamento e exportação de modelos	studio.edgeimpulse.com (conta gratuita)
Arduino Cloud	Dashboards IoT e controle remoto	cloud.arduino.cc (nível gratuito)
App Arduino IoT Remote	Celular como câmera e como painel móvel	App Store / Google Play (opcional)

O Arduino IDE 2.x **não** é necessário: o App Lab compila e grava o sketch do MCU diretamente na placa.

Material de apoio: o repositório e o kit de apps

Este manual, os slides dos laboratórios e um kit de apps prontos ficam em um repositório público: <https://github.com/joaodullius/workshop-edge-ai-uno-q>.

O **kit** traz, para cada laboratório que tem código de app, um app pronto: o ponto de partida ou o resultado final (gabarito) do laboratório. Um instalador copia esses apps para a placa, e eles aparecem em **My Apps**, no App Lab, com nomes que começam por “WS”, como **WS Lab 1 Blink**. O código de cada um é o mesmo que este manual manda escrever. Em um workshop, o instrutor costuma deixar o kit instalado nas placas; quando o manual disser para criar um app ou copiar um exemplo, você pode abrir o app “WS” do mesmo laboratório. O repositório também traz duas fotos de teste para o Laboratório 2 (fotos-de-teste/) e os scripts dos Laboratórios 10 e 15 (scripts/).

Aviso: não copie código deste PDF. As linhas longas são quebradas na página, e o código colado pode dar `SyntaxError` ou um comando pela metade. Todos os blocos de código do manual estão na pasta `codigo/` do repositório, um arquivo por laboratório, na mesma ordem e prontos para copiar. Os apps do kit (apps/) trazem os mesmos arquivos já montados.

Ícones e caixas

Dica: atalhos, boas práticas e explicações extras.

Aviso: riscos para o hardware, perda de dados ou erros frequentes.

Exemplo resolvido: um cálculo ou raciocínio completo, com números medidos nesta placa.

Cada lab termina com **Principais lições**, um resumo em poucos tópicos.

Resumo do equipamento

O hardware abaixo cobre todo o manual. Cada laboratório lista exatamente o que precisa.

Item	Obrigatório / opcional	Usado em
Arduino UNO Q 4GB (com antena)	Obrigatório	Todos
Cabo USB-C com dados	Obrigatório	Todos
Fonte USB-C 5 V / 3 A	Obrigatório	Todos
Computador Windows 10+, macOS 11+ ou Ubuntu 22.04+	Obrigatório	Todos
Hub USB-C com Power Delivery e pelo menos uma porta USB-A	Obrigatório nos labs com webcam	7, 8, 9, 11, 12, 13, 14, 15
Webcam USB (UVC, a maioria é)	Obrigatório nos labs de visão ao vivo	7, 8, 9, 11, 12, 13, 14, 15 (no 9 e no 11 o celular pode substituir)
Potenciômetro (10 kΩ)	Recomendado	4, 5
Botão (push-button)	Recomendado	3
Jumpers e protoboard	Recomendado	3, 4, 5
Servomotor SG90 ou similar	Opcional	5
Sensores Modulino (Thermo, Distance, Pixels) com cabos Qwiic	Obrigatório no 6	6
Smartphone	Recomendado no 11, para fotografar as amostras de treinamento; pode substituir a webcam no 9; opcional nos demais, com o app Arduino IoT Remote	1, 5, 6, 7, 9, 11, 13
Dongle USB-C com HDMI + monitor + teclado + mouse	Opcional (modo SBC)	0
Dissipador de calor passivo	Recomendado nos testes sustentados	13, 15

Dica: um único hub USB-C com Power Delivery, HDMI e portas USB-A cobre o modo SBC, a webcam e a alimentação ao mesmo tempo. É o acessório mais útil para este curso.

Laboratório 0: Preparando-se

Duração	Dificuldade	Pré-requisitos
45-60 min	Iniciante	Nenhum

Objetivo

Desembalar o Arduino UNO Q 4GB, instalar o software necessário, criar as contas e verificar que a placa se comunica com o computador de duas formas: pelo cabo USB e pela rede Wi-Fi.

O que você vai aprender

1. Configurar a placa pela primeira vez com o Arduino App Lab (nome, senha e Wi-Fi).
2. Instalar o App Lab e o ADB no computador.
3. Criar contas no Arduino e no Edge Impulse.
4. Acessar o terminal Linux da placa por USB (ADB) e por Wi-Fi (SSH).
5. Instalar o conector do Arduino Cloud na placa.

Materiais necessários

Item	Detalhes
Arduino UNO Q 4GB	Com antena e acessórios incluídos
Cabo USB-C	Com dados, não só carga
Computador	Windows 10+, macOS 11+ ou Ubuntu 22.04+
Conexão com a internet	Para downloads e criação de contas
Navegador	Chrome recomendado
Dongle USB-C com HDMI, monitor, teclado e mouse	Opcional, só para o modo SBC

Visão geral do conceito

O UNO Q tem dois lados. Um roda Debian Linux em um Qualcomm QRB2210: é onde ficam seu código Python, os modelos de IA e a rede. O outro roda Zephyr em um STM32U585: é onde ficam sensores, motores e tudo que precisa de temporização precisa. A Arduino Bridge liga os dois.

Há três formas de conectar a placa, e você vai usar todas ao longo do curso:

Modo	Como	Quando usar
Desktop (USB)	Cabo USB-C do computador à placa	Primeira configuração e labs sem periféricos USB

Modo	Como	Quando usar
Rede (Wi-Fi)	App Lab ou SSH pela rede local	Sempre que a porta USB-C estiver ocupada por um hub com webcam (a partir do Laboratório 7)
SBC	Monitor, teclado e mouse via dongle USB-C	Usar a placa como um computador independente

Você não escolhe o modo em um menu. O App Lab procura a placa pelo cabo USB e pela rede e mostra o que encontrou; a barra inferior da janela traz o nome da placa conectada. Com a placa no cabo e o Wi-Fi configurado, os dois caminhos funcionam ao mesmo tempo.

Este laboratório é só de configuração. Nenhum código é escrito; o objetivo é garantir que a cadeia de hardware, software e contas está funcionando.

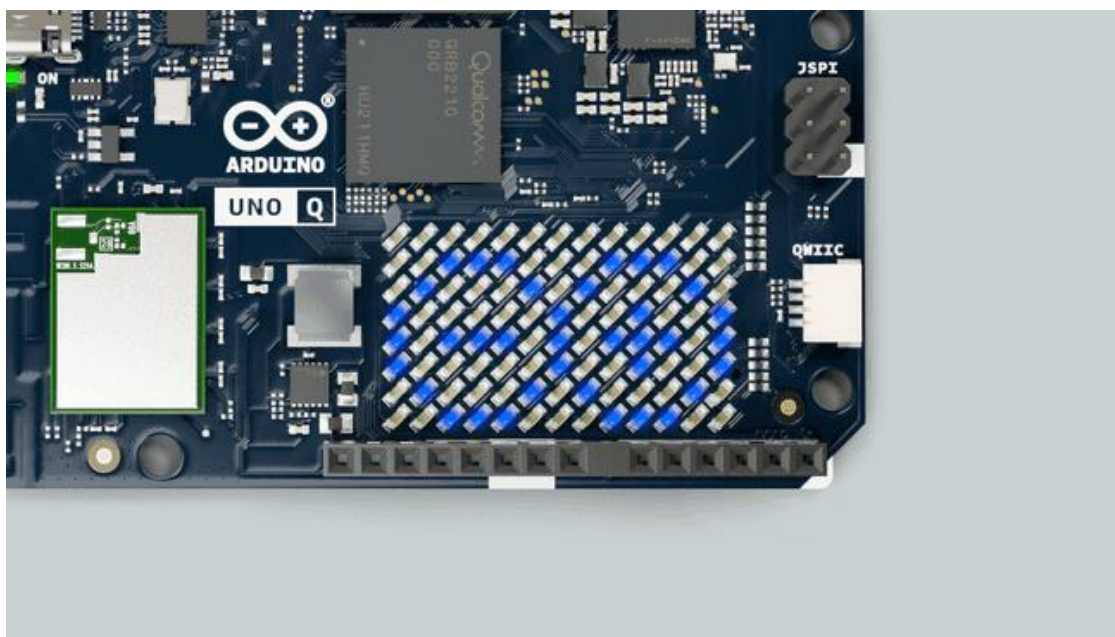


Figura 0.1: O Arduino UNO Q 4GB, vista superior

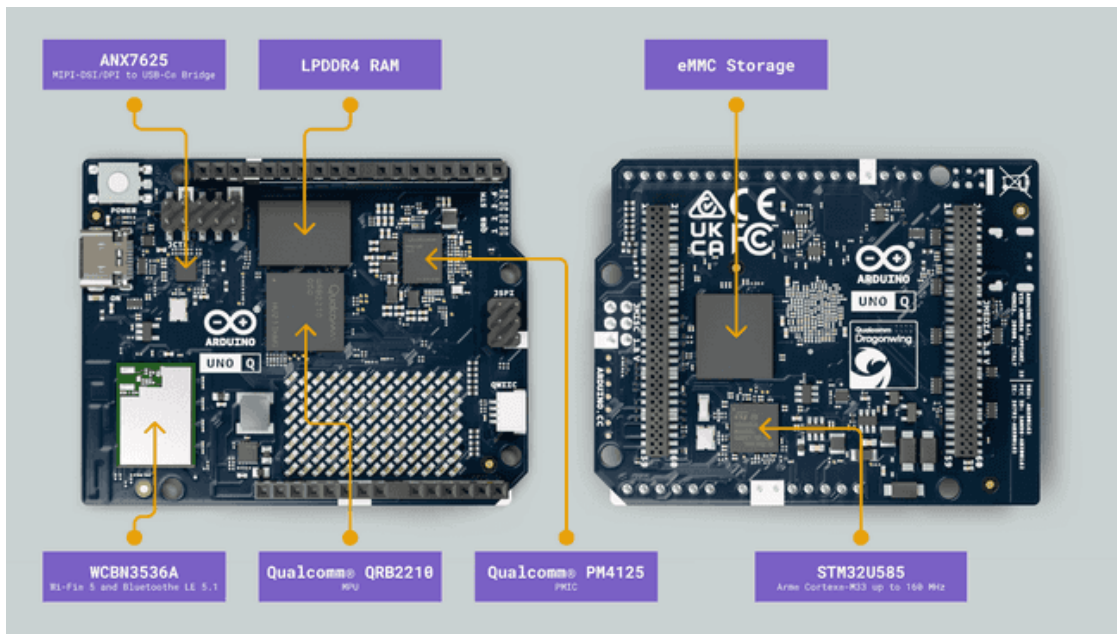


Figura 0.2: Disposição dos componentes do UNO Q

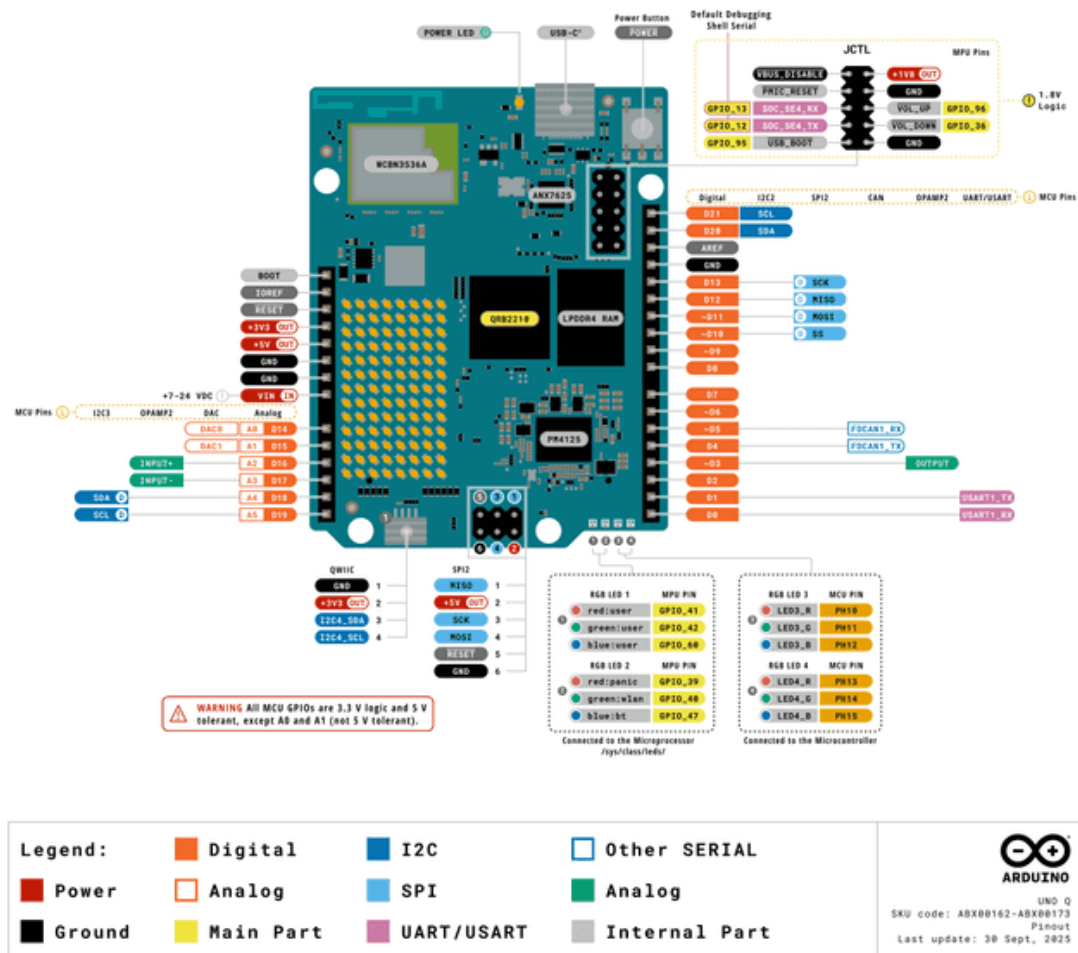


Figura 0.3: Referência de pinagem do UNO Q

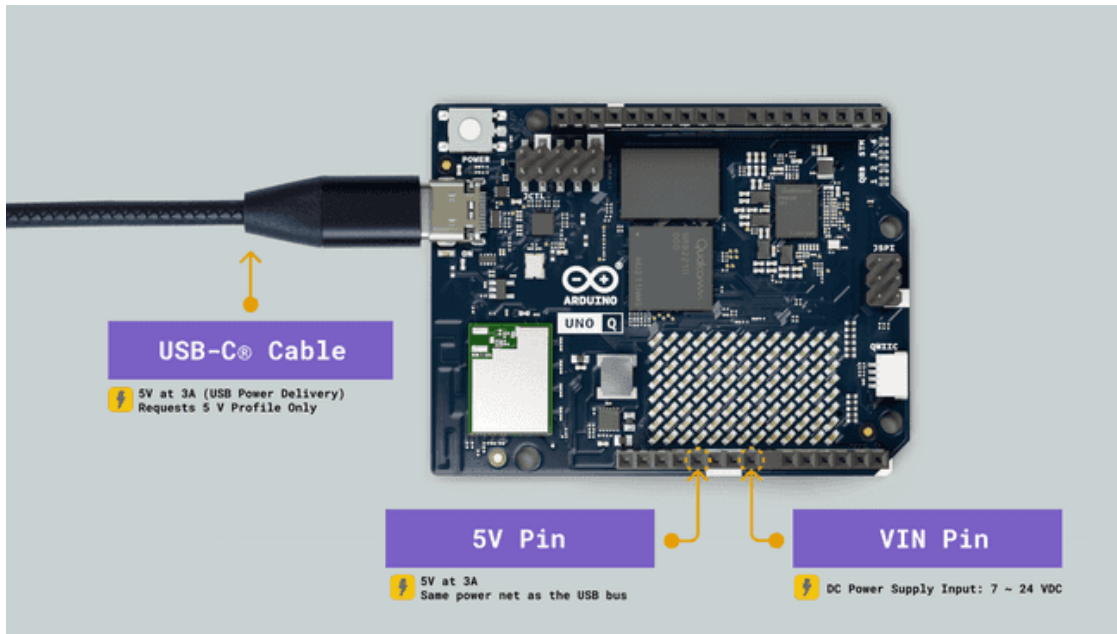


Figura 0.4: Porta de conexão USB-C

Procedimento

Parte A: Crie suas contas

Passo 1: Crie uma conta Arduino. Registre uma conta gratuita em auth.arduino.cc e confirme o e-mail. Essa conta cobre o Arduino Cloud e o login no App Lab.

Passo 2: Crie uma conta no Edge Impulse. Em studio.edgeimpulse.com, use a opção de cadastro. O Edge Impulse cuida do treinamento dos modelos e da exportação para o UNO Q.

Passo 3: Instale o app Arduino IoT Remote (opcional). Procure “Arduino IoT Remote” na App Store ou no Google Play e entre com a conta Arduino. O app transforma o celular em câmera sem fio e em painel móvel para a placa.

Parte B: Instale o Arduino App Lab

Passo 4: Baixe o App Lab. Em arduino.cc/en/software, baixe o Arduino App Lab para o seu sistema operacional.

Passo 5: Instale.

- macOS: abra o .dmg e arraste o App Lab para Aplicativos.
- Windows: execute o instalador .exe.
- Linux (Ubuntu): extraia o .tar.gz e instale a dependência:

```
sudo apt install libwebkit2gtk-4.1-0
```

Dica: o App Lab também vem pré-instalado na própria placa. Com monitor, teclado e mouse ligados a um dongle USB-C (modo SBC) você desenvolve sem instalar nada no computador.

Parte C: Conecte o UNO Q

Passo 6: Ligue a placa ao computador. Conecte o cabo USB-C na placa e no computador. Espere cerca de 20 segundos: o lado Linux faz um boot completo, como um mini laptop, e demora mais que um microcontrolador comum.

Aviso: muitos cabos USB-C são só de carga. Se a placa liga mas não aparece no App Lab, troque o cabo antes de qualquer outra coisa.

Passo 7: Abra o App Lab. Ele detecta o UNO Q automaticamente pela USB e mostra a placa para você selecionar. Se aparecer uma janela de atualização (“Checking for updates”), deixe terminar: o App Lab está atualizando a si mesmo ou os componentes da placa.

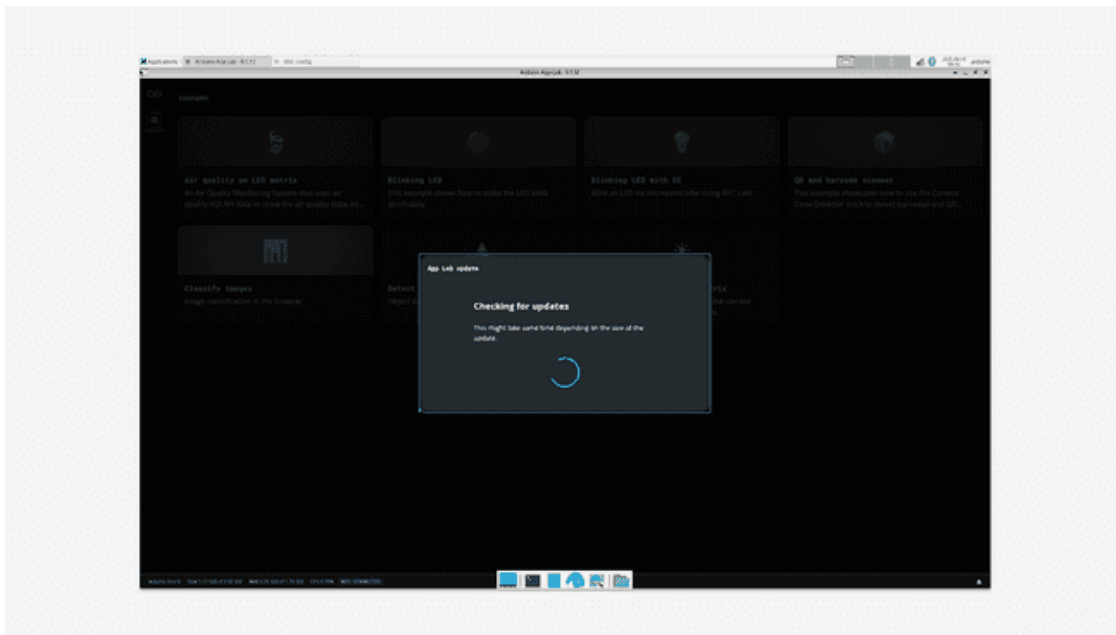


Figura 0.5: Ao abrir, o App Lab pode verificar atualizações

Passo 8: Complete a configuração inicial. O assistente pede:

- o nome da placa (por exemplo myunoq), que vira o nome de rede;
- uma senha **nova**, criada por você, para o usuário Linux arduino. Não existe senha de fábrica. Guarde essa senha: ela será pedida no SSH e em comandos com sudo, e quem a esquece precisa regravar a placa;
- a rede Wi-Fi (SSID e senha);
- opcionalmente, o login na conta Arduino.

Dica: o nome da placa vira o endereço na rede. Se você a chamou de `myunoq`, ela responde em `myunoq.local` em qualquer dispositivo da mesma rede. Configure o Wi-Fi agora, mesmo que vá trabalhar por USB: nos labs com câmera o acesso será obrigatoriamente pela rede.

Parte D: Verifique o acesso por USB (ADB)

O ADB (Android Debug Bridge) abre um terminal no Linux da placa através do cabo USB. Um terminal é uma interface de texto: você digita um comando, aperta Enter e lê a resposta.

Passo 9: Instale o ADB.

- macOS: `brew install android-platform-tools`
- Windows (PowerShell): `winget install Google.PlatformTools`. Aceite os termos se o `winget` perguntar e, ao final, abra um terminal novo. Se o comando `winget` não existir, instale o “Instalador de Aplicativo” pela Microsoft Store.
- Linux: `sudo apt-get install android-sdk-platform-tools`

Passo 10: Teste a conexão.

```
adb devices
adb shell
```

O primeiro comando lista a placa com um número de série. O segundo abre o shell dentro dela. Verifique o sistema:

```
uname -a           # Linux ... aarch64
cat /etc/os-release # Debian GNU/Linux 13 (trixie)
free -h            # cerca de 3,6 GiB de RAM
df -h              # armazenamento interno (eMMC): / com ~10 GB e /home/arduino com ~18 GB
exit               # volta ao computador
```

Repare na coluna `Avail` do `df -h`: a partição `/` tem cerca de 10 GB, mas a maior parte já está ocupada pelo sistema. Os containers dos Bricks, que você vai baixar a partir do Laboratório 2, também ficam nela.

Aviso: o ADB só enxerga a placa quando ela está ligada ao computador por USB. Com a placa em Wi-Fi, `adb devices` retorna uma lista vazia, sem mensagem de erro. Nesse caso use SSH (Parte E).



Figura 0.6: Conexão ADB, listagem de dispositivos

Parte E: Verifique o acesso por Wi-Fi (SSH)

Passo 11: Conecte por SSH. No terminal do computador:

```
ssh arduino@<nome-da-placa>.local
```

Na primeira vez o SSH pede para confirmar a identidade da placa (responda `yes`) e depois a senha do usuário `arduino`. No Windows 11, macOS e Linux o nome `.local` costuma resolver sem instalar nada.

A placa pode continuar ligada ao computador pelo cabo USB durante este teste. O SSH usa a rede Wi-Fi, não o cabo.

Se o nome não resolver, use o endereço IP. Descubra o IP do Wi-Fi pelo ADB:

```
adb shell ip -4 addr show wlan0
```

O endereço é o número depois de `inet`, por exemplo `192.168.15.14`. Conecte com `ssh arduino@192.168.15.14`. Não use `hostname -I` para isso: ele lista também dezenas de endereços internos dos containers, e o do Wi-Fi fica perdido no meio.

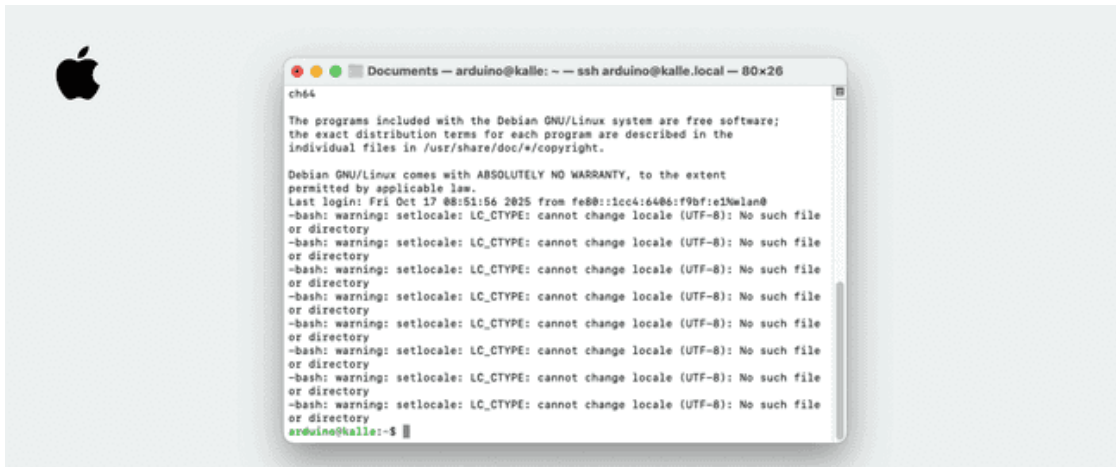


Figura 0.7: Conexão SSH com o UNO Q via Wi-Fi

Parte F: Instale o conector do Arduino Cloud

Os Laboratórios 1 e 6 usam o Arduino Cloud. A placa precisa de um serviço chamado `arduino-cloud-connector`, que pode não vir instalado na imagem de fábrica. Instale agora, no shell da placa (ADB ou SSH). A placa precisa estar com o Wi-Fi funcionando, porque o pacote é baixado da internet. O `sudo` pede a senha do usuário `arduino`; ao digitar, nada aparece na tela, e isso é normal.

```
sudo apt update && sudo apt install -y arduino-cloud-connector
systemctl status arduino-cloud-connector --no-pager | head -3
```

O `apt update` só atualiza a lista de pacotes disponíveis. O `apt install` instala o pacote indicado. A última linha da saída deve mostrar `active (running)`. Se não mostrar, confira se a placa está com internet e repita o comando de instalação. Depois feche e reabra o App Lab: em **Settings** aparece a seção **Arduino Cloud** com o botão **Connect device**. A conexão em si você fará no Laboratório 1.

Aviso: O `apt update` vai avisar que há muitos pacotes atualizáveis no Debian. Não rode `sudo apt upgrade`, que atualizaria todos eles, incluindo o kernel e os drivers da câmera. Atualize só os componentes Arduino, pelo App Lab (Settings → System).

Perguntas de checkpoint

P1: Quais são os dois processadores do UNO Q e qual sistema operacional cada um roda?

P2: Por que o UNO Q leva cerca de 20 segundos para iniciar, enquanto um Arduino tradicional liga na hora?

P3: Qual é a diferença entre conectar por USB (ADB) e por Wi-Fi (SSH)? Em que situação a USB não estará disponível para o computador?

Principais lições

- O UNO Q combina uma MPU Linux com um MCU em tempo real; o App Lab e a Bridge escondem a complexidade dessa dupla.
- Há três modos de conexão: USB (ADB e App Lab em modo Desktop), Wi-Fi (SSH e App Lab em modo Rede) e SBC. Como a porta USB-C é única, os labs com câmera usam Wi-Fi.
- Cabo com dados, Wi-Fi configurado e conector do Cloud instalado agora evitam horas de depuração depois.

O que entregar

Não há entrega formal. Mostre ao instrutor que você consegue:

1. Abrir o App Lab e ver o UNO Q conectado.
2. Abrir o terminal da placa por ADB, com o cabo USB ligado, e por SSH, pela rede.
3. Entrar no Edge Impulse e no Arduino Cloud.

Solução de problemas

Problema	Solução
Placa não aparece no App Lab	Troque o cabo USB-C por um com dados. Espere 20 segundos após conectar.
<code>adb devices</code> não lista nada	A placa está no USB do computador? Com a placa em Wi-Fi o ADB não funciona. No Linux, podem ser necessárias regras udev (veja a documentação do UNO Q).
<code>ssh: Could not resolve hostname</code>	Use o IP da placa (<code>adb shell ip -4 addr show wlan0</code> , número depois de <code>inet</code>). Confirme que placa e computador estão na mesma rede.
Conexão SSH recusada	O Wi-Fi foi configurado no assistente? Reabra o assistente pelo App Lab.
Esqueci a senha da placa	Regrave a imagem da placa seguindo o guia oficial de flashing.
App Lab trava no Linux	Instale <code>libwebkit2gtk-4.1-0</code> .
A seção Arduino Cloud não aparece em Settings	Instale o <code>arduino-cloud-connector</code> (Parte F) e reabra o App Lab.

Notas

Espaço para suas anotações.

Laboratório 1: Seu primeiro app e os fundamentos do App Lab

Duração	Dificuldade	Pré-requisitos
60-90 min	Iniciante	Laboratório 0 concluído

Objetivo

Rodar seu primeiro programa no UNO Q com o Arduino App Lab, entender como a MPU e o MCU trabalham juntos no exemplo do LED piscante, conhecer a estrutura de um app e controlar o LED remotamente pelo Arduino Cloud.

O que você vai aprender

1. Abrir o App Lab, conectar à placa e rodar um exemplo.
2. Entender a estrutura de um app: `app.yaml`, `python/main.py`, `sketch/sketch.ino`, `assets/`.
3. Ler o código do Blink e identificar o que roda na MPU e o que roda no MCU.
4. Gerenciar apps pela linha de comando com `arduino-app-cli`.
5. Conectar a placa ao Arduino Cloud e controlar o LED por um dashboard.

Materiais necessários

Item	Detalhes
Arduino UNO Q 4GB	Configurado no Laboratório 0
Cabo USB-C com dados	Ou acesso por Wi-Fi. A Parte D exige a placa no Wi-Fi, com internet.
Computador com App Lab	Instalado no Laboratório 0
Conta Arduino Cloud	Nível gratuito (Parte D)
Smartphone com IoT Remote	Opcional (extensão)

Visão geral do conceito

O Arduino App Lab é o ambiente de desenvolvimento principal do UNO Q. Diferente do Arduino IDE tradicional, que só grava um sketch em um microcontrolador, o App Lab gerencia um **app** com duas metades:

- `python/main.py` roda no lado Linux (MPU), dentro de um container. É onde ficam a IA, a rede e a lógica da aplicação.
- `sketch/sketch.ino` roda no STM32 (MCU). É onde ficam pinos, sensores e temporização.

O App Lab compila o sketch na própria placa e o grava no MCU; você não precisa de outra ferramenta. A **Arduino Bridge** liga as duas metades: o Python chama funções do sketch pelo nome, e o sketch pode enviar dados para o Python.

O App Lab funciona em três modos de conexão, apresentados no Laboratório 0: Desktop (USB), Rede (Wi-Fi) e SBC.

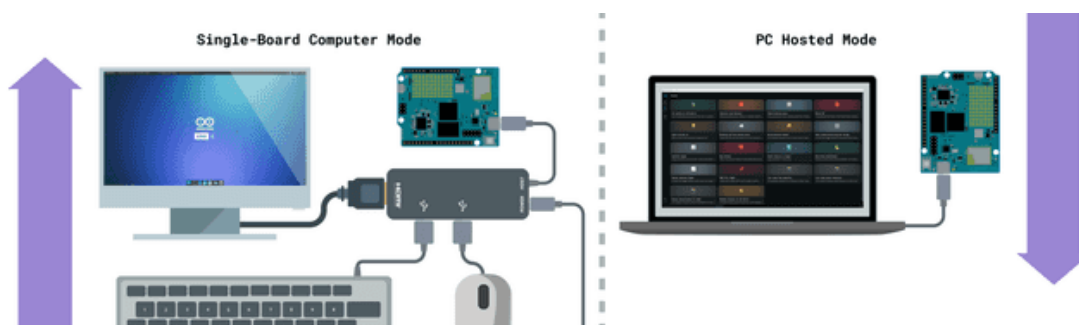


Figura 1.1: Modos de conexão do UNO Q

Procedimento

Parte A: Rodando o exemplo Blink

Passo 1: Abra o App Lab. Ele mostra as placas que encontrou pelo cabo e pela rede; clique na sua. Quando a barra inferior da janela mostrar o nome da placa, você está conectado.

Passo 2: Na barra lateral esquerda, sob *Learn and Explore*, abra **Examples** e procure o cartão **Blink LED from Python**. Existe um exemplo com o mesmo nome em **Inspirations**; os dois são iguais. Com o exemplo aberto, clique em **Copy and edit app**, no canto superior direito, e dê um nome à cópia, por exemplo `meu-blink`. Os exemplos originais não podem ser editados; você sempre trabalha em uma cópia.


```

void loop() {
}

void set_led_state(bool state) {
    digitalWrite(LED_BUILTIN, state ? LOW : HIGH);    // LED_BUILTIN acende em LOW
}

```

Leia o fluxo: o Python decide quando alternar o LED, a Bridge leva o comando ao MCU, e o MCU muda o pino. O `loop()` do sketch fica vazio porque a função só roda quando o Python a chama.

A primeira linha do Python, `from arduino.app_utils import *`, importa tudo o que o App Lab oferece, incluindo App e Bridge. Outros exemplos importam só o que usam, com `from arduino.app_utils import App, Bridge`. As duas formas funcionam.

Dica: `App.run(user_loop=loop)` é o coração de todo app. Ele inicia os componentes do app, chama a sua função `loop` repetidamente e mantém o app vivo até você apertar Stop. Nunca escreva um `while True` antes de `App.run()`: o app nunca chegaria a iniciar.

Passo 4: Clique em **Run**, no canto superior direito. O App Lab compila o sketch, grava no MCU, cria o container do Python e inicia. Isso leva de 1 a 2 minutos, e mais na primeira vez; o console mostra o progresso, e o app só está no ar quando aparece App started. O LED RGB da placa identificado como LED3 pisca em vermelho a cada segundo. Enquanto o app roda, o botão Run vira **Stop**; clique nele quando terminar.

Depois de ver o LED piscar, experimente mudar o comportamento sem tocar no sketch: em `python/main.py`, troque `time.sleep(1)` por `time.sleep(0.2)` e clique em **Run** de novo. Como o sketch não mudou, a partida leva cerca de 10 segundos, e o LED passa a piscar cinco vezes mais rápido.

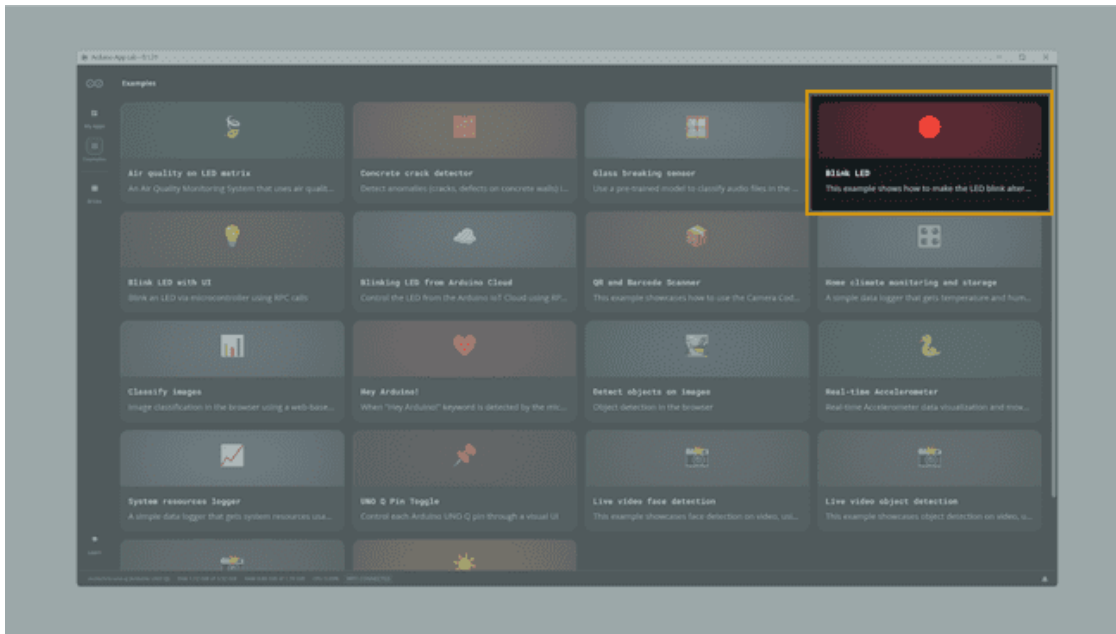


Figura 1.3: Painel do App Lab com os exemplos (captura de uma versão anterior, em que todos apareciam na mesma tela; hoje parte deles fica em Inspirations)

Parte B: Entendendo a estrutura do app

Passo 5: Explore os arquivos no painel **Files**, à esquerda. Um app tem esta estrutura:

```
meu-app/
  app.yaml           # nome, icone e lista de Bricks
  python/main.py     # aplicacao Python (roda na MPU)
  sketch/sketch.ino  # codigo C++ do MCU
  sketch/sketch.yaml # plataforma e bibliotecas do sketch (gerado pelo App Lab)
  assets/            # so em apps com interface web: HTML, CSS, JS
  README.md          # so em alguns apps
```

O `app.yaml` e o `python/main.py` existem sempre. A pasta `sketch/` só existe em apps que usam o MCU, e quando existe precisa dos dois arquivos, `sketch.ino` e `sketch.yaml`; os exemplos de interface web e de IA que você vai copiar no Laboratório 2 vêm sem ela. A pasta `assets/` só aparece em apps com interface web.

O `app.yaml` do Blink é curto:

```
name: Blink LED from Python
icon: 🟡
description: This example shows how to make the LED blink alternately from Python.
```

Em um app com Bricks, o arquivo ganha a lista `bricks:`. Um app criado do zero traz ainda linhas de comentário, começadas por `#`, e os campos `ports:` `[]` e `bricks:` `[]` vazios. O campo `icon` aceita um emoji; uma letra ou palavra impede o app de iniciar. O comentário que o próprio arquivo traz diz “an emoji or a short string”, mas uma letra é recusada.

A lista `bricks` diz quais componentes prontos o app usa. Você vai conhecê-los no Laboratório 2.

Parte C: Usando a linha de comando

Passo 6: Abra o terminal da placa (ADB ou SSH, como no Laboratório 0).

Passo 7: Use o `arduino-app-cli`, a ferramenta que o próprio App Lab usa por baixo:

```
arduino-app-cli app list # Lista apps e exemplos
arduino-app-cli app start examples:core-and-foundational/01-led-blink/03-blinking-an-led-from-python
arduino-app-cli app logs examples:core-and-foundational/01-led-blink/03-blinking-an-led-from-python --all # traz tambem a saida de execucoes anteriores
arduino-app-cli app stop examples:core-and-foundational/01-led-blink/03-blinking-an-led-from-python
```

Os IDs dos apps têm a forma `examples:<caminho>` para exemplos e `user:<nome>` para os seus. A sua cópia do Blink aparece em `app list` como `user:meu-blink`, e os mesmos comandos funcionam com ela. O `start` leva de 1 a 2 minutos, como o botão Run.

Passo 8: Crie um app pela linha de comando:

```
arduino-app-cli app new "my-first-app"
ls /home/arduino/ArduinoApps/my-first-app
```

Os apps do usuário ficam em `/home/arduino/ArduinoApps/`. O app novo aparece no App Lab, em **Apps**, e vem com um `main.py` que imprime `Hello world!`. O mesmo se faz pela interface: o botão **+** ao lado de **Apps**, na barra lateral, cria um app vazio. Se o nome já existir na placa, escolha outro.

Dica: cada app que você roda deixa na placa um container parado e uma rede interna, mesmo depois do Stop. Com cerca de 30 apps acumulados, a placa fica sem redes disponíveis e nenhum app novo inicia. Quando terminar de usar um app, libere esses recursos com `arduino-app-cli app destroy user:<nome>`. Apesar do nome, o comando não apaga os seus arquivos: remove só o container, a rede e o cache, e o app volta a funcionar no próximo Run.

Parte D: Controlando o LED pelo Arduino Cloud

Nesta parte a placa passa a ser controlada por um dashboard na nuvem. Não é preciso copiar credenciais: a placa é registrada diretamente pelo App Lab. A placa precisa estar no Wi-Fi, com acesso à internet.

Passo 9: Conecte a placa ao Cloud. No App Lab, abra **Settings**, role até **Arduino Cloud** e clique em **Connect device**. O ícone de engrenagem que abre Settings fica no canto inferior esquerdo da janela. Entre com a conta Arduino se for pedido e escolha o espaço da Cloud; um espaço é a área da sua conta onde ficam os dispositivos, e

contas pessoais têm um só. Espere o status mudar para **Connected**. A partir daí a placa aparece em cloud.arduino.cc → Devices, como ONLINE.

Dica: se ao rodar um app com o Brick Arduino Cloud aparecer o diálogo “Provisioning required”, clique em **Connect device** ali mesmo: é o mesmo fluxo. Se a seção Arduino Cloud não existir em Settings, volte à Parte F do Laboratório 0.

Passo 10: Crie a Thing e o dashboard. Uma Thing é a representação da sua placa na nuvem, com as variáveis que ela troca. Em cloud.arduino.cc, com a mesma conta:

1. No menu lateral, abra **Things** e crie uma Thing (por exemplo unoq-blink).
2. Na página da Thing, no quadro *Associated Device*, escolha a sua placa.
3. Ainda na página da Thing, adicione uma variável. O formulário pede o nome, o tipo, a permissão e a política de atualização: use exatamente `led`, Boolean, Read & Write e On change.
4. No menu lateral, abra **Dashboards**, crie um dashboard, adicione um widget **Switch** e ligue-o à variável `led`.

Passo 11: Rode o exemplo. No App Lab, abra o exemplo **Blinking LED from Arduino Cloud**, que fica em **Inspirations**, na barra lateral. Faça **Copy and edit app** e clique em **Run**. O Python do exemplo é:

```
from arduino.app_bricks.arduino_cloud import ArduinoCloud
from arduino.app_utils import App, Bridge

# A conexao com o Cloud e feita pelo servico da placa; nenhuma credencial aqui
iot_cloud = ArduinoCloud()

def led_callback(client: object, value: bool):
    # Chamado sempre que a variavel "Led" muda na nuvem
    print(f"LED blink value updated from cloud: {value}")
    Bridge.call("set_led_state", value) # repassa ao sketch, que controla o pino

iot_cloud.register("led", value=False, on_write=led_callback)

App.run()
```

O sketch é o mesmo do Blink: `Bridge.provide("set_led_state", set_led_state)`.

Passo 12: Teste. Abra o dashboard no navegador e alterne o switch. O LED da placa deve acompanhar, e o console do App Lab (**Open console**, abaixo do editor) mostra LED blink value updated from cloud: True / False.

Dica: o caminho dos dados: Dashboard → Arduino Cloud → serviço arduino-cloud-connector na placa → Brick ArduinoCloud no Python → `Bridge.call` → sketch → LED. Logo após criar a Thing, o serviço pode levar de um a alguns minutos para associá-la; o console avisa no thing assigned yet enquanto isso. Se

demorar, `sudo systemctl restart arduino-cloud-connector` no terminal da placa acelera; o comando pede a senha do usuário `arduino`.



Figura 1.4: O caminho de um comando, do dashboard ao LED

Perguntas de checkpoint

P1: No Blink, qual processador realmente muda o estado do LED? Como o Python faz isso acontecer?

P2: Para que serve o `app.yaml`? O que muda nele quando você adiciona um Brick?

P3: Cite duas maneiras de iniciar um app no UNO Q.

P4: Por que `LED_BUILTIN` acende com `LOW` neste sketch? O que aconteceria se você trocasse por `HIGH`?

Principais lições

- Um app do App Lab tem duas metades: Python na MPU (`python/main.py`) e sketch no MCU (`sketch/sketch.ino`). O App Lab compila e grava o sketch sozinho.
- `App.run(user_loop=...)` mantém o app vivo; `Bridge.call` leva comandos do Python ao MCU; `Bridge.provide` expõe funções do sketch.
- Você pode operar apps pela interface gráfica, pela linha de comando (`arduino-app-cli`) e remotamente pelo Arduino Cloud, sem copiar credenciais.

Extensão opcional

Se instalou o app Arduino IoT Remote no celular, entre com a mesma conta Arduino. O dashboard com o switch aparece no app; controle o LED pelo celular.

O que entregar

1. Uma captura de tela do Blink rodando no App Lab.
2. Uma captura de tela do dashboard do Arduino Cloud controlando o LED.
3. Respostas escritas às perguntas de checkpoint.

Solução de problemas

Problema	Solução
LED não pisca	Confirme que clicou em Run e que a compilação terminou sem erro. Veja o console. O app só está no ar depois de App started.
O Run parece travado	A primeira execução leva de 1 a 2 minutos, ou mais se a placa estiver baixando um container. Acompanhe o console.
icon ... is not a valid single emoji	O campo icon do app.yaml precisa de um emoji, ou ficar vazio.
could not find an available, non-overlapping IPv4 address pool	A placa acumulou redes de apps antigos. Libere com <code>arduino-app-cli app destroy user:<nome></code> para cada app que não está em uso. Os arquivos dos apps são preservados.
arduino-app-cli: command not found	Você está no terminal do computador. Entre na placa com <code>adb shell</code> ou SSH primeiro.
App Lab mostra “Board disconnected”	Reconecte o cabo USB, espere 20 segundos, atualize o App Lab.
Diálogo “Provisioning required”	Clique em Connect device e complete o fluxo de Settings → Arduino Cloud.
Console repete no thing assigned yet	Crie a Thing associada à placa (Passo 10) e aguarde; se passar de alguns minutos, reinicie o serviço <code>arduino-cloud-connector</code> .
Switch do dashboard não muda o LED	O nome da variável na Thing precisa ser exatamente <code>led</code> . Confira o Wi-Fi da placa.

Notas

Espaço para suas anotações.

Laboratório 2: Bricks do App Lab e IA em uma foto

Duração	Dificuldade	Pré-requisitos
60-90 min	Iniciante	Laboratório 1 concluído

Objetivo

Entender os Bricks como blocos reutilizáveis do App Lab, usar o Brick Web UI para servir uma página do navegador diretamente da placa e rodar o primeiro modelo de IA da placa sobre uma foto enviada pelo navegador, sem câmera.

O que você vai aprender

1. Explicar o que é um Brick e como ele se conecta ao app pelo `app.yaml`.
2. Adicionar o Brick Web UI e servir uma página HTML na porta 7000.
3. Consultar a API de um Brick (exemplo: Object Detection).
4. Detectar objetos em uma foto e medir o tempo que o modelo leva.
5. Entender por que os Bricks rodam em containers Docker.

Materiais necessários

Item	Detalhes
Arduino UNO Q 4GB	Por USB ou Wi-Fi
Computador com App Lab	
Uma ou duas fotos	JPG ou PNG de até 500 KB cada, com pessoas ou objetos comuns. Qualquer foto do seu computador serve

Visão geral do conceito

Bricks são a abstração central do App Lab: pacotes Python prontos que embrulham funcionalidades complexas (modelos de IA, servidores web, conexão com a nuvem) atrás de APIs simples. Você adiciona um Brick com um clique, ele aparece no `app.yaml`, e você o importa no Python.

Há três famílias:

- **Interfaces:** Web UI (HTML), Web UI (Streamlit), Arduino Cloud.
- **Modelos:** Video Object Detection, Video Image Classification, Audio Classification, Visual Anomaly Detection, Keyword Spotting, entre outros.
- **Serviços:** bancos de dados locais, MQTT, Telegram, LLMs locais e na nuvem.

Alguns Bricks que você vai usar neste manual:

Brick (nome no <code>app.yaml</code>)	O que faz
--	-----------

Brick (nome no app.yaml)	O que faz
arduino:web_ui	Serve uma página web na porta 7000 e troca mensagens com o navegador
arduino:video_object_detection	Detecta objetos em vídeo de câmera com um modelo pré-treinado ou seu
arduino:video_image_classification	Classifica cada quadro de vídeo (uma classe por quadro)
arduino:object_detection	Detecta objetos em uma imagem avulsa
arduino:arduino_cloud	Sincroniza variáveis com o Arduino Cloud
arduino:motion_detection	Classifica movimento a partir de um acelerômetro
arduino:visual_anomaly_detection	Detecta anomalias visuais (rachaduras, defeitos)

Sensores Modulino são outra história: eles são lidos pelo **MCU**, com a biblioteca `Arduino_Modulino` no sketch, e as leituras chegam ao Python pela Bridge. Não há Brick nem pacote Python para Modulino. O Laboratório 6 trata deles.

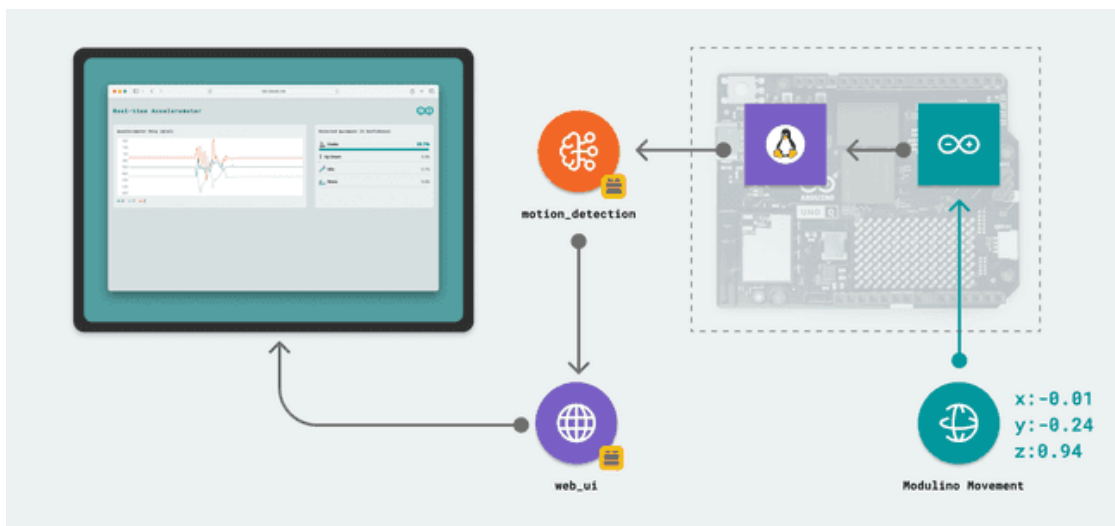


Figura 2.1: Bricks, visão geral da arquitetura

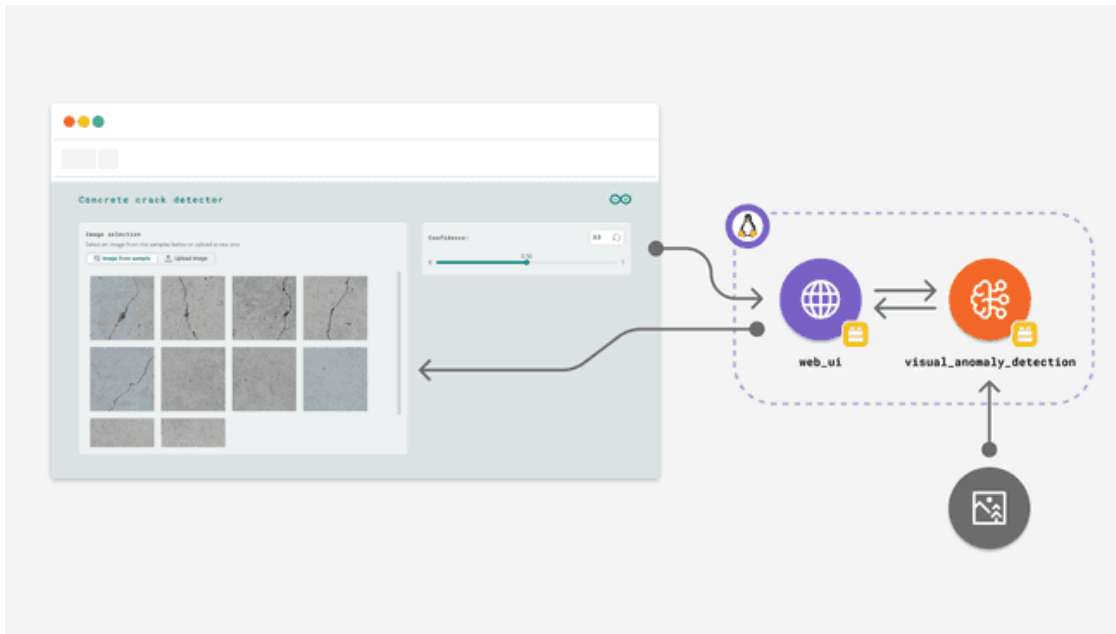


Figura 2.2: UNO Q conectado por Wi-Fi à Web UI e à nuvem

Procedimento

Parte A: Adicionando e usando um Brick

Passo 1: Na barra lateral, abra **Examples** e procure **Simple web page**. Clique em **Copy and edit app** e chame a cópia de brick-explorer. Partir do exemplo é o caminho mais curto: ele já traz o Brick Web UI e a pasta `assets/`, que um app criado do zero não tem.

Passo 2: Veja o painel **Bricks**, à esquerda: ele lista **WebUI - HTML**. Abra o `app.yaml` e confira a linha correspondente:

```
bricks:  
  - arduino:web_ui
```

O App Lab pode gravar a linha como `- arduino:web_ui: {}`; é a mesma coisa. O mesmo Brick tem três nomes: “WebUI - HTML” na interface, `arduino:web_ui` no `app.yaml` e `WebUI` no Python. Em um app criado do zero, você adicionaria o Brick pelo primeiro dos três ícones acima do painel Bricks.

Passo 3: Abra `python/main.py`. O essencial dele é isto; se quiser, substitua todo o conteúdo do arquivo por estas linhas:

```
from arduino.app_utils import App  
from arduino.app_bricks.web_ui import WebUI
```

```
# Serve os arquivos da pasta assets/ em http://<nome-da-placa>.local:7000  
web_ui = WebUI()
```

```
App.run()
```

Passo 4: Abra `assets/index.html` e substitua todo o conteúdo por:

```
<!DOCTYPE html>
<html>
<head><title>My First Brick App</title></head>
<body>
<h1>Hello from UNO Q!</h1>
<p>This web page is served directly from your Arduino board.</p>
<p>Hora no seu navegador: <span id="time"></span></p>
<script>
setInterval(() => {
    document.getElementById('time').textContent = new Date().toLocaleTimeString();
}, 1000);
</script>
</body>
</html>
```

O HTML e o texto fixo vêm da placa. O relógio é um trecho de JavaScript que roda no seu navegador.

Passo 5: Clique em **Run**. O console mostra as URLs (Local URL: `http://localhost:7000` e Network URL: `http://<ip>:7000`). Abra `http://<nome-da-placa>.local:7000` no navegador do computador: a página vem da placa.

Aviso: o Brick confere a pasta `assets/` quando o app inicia. Se o `assets/index.html` não existir nesse momento, o console mostra `RuntimeError: 'index.html' is required but was not found in /app/assets`, o app aparece como iniciado mesmo assim, e a página não abre. Criar o arquivo depois não resolve: pare o app e rode de novo. Já as mudanças em um `index.html` que existia na partida aparecem ao recarregar a página.

Dica: a Web UI faz mais que servir arquivos. Com `web_ui.send_message("evento", dados)` o Python empurra dados para a página, com `web_ui.on_message("evento", funcao)` a página envia comandos ao Python, e `web_ui.expose_api("GET", "/rota", funcao)` cria uma API REST. Os exemplos **Data transmission to Web UI** e **Button Action**, em **Examples**, mostram cada um.

Parte B: Explorando a API de um Brick

Passo 6: Na barra lateral, sob *Bricks Manager*, abra **Bricks**. A lista mostra todos os Bricks instalados; você não precisa adicioná-los a um app para ler sobre eles. Clique em **Object Detection**: a página traz a descrição, um README com exemplo de uso e a documentação da API. O método principal é `detect()`. Leia este trecho, que resume o uso:

```
from arduino.app_bricks.object_detection import ObjectDetection

detector = ObjectDetection() # Limiar de confiança padrao: 0.3

with open("assets/image.jpg", "rb") as f:
    img = f.read()
```

```

out = detector.detect(img) # aceita bytes JPEG/PNG ou uma imagem PIL
# out = {"detection": [{"class_name": "person", "confidence": "87.30",
#                       "bounding_box_xyxy": [x1, y1, x2, y2]}, ...]}
for det in out.get("detection", []):
    print(det["class_name"], det["confidence"], det["bounding_box_xyxy"])

```

Neste Brick a confiança vem em porcentagem, de 0 a 100, e como texto ("87.30"): para comparar com um número, converta antes, com `float(det["confidence"])`. Os Bricks de vídeo, que você usa a partir do Laboratório 8, devolvem a confiança de 0 a 1 e entregam os resultados por callback, que é uma função sua chamada pelo Brick a cada resultado.

Parte C: IA em uma foto

Agora veja o Brick funcionando. Este é o primeiro modelo de IA que você roda na placa, e ele não precisa de câmera: a foto sai do seu computador, pelo navegador.

Passo 7: Na barra lateral, abra **Inspirations**, escolha o exemplo **Detect objects on images**, faça **Copy and edit app** com o nome `detect-foto` e clique em **Run**. Na primeira vez que uma placa usa este Brick ela baixa o container do modelo, de quase 1 GB, o que leva de 5 a 10 minutos; nas seguintes, o app inicia em menos de meio minuto (medido: 12 segundos).

Passo 8: Quando o console mostrar as URLs, abra `http://<nome-da-placa>.local:7000`. A página pede uma imagem JPG ou PNG de até 500 KB. Escolha uma foto com pessoas ou objetos comuns (uma mesa de trabalho, uma sala, uma rua) e clique em **Run Detection**. A foto volta com uma caixa em volta de cada objeto encontrado, com o nome da classe e a confiança, e a página mostra “Detection completed successfully!”.

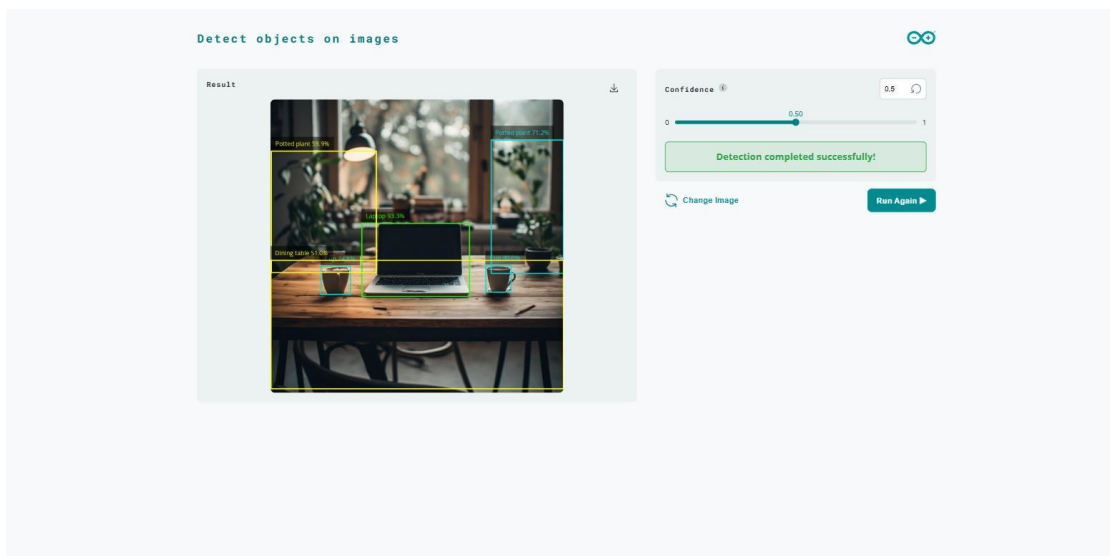


Figura 2.3: Página do exemplo “Detect objects on images” com o resultado da detecção em uma foto

Na foto de referência, uma mesa com notebook (é a fotos-de-teste/sem-pessoa.jpg do repositório), o modelo encontrou o notebook com 93% de confiança, dois vasos de planta com 71% e 60%, duas xícaras perto de 80% e a mesa com 51%.

Passo 9: Mexa no controle **Confidence** e clique em **Run Again**. Com o limiar em 0,8 sobram só as detecções mais seguras; em 0,2 aparecem caixas duvidosas. É o mesmo limiar de confiança que você vai ajustar em todos os laboratórios de visão. Use **Change Image** para testar outra foto.

Passo 10: Meça quanto tempo a placa leva para pensar. Abra `python/main.py` e localize as duas chamadas que fazem o trabalho: `object_detection.detect(...)`, que roda o modelo, e `object_detection.draw_bounding_boxes(...)`, que desenha as caixas. Logo depois da chamada a `detect`, o exemplo calcula o tempo gasto, na linha `diff = time.time() * 1000 - start_time`, mas não mostra o valor em lugar nenhum. Acrescente estas duas linhas logo abaixo dela, com o mesmo recuo (oito espaços):

```
n_obj = len(results.get("detection", [])) if results else 0
print(f"Deteccao: {n_obj} objetos em {diff:.0f} ms")
```

Clique em Stop e em Run, recarregue a página no navegador, envie a foto de novo e leia o console do App Lab. A linha aparece no meio das mensagens do runner do modelo. Saída de referência:

Deteccao: 6 objetos em 1149 ms

Pouco mais de um segundo por foto. Guarde esse número: é o custo do passo PENSAR com este modelo, e os Laboratórios 8 e 9 vão mostrar por que ele decide o que dá para fazer em tempo real.

Parte D: Bricks e containers Docker

Por baixo, cada app roda em um container Docker na placa, e Bricks de IA sobem containers próprios (um runner de modelos, por exemplo). Isso traz três benefícios:

- cada Brick tem suas próprias dependências, sem conflito com as suas;
- Bricks podem ser atualizados de forma independente;
- vários Bricks rodam ao mesmo tempo, isolados.

Ao adicionar um Brick de IA pela primeira vez, o App Lab pode baixar uma imagem de container de centenas de megabytes. Isso acontece uma vez por tipo de Brick.

Aviso: a partição do sistema da placa tem cerca de 10 GB, e as imagens de container ocupam mais de 2 GB. Se faltar espaço, `arduino-app-cli system cleanup` remove imagens que nenhum app está usando; se você precisar delas de novo, a placa baixa outra vez. O Python do terminal da placa não enxerga as bibliotecas dos Bricks: elas só existem dentro do container do app.

Perguntas de checkpoint

P1: O que é um Brick no App Lab e em que ele difere de uma biblioteca Python comum?

P2: Onde você encontra a documentação da API de cada Brick?

P3: Se quisesse adicionar detecção de objetos em vídeo ao seu app, qual Brick adicionaria e como receberia os resultados?

P4: Quanto tempo a placa levou para detectar os objetos da sua foto? Esse modelo conseguiria analisar 10 fotos por segundo?

Principais lições

- Bricks são pacotes reutilizáveis que adicionam interface, IA e serviços ao app com um clique; o `app.yaml` registra quais estão em uso.
- O Brick Web UI serve `assets/index.html` na porta 7000 e permite trocar mensagens entre a página e o Python.
- Bricks e apps rodam em containers Docker isolados; o Python do terminal da placa não os enxerga.
- O modelo de detecção roda inteiro na placa: a foto vai, as caixas voltam, e nada sai da rede local. Cada foto custa cerca de um segundo com o modelo padrão.

O que entregar

1. Uma captura de tela da sua Web UI no navegador.

2. Seu `python/main.py` do app `brick-explorer`.
3. Uma captura de tela da página de detecção com as caixas sobre a sua foto, e a linha do console com o tempo.

Solução de problemas

Problema	Solução
Web UI não abre	O app está rodando? O console mostrou as URLs? Tente pelo IP mostrado na Network URL.
A página não abre, ou responde <code>{"detail": "Not Found"}</code> OU <code>Internal Server Error</code> ; o console mostra <code>RuntimeError: 'index.html' is required</code>	Falta o <code>assets/index.html</code> , ou ele foi criado ou apagado com o app rodando. Confira o arquivo, pare o app e rode de novo.
Falha ao importar o Brick	Confira se ele está na lista <code>bricks</code> : do <code>app.yaml</code> .
A página não reage ao envio da foto, ou o botão não aparece	Recarregue a página; se continuar, abra o endereço em uma aba nova.
A página de detecção recusa a foto	O exemplo aceita JPG ou PNG de até 500 KB. Reduza a foto ou escolha outra.
A linha <code>Deteccao: ...</code> não aparece no console	Confira o recuo das duas linhas novas (oito espaços, igual ao da linha <code>diff = ...</code>) e se você parou e rodou o app de novo.
Já existe um app com o nome que o manual pede	Em placa compartilhada, acrescente suas iniciais ao nome.
Brick demora para carregar na primeira vez	Download de imagem de container. Espere de 5 a 10 minutos.

Notas

Espaço para suas anotações.

Laboratório 3: Os dois mundos, Linux e o MCU

Duração	Dificuldade	Pré-requisitos
60-90 min	Intermediário	Laboratório 1 concluído

Objetivo

Explorar a arquitetura de dois processadores do UNO Q: um app Python rodando no lado Linux (MPU) e um sketch rodando no microcontrolador (MCU), ao mesmo tempo. Entender o que pertence a cada lado, como o App Lab compila e grava o sketch, e como o Python comanda o MCU pela Bridge.

O que você vai aprender

1. Diferenciar o MPU Qualcomm e o MCU STM32 em hardware, sistema operacional e uso.
2. Explorar o ambiente Debian Linux do MPU e entender onde o seu app roda de fato.
3. Editar, compilar e gravar um sketch no MCU a partir do App Lab.
4. Usar a Bridge para chamar uma função do sketch a partir do Python.
5. Decidir quais tarefas pertencem a cada processador, com números de latência reais.

Materiais necessários

Item	Detalhes
Arduino UNO Q 4GB	Conectado por USB-C ao computador (modo Desktop) ou por Wi-Fi
Computador com Arduino App Lab	Instalado no Laboratório 0
Cabo USB-C com dados	Para o modo Desktop e o terminal ADB
Botão momentâneo (push-button) e resistor de 10 kΩ	Um terminal do botão no pino 3V3, o outro no pino D2; o resistor entre D2 e GND. Sem botão, use um jumper: veja o Passo 7.
Protoboard e jumpers	Para o botão

Visão geral do conceito

Até aqui você usou o UNO Q como uma caixa preta. Agora você abre a caixa. Sistemas embarcados industriais, de controladores CNC a equipamentos médicos, quase sempre combinam um processador de aplicação para lógica e IA com um microcontrolador dedicado para o hardware em tempo real. O UNO Q traz os dois na mesma placa:

	Qualcomm QRB2210 (MPU)	STM32U585 (MCU)
Função	Processador de aplicação	Controlador em tempo real
CPU	Quad-core Cortex-A53 a 2,0 GHz	Cortex-M33 a 160 MHz
Memória	4 GB LPDDR4 (3,6 GiB visíveis ao Linux)	O chip tem 786 KB de RAM e 2 MB de flash. Para o sketch ficam 768 KB de flash e 256 KB de RAM: são os dois <code>Maximum is</code> que a compilação mostra, 786432 e 262144 bytes.
Sistema	Debian Linux 13	Zephyr RTOS (sistema operacional de tempo real, feito para temporização precisa, não para computação geral)
Linguagem	Python (e qualquer ferramenta Linux)	C/C++ (sketch Arduino)
Melhor para	Inferência, rede, servidores web, lógica complexa	Leitura de sensores, controle de motores, temporização exata
Como você acessa	App Lab, SSH, ADB	Pelo App Lab (sketch) e pela Bridge (a partir do Python)

Regra simples: o pensamento pesado vai para o MPU (IA, web, arquivos). Tudo que precisa de temporização de frações de milissegundo vai para o MCU (ler sensor, controlar motor, piscar LED em intervalo exato). A Bridge é o elo entre eles.

Um detalhe que muda como você depura: o seu `python/main.py` não roda no Python “solto” do Linux. O App Lab o executa dentro de um **container Docker** que já traz as bibliotecas `arduino.app_utils`, `arduino.app_bricks`, `numpy` e `cv2`. O Python que você encontra pelo SSH não tem nada disso. Por isso, tudo que envolve Bricks ou Bridge é escrito como app do App Lab, não como script avulso.

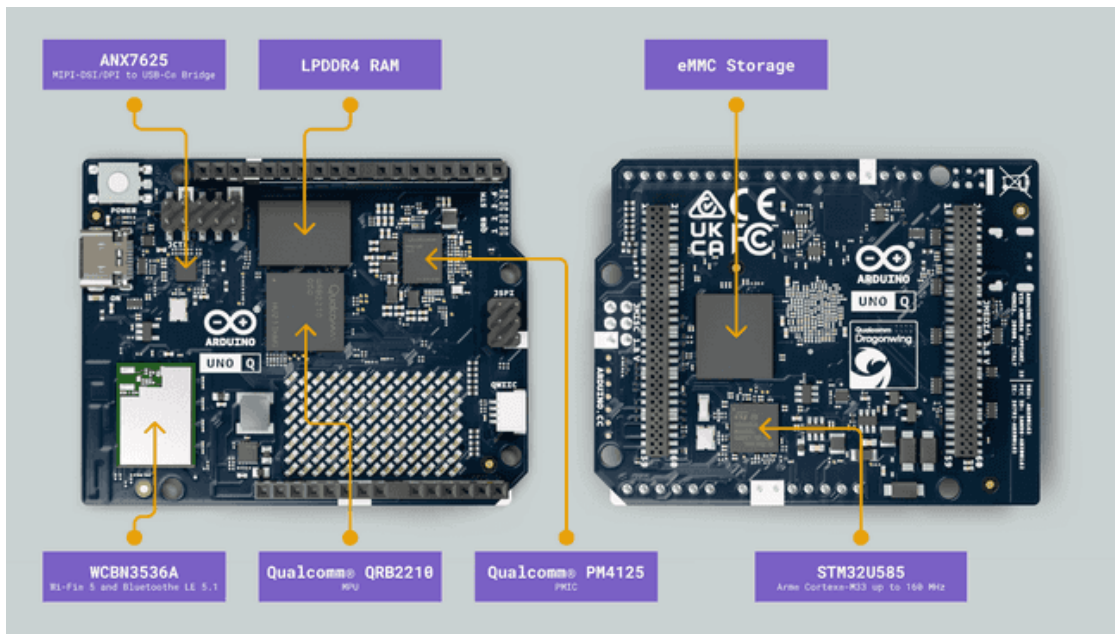


Figura 3.1: Arquitetura de dois processadores do UNO Q

Procedimento

Parte A: Explorar o lado Linux

Passo 1: abra um terminal na placa. Com a placa no USB do computador, use `adb shell`. Pela rede, use `ssh arduino@<nome-da-placa>.local`.

Passo 2: execute os comandos abaixo e compare com a tabela acima.

```
uname -a                # kernel 6.16, arquitetura aarch64
cat /etc/os-release      # Debian 13 (trixie)
nproc                   # 4 núcleos
free -h                 # 3,6 GiB de RAM e 1,8 GiB de swap
ls /home/arduino/ArduinoApps/ # seus apps
ls /var/lib/arduino-app-cli/examples/ # exemplos do App Lab
ps aux | grep -E "python|docker" | head # processos dos apps e do Docker
```

Passo 3: confirme onde as bibliotecas dos apps vivem.

```
python3 --version        # Python do host
pip list 2>/dev/null | grep -i arduino # não imprime nada: o host nem tem o pip
docker images            # python-apps-base e ei-models-runner
```

O resultado vazio é esperado. O Linux da placa não tem as bibliotecas do App Lab, nem mesmo o gerenciador `pip`. Elas estão na imagem `python-apps-base`, e cada app roda em um container criado a partir dela.

Parte B: Editar o sketch no App Lab

O sketch do MCU faz parte do app: fica em `sketch/sketch.ino`. O App Lab compila o sketch na própria placa e grava o binário no STM32 por uma ligação interna entre os

dois processadores, usando um programa chamado OpenOCD. Não há Arduino IDE nem cabo extra envolvidos.

Passo 4: no App Lab, crie um app vazio chamado `mcu-basics`, pelo botão **+** ao lado de **Apps**, na barra lateral.

Monte o botão antes de rodar. Ligue um terminal do botão ao pino 3V3 e o outro ao pino D2. Ligue o resistor de 10 kΩ entre D2 e GND. Esse resistor, chamado de pull-down, mantém o pino em nível baixo enquanto o botão está solto; sem ele o pino fica flutuando e lê valores aleatórios.

Passo 5: no painel **Files**, abra a pasta `sketch` e o arquivo `sketch.ino`. Substitua todo o conteúdo por este sketch, que lê um botão e acende o LED:

```
// sketch/sketch.ino - Lê o botão em D2 e reflete no LED
#include <Arduino_RouterBridge.h>

#define BUTTON_PIN D2

void setup() {
  Monitor.begin(115200);           // Log do MCU; Leia com arduino-app-cli monitor
  pinMode(LED_BUILTIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT);
  Bridge.begin();                 // obrigatório em todo sketch de app
  Monitor.println("Sketch do MCU iniciado");
}

void loop() {
  int pressed = digitalRead(BUTTON_PIN);
  // LED_BUILTIN acende com LOW no UNO Q
  digitalWrite(LED_BUILTIN, pressed ? LOW : HIGH);
  Monitor.println(pressed ? "Botao pressionado, LED ON" : "Botao solto, LED OFF");
  delay(100);                     // debounce simples
}
```

Passo 6: deixe `python/main.py` com o mínimo, só para o app subir:

```
from arduino.app_utils import App
App.run()
```

Passo 7: clique em **Run** e abra **Open console**, abaixo do editor. O console mostra a compilação (`Sketch uses ... bytes`), a gravação pelo OpenOCD e depois o início do container Python. Linhas como `level=error msg="Stopped decode loop: EOF"` aparecem em toda execução e não indicam problema.

As mensagens do `Monitor.println` vêm do microcontrolador e não aparecem nesse console. Para lê-las, abra um terminal na placa e rode:

```
arduino-app-cli monitor
```

O comando mostra as mensagens ao vivo e precisa de um terminal interativo; saia com `Ctrl+C`. Pressione o botão e observe o LED e as mensagens. Sem botão, encoste

um jumper entre D2 e 3V3 para simular o botão pressionado, e entre D2 e GND para simular o botão solto.

Dica: bibliotecas de sketch além das que vêm com o core (Wire, SPI, LED Matrix, CAN, RTC) são adicionadas pelo botão **Add Sketch Library**, o ícone de livro, o do meio dos três ícones acima dos painéis da esquerda. O App Lab baixa a biblioteca e a declara em `sketch/sketch.yaml`. Você vai usar isso no Laboratório 5 com a biblioteca Servo.

Parte C: Entender a Bridge

A Bridge é o canal de comunicação entre o Python no MPU e o sketch no MCU. Funciona como chamada de procedimento remoto (RPC): o sketch **expõe** funções pelo nome, e o Python as **chama** pelo nome, passando argumentos e recebendo o retorno. O caminho contrário também existe: o sketch avisa o Python com `Bridge.notify` (Laboratório 4).

Do lado do sketch, cada função exposta precisa de uma linha `Bridge.provide`. Este é um sketch completo, que você vai usar na Parte D:

```
// sketch/sketch.ino - expõe set_led ao Python
#include <Arduino_RouterBridge.h>

void set_led(bool state) {
    digitalWrite(LED_BUILTIN, state ? LOW : HIGH);
}

void setup() {
    pinMode(LED_BUILTIN, OUTPUT);
    Bridge.begin();
    Bridge.provide("set_led", set_led);    // agora o Python pode chamar "set_led"
}

void loop() {
    // pode ficar vazio: a Bridge atende as chamadas em segundo plano
}
```

O App Lab acrescenta a biblioteca da Bridge ao sketch mesmo sem a linha `#include`. Escreva a linha sempre: ela deixa claro de onde vêm `Bridge` e `Monitor`.

Do lado do Python, a chamada é uma linha. Este é só um trecho; o arquivo completo vem no Passo 9:

```
from arduino.app_utils import App, Bridge

Bridge.call("set_led", True)    # executa set_led(true) no MCU e espera a resposta
```

Dica: na maioria dos apps você não escreve a Bridge na mão, porque os Bricks já fazem isso. Mas entender o mecanismo é o que permite depurar quando “o MCU não responde”.

Parte D: Controlar o LED do MCU a partir do Python

Passo 8: no mesmo app `mcu-basics`, substitua todo o conteúdo de `sketch/sketch.ino` pelo sketch completo da Parte C, o que expõe `set_led`.

Passo 9: substitua todo o conteúdo de `python/main.py` por:

```
# python/main.py – pisca o LED do MCU a partir do Python
from arduino.app_utils import App, Bridge
import time

state = False

def loop():
    global state
    state = not state
    Bridge.call("set_led", state)
    print("LED do MCU:", "ON" if state else "OFF")
    time.sleep(1)

App.run(user_loop=loop)  # o App Lab chama loop() repetidamente
```

Passo 10: clique em Run. O LED deve piscar a cada segundo, e o console imprime o estado. O Python decide, o MCU executa.

Aviso: nunca escreva um `while True:` antes de `App.run()`. O `App.run()` é quem inicia os Bricks e mantém o app vivo; um laço infinito antes dele impede tudo isso. Use `App.run(user_loop=sua_funcao)`.

Exemplo resolvido: uma chamada `Bridge.call` custa cerca de 7 ms ida e volta (você vai medir isso no Laboratório 4). Uma câmera de segurança processando 10 quadros por segundo tem um orçamento de 100 ms por quadro. A chamada ocupa 7% desse orçamento. O gargalo quase certamente será a inferência ou a captura, não a Bridge. Saber disso direciona o esforço de otimização para o lugar certo.

Perguntas de checkpoint

P1: cite três tarefas mais adequadas ao MPU e três mais adequadas ao MCU.

P2: onde o seu `python/main.py` roda de fato, e por que o `pip list` do SSH não mostra as bibliotecas dos Bricks?

P3: por que `Bridge.begin()` sozinho não basta para o Python chamar uma função do sketch? O que falta?

P4: quando você escreveria um sketch próprio em vez de depender só dos Bricks?

Principais lições

- O UNO Q combina o processador certo com o trabalho certo: o MPU faz inferência, rede e software complexo; o MCU faz tudo que exige temporização de microssegundos ou acesso direto a pinos.
- O sketch é parte do app e é compilado e gravado pelo próprio App Lab, na placa.
- A Bridge expõe funções do sketch pelo nome; o Python as chama sem escrever protocolo serial.
- O app Python roda em container; o Python do host é só para exploração.

O que entregar

1. Uma captura de tela do console do App Lab mostrando a compilação do sketch e as mensagens do Monitor com o botão.
2. A saída do console do app da Parte D, com o LED alternando.
3. Respostas escritas para as perguntas de checkpoint.

Solução de problemas

Problema	Solução
O sketch não compila e cita uma biblioteca ausente	Adicione-a pelo botão Add Sketch Library. Só Wire, SPI, LED Matrix, CAN e RTC vêm com o core.
O LED acende invertido	O <code>LED_BUILTIN</code> do UNO Q é ativo em LOW. Use <code>state ? LOW : HIGH</code> .
<code>Bridge.call</code> falha com função não encontrada	O nome em <code>Bridge.provide("nome", fn)</code> tem que ser idêntico ao usado em <code>Bridge.call</code> . Confira se o sketch foi regravado.
O botão não responde	Confira o pull-down em D2 e a alimentação em 3,3 V. Sem resistor o pino fica flutuando.
Não vejo as mensagens do Monitor	Elas não saem no console do app. Rode <code>arduino-app-cli monitor</code> em um terminal da placa, com o app rodando.
<code>could not find an available, non-overlapping IPv4 address pool</code>	A placa acumulou redes de apps antigos. Libere com <code>arduino-app-cli app destroy user:<nome></code> para cada app que não está em uso; os arquivos ficam. Veja a dica do Laboratório 1, Parte C.
O app fica “iniciando” e nada acontece	Verifique se há um <code>while True</code> antes de <code>App.run()</code> .

Notas

Espaço para suas anotações.

Laboratório 4: Comunicação entre processadores

Duração	Dificuldade	Pré-requisitos
90 min	Intermediário	Laboratório 3 concluído

Objetivo

Usar a Bridge para trocar dados entre o processador Linux (MPU) e o microcontrolador STM32 (MCU), medir a latência dessa comunicação e entender os padrões que sustentam toda aplicação do UNO Q.

Aviso: a partir daqui o laboratório passa a envolver medição e análise, não só execução. Até aqui, na maior parte do tempo, você rodava código e observava. Agora você projeta um experimento, coleta números e tira conclusões deles. Essa é uma competência central de engenharia e ela se mantém até o fim do manual.

O que você vai aprender

1. Expor funções do sketch e chamá-las a partir do Python com `Bridge.call`.
2. Medir e analisar a latência de uma chamada pela Bridge.
3. Construir um laço de monitoramento de sensor com a Bridge.
4. Distinguir `call`, `notify` e `provide`, e saber quando a sobrecarga da Bridge é aceitável.

Materiais necessários

Item	Detalhes
Arduino UNO Q 4GB	Conectado por USB ou Wi-Fi
Computador com Arduino App Lab	
Potenciômetro de 10 kΩ (opcional)	Extremos em 3,3 V e GND, cursor em A0. Sem ele, A0 lê ruído, o que basta para medir latência
Protoboard e jumpers	Para o potenciômetro

Visão geral do conceito

No Laboratório 3 você viu os dois processadores rodando de forma independente. O poder do UNO Q está em fazê-los colaborar, e o custo dessa colaboração é o primeiro número que um engenheiro de sistemas embarcados mede, porque ele define o teto de desempenho da aplicação.

A Bridge implementa **RPC** (chamada de procedimento remoto): o Python chama uma função pelo nome, a Bridge empacota o nome e os argumentos, envia por um canal serial interno ao MCU, espera o MCU executar e devolver o resultado, e entrega o

valor ao Python. O tempo dessa ida e volta é a **latência da Bridge**. Neste laboratório você vai medi-la: fica em torno de 7 ms.

Isso é rápido o bastante para controle guiado por IA, em que a inferência custa dezenas ou centenas de milissegundos, e lento demais para tarefas que exigem microssegundos, como gerar PWM preciso. Essas tarefas ficam no MCU, e o Python só ajusta parâmetros.

Há três verbos na Bridge:

Verbo	Quem chama	O que faz
<code>Bridge.call("fn", args)</code>	Python → MCU	Executa a função e espera o retorno . O sketch também pode chamar o Python assim, mas neste manual o MCU sempre usa <code>notify</code> .
<code>Bridge.notify("fn", args)</code>	qualquer lado	Executa a função sem esperar resposta
<code>Bridge.provide("fn", fn)</code>	quem expõe	Registra uma função para o outro lado chamar

O fluxo típico de uma aplicação: o MCU lê sensores e envia ao Python (`notify`), o Python decide (às vezes com IA) e comanda atuadores no MCU (`call`).

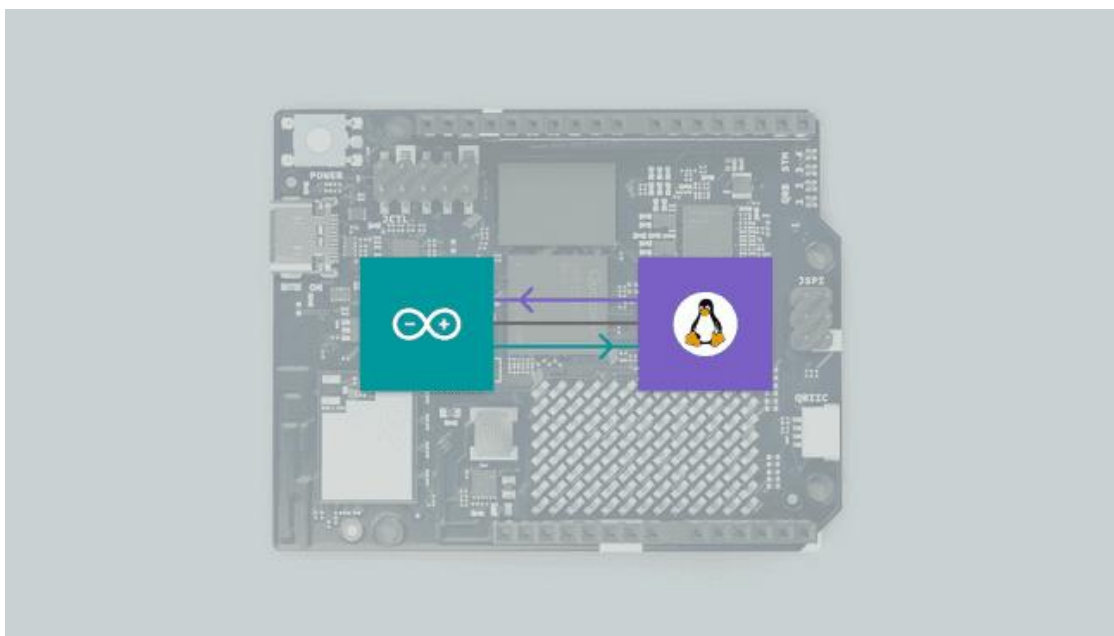


Figura 4.1: Arquitetura da Bridge entre MCU e Linux

Procedimento

Parte A: Comunicação básica pela Bridge

Passo 1: no App Lab, crie um app vazio chamado `bridge-test`, pelo botão **+** ao lado de **Apps**. Se a placa for compartilhada e o nome já existir, acrescente suas iniciais. Você vai editar dois arquivos: `sketch/sketch.ino` (MCU) e `python/main.py` (MPU).

Passo 2: substitua todo o conteúdo de `sketch/sketch.ino` por este sketch. Cada função que o Python vai chamar precisa da sua linha `Bridge.provide`. O `analogRead` devolve um número de 0 a 1023, que a função converte em volts.

```
// sketch/sketch.ino – funções expostas ao Python pela Bridge
#include <Arduino_RouterBridge.h>

float read_sensor() {
    return analogRead(A0) * 3.3 / 1024.0;    // tensão em A0, em volts
}

void set_led(bool state) {
    digitalWrite(LED_BUILTIN, state ? LOW : HIGH);    // LED_BUILTIN e ativo em LOW
}

int add_numbers(int a, int b) {
    return a + b;
}

void setup() {
    pinMode(LED_BUILTIN, OUTPUT);
    pinMode(A0, INPUT);
    Bridge.begin();                                // inicializa a Bridge
    Bridge.provide("read_sensor", read_sensor);      // expoe cada funcao pelo nome
    Bridge.provide("set_led", set_led);
    Bridge.provide("add_numbers", add_numbers);
}

void loop() {
    // vazio: a Bridge atende as chamadas em segundo plano
}
```

Passo 3: substitua todo o conteúdo de `python/main.py` por:

```
# python/main.py – chama funcoes do sketch
from arduino.app_utils import App, Bridge
import time

# Passa parametros e recebe o retorno
print("7 + 3 =", Bridge.call("add_numbers", 7, 3))

# Le o sensor
voltage = Bridge.call("read_sensor")
print(f"Tensao no sensor: {voltage:.2f} V")

# Controla o LED
Bridge.call("set_led", True)
```

```
time.sleep(1)
Bridge.call("set_led", False)
```

```
App.run()
```

Passo 4: clique em Run. O console deve mostrar $7 + 3 = 10$, a tensão lida e o LED deve acender por um segundo. Repare que `Bridge.call` funciona mesmo antes de `App.run()`: a Bridge fica pronta assim que o app inicia.

Parte B: Medir a latência da Bridge

Passo 5: acrescente a medição ao mesmo `python/main.py`. Cole o bloco abaixo logo acima da última linha, `App.run()`. O `import statistics` pode ficar onde está no bloco; se preferir, mova-o para junto dos outros, no topo.

```
import statistics

times = []
print("Medindo a latencia da Bridge (100 chamadas)...")
for _ in range(100):
    t0 = time.perf_counter()
    Bridge.call("read_sensor")
    times.append((time.perf_counter() - t0) * 1000) # em ms

print(f"Media: {statistics.mean(times):.2f} ms")
print(f"Min:    {min(times):.2f} ms")
print(f"Max:    {max(times):.2f} ms")
print(f"Desvio padrao: {statistics.stdev(times):.2f} ms")
```

Passo 6: rode e registre os quatro números. Em três medições na placa de referência a média ficou entre 6,9 e 7,3 ms, com mínimo perto de 6, máximo perto de 8 e desvio padrão de 0,4 a 0,5 ms. A variação pequena é boa notícia: a latência é previsível.

Dica: `time.perf_counter()` tem resolução maior que `time.time()` e é a escolha certa para medir intervalos curtos.

Parte C: Laço de monitoramento de sensor

Passo 7: transforme o app em um monitor contínuo. Antes, copie o `main.py` da Parte B para um arquivo à parte ou para o seu relatório, porque ele faz parte da entrega. Depois substitua todo o conteúdo de `python/main.py` pelo código abaixo. O laço vai dentro de uma função passada a `App.run`, nunca em um `while True` solto.

```
# python/main.py - monitora A0 e acende o LED acima do limiar
from arduino.app_utils import App, Bridge
import time

THRESHOLD = 1.5 # volts

def loop():
    voltage = Bridge.call("read_sensor")
    if voltage > THRESHOLD:
        Bridge.call("set_led", True)
```

```

        print(f"ALERTA: {voltage:.2f} V acima do limiar")
    else:
        Bridge.call("set_led", False)
        print(f"Normal: {voltage:.2f} V")
    time.sleep(0.5)

```

App.run(user_loop=loop)

Passo 8: rode. Gire o potenciômetro e veja o LED e o console reagirem. Sem potenciômetro, toque com o dedo o pino A0, o primeiro do conector de entradas analógicas (Figura 0.3 do Laboratório 0); o toque injeta ruído e muda a leitura. Este é o padrão central de todo app do UNO Q: o MCU sente o mundo físico, o MPU decide, o MCU age.

Parte D: O caminho inverso, MCU avisando o Python

Passo 9: a Bridge também deixa o sketch chamar o Python. É assim que os sensores Modulino entregam as leituras, no Laboratório 6. No sketch, **substitua** a função `loop()` vazia pelas linhas abaixo; um sketch só pode ter uma função `loop`. A variável `last` fica fora da função, como está no bloco:

```

unsigned long last = 0;

void loop() {
    if (millis() - last >= 1000) {
        last = millis();
        Bridge.notify("on_tick", (int)(millis() / 1000)); // avisa o Python, sem espera
    }
}

```

No `python/main.py` da Parte C, cole estas linhas logo acima da última, `App.run(user_loop=loop)`:

```

def on_tick(seconds: int):
    print(f"MCU ligado ha {seconds} s")

```

`Bridge.provide("on_tick", on_tick)` *# expoe a funcao ao sketch*

Passo 10: rode e observe as duas direções coexistindo: o Python chama `read_sensor` no laço, e o MCU chama `on_tick` no Python.

Aviso: não chame `Bridge.call` **de dentro** de uma função exposta com `Bridge.provide` (um callback RPC). A biblioteca avisa que isso pode travar a comunicação entre os dois lados. Dentro de callbacks, use `Bridge.notify` ou guarde o dado para o laço principal usar.

Parte E: Desafio, a foto que acende o LED

Você já tem as três peças do padrão Sentir-Pensar-Agir: a foto enviada pelo navegador (sentir), o modelo de detecção do Laboratório 2 (pensar) e a Bridge até o LED (agir). O desafio é juntá-las: **o LED da placa deve acender quando a foto tiver**

uma pessoa e apagar quando não tiver. O Passo 11 prepara o app, e você precisa fazê-lo de qualquer jeito. Depois dele, tente escrever sozinho as três linhas de Python antes de ler o Passo 12; não há nenhuma linha nova de sketch.

Passo 11: pare o bridge-test. O app detect-foto, que você criou no Laboratório 2 a partir de um exemplo, não tem a pasta sketch/, e sem ela não há função para o Python chamar. Um app com sketch precisa de dois arquivos nessa pasta, sketch.ino e sketch.yaml. Copie a pasta do exemplo Blink, que já traz os dois. No terminal da placa (adb shell pelo cabo, ou SSH):

```
cp -r /var/lib/arduino-app-cli/examples/core-and-foundational/01-led-blink/03-blinking-an-  
-led-from-python/sketch ~/ArduinoApps/detect-foto/  
ls ~/ArduinoApps/detect-foto/sketch
```

O ls deve mostrar sketch.ino e sketch.yaml. Se o seu app tem outro nome (com as suas iniciais, por exemplo), troque detect-foto nos dois comandos pelo nome da pasta do seu app; ls ~/ArduinoApps mostra as pastas. Com o nome errado o cp não reclama: ele cria uma pasta nova chamada detect-foto, e só o ls acusa o erro. Esse sketch expõe a função set_led_state, a mesma do Laboratório 1: recebe verdadeiro ou falso e acende ou apaga o LED. No App Lab, feche e abra o app detect-foto para a pasta aparecer no painel **Files**.

Passo 12: abra python/main.py. A resposta do modelo está na variável results, no formato que você viu no Laboratório 2: {"detection": [{"class_name": "person", "confidence": "64.88", ...}, ...]}. Localize a linha que desenha as caixas, img_with_boxes = object_detection.draw_bounding_boxes(pil_image, results), e cole estas três linhas logo acima dela, com o mesmo recuo (oito espaços):

```
        pessoa = any(d.get("class_name") == "person" for d in results.get("detection",  
[[]]))  
        Bridge.call("set_led_state", pessoa)  
        print("Pessoa na foto:", pessoa, "-> LED", "aceso" if pessoa else "apagado")
```

O arquivo já importa a Bridge, na linha from arduino.app_utils import *. Atenção ao nome da classe: a página mostra o rótulo como Person, com maiúscula, mas em class_name ele vem em minúsculas, person. Se você comparar com "Person", não aparece erro nenhum, e o LED simplesmente nunca acende.

Passo 13: clique em Run. A primeira partida demora um ou dois minutos, porque agora o App Lab compila e grava o sketch. Abra <http://<nome-da-placa>.local:7000> (se a página já estava aberta, recarregue), envie uma foto com uma pessoa e depois uma sem ninguém. O LED vermelho da placa (LED3) acompanha, e o console registra:

```
Deteccao: 1 objetos em 1106 ms  
Pessoa na foto: True -> LED aceso  
Deteccao: 6 objetos em 1149 ms  
Pessoa na foto: False -> LED apagado
```

A linha `Deteccao`: ... só aparece se você fez o Passo 10 do Laboratório 2. Repare na proporção: o modelo leva mais de um segundo para pensar, e a Bridge leva 7 ms para agir. Em Edge AI, o tempo quase sempre está no modelo.

Perguntas de checkpoint

P1: qual foi a sua latência média? Uma câmera de segurança a 10 quadros por segundo tem 100 ms por quadro. Que fração desse orçamento a Bridge consome?

P2: um laço faz três chamadas por ciclo: ler o sensor, mover o servo, acender o LED. Com a latência que você mediu, qual é a frequência máxima desse laço? Como você reduziria o custo sem mudar o hardware?

P3: o que acontece se a Bridge parar de responder com o LED aceso? Como você projetaria um comportamento seguro no sketch?

P4: em que situação `notify` é preferível a `call`, e o que você perde ao usá-lo?

Principais lições

- A Bridge é a espinha dorsal de todo app do UNO Q: o Python chama uma função pelo nome, o MCU executa e devolve o resultado, em cerca de 7 ms.
- `call` espera resposta, `notify` não, `provide` expõe. O caminho usual do MCU para o Python é o `notify`.
- Agrupar várias operações em uma única função do MCU reduz o custo proporcionalmente ao número de chamadas evitadas. O Laboratório 5 mede isso.
- Nunca chame `Bridge.call` dentro de um callback RPC.

O que entregar

1. Seu `sketch/sketch.ino` e `python/main.py`.
2. Os resultados da medição de latência (média, mínimo, máximo, desvio padrão).
3. Respostas escritas para as perguntas de checkpoint.

Solução de problemas

Problema	Solução
Erro de função não encontrada	O nome em <code>Bridge.call</code> tem que ser idêntico ao do <code>Bridge.provide</code> . Salve e rode de novo para regravar o sketch.
<code>Bridge.call</code> demora e estoura o tempo	A função do MCU está bloqueando (um <code>delay</code> longo, por exemplo). Simplifique-a, ou passe <code>timeout=</code> na chamada.
Leituras inconsistentes	Chamadas em sequência muito

Problema	Solução
A comunicação trava depois de um tempo	apertada; um <code>time.sleep</code> curto entre iterações resolve. Verifique se alguma função que o Python registrou com <code>Bridge.provide</code> (um callback RPC, chamado pelo MCU) chama <code>Bridge.call</code> . Troque por <code>notify</code> .
O sketch não compila redefinition of 'void loop()'	Confira se cada <code>provide</code> aponta para uma função existente com esse nome. O sketch ficou com duas funções <code>loop</code> . No Passo 9, a nova substitui a vazia.

Notas

Espaço para suas anotações.

Laboratório 5: Laço de controle entre domínios

Duração	Dificuldade	Pré-requisitos
90 min	Intermediário	Laboratório 4 concluído

Objetivo

Construir um laço de controle em que o processador Linux decide e o MCU aciona atuadores em tempo real. Medir a frequência e o jitter do laço, comparar formas de usar a Bridge e entender o que “tempo real” significa em um sistema com dois processadores.

O que você vai aprender

1. Fechar um laço sentir-decidir-agir que atravessa a fronteira entre MPU e MCU.
2. Medir e reportar frequência e jitter do laço.
3. Quantificar o efeito de agrupar chamadas da Bridge.
4. Identificar o gargalo em um sistema de controle heterogêneo.

Materiais necessários

Item	Detalhes
Arduino UNO Q 4GB	Conectado por USB ou Wi-Fi
Computador com Arduino App Lab	
Servomotor SG90 ou similar (opcional)	Fio laranja (sinal) em D9, vermelho em 5V, marrom em GND. Sem servo, o LED embutido serve de atuador
Potenciômetro de 10 kΩ (opcional)	Extremos em 3,3 V e GND, cursor em A0
Protoboard e jumpers	

Aviso: o servo consome corrente ao se mover. Alimente-o pelo pino 5V da placa, nunca pelo 3V3, e use uma fonte de 3 A.

Visão geral do conceito

O Laboratório 4 mostrou como chamar funções do MCU a partir do Python. Este laboratório fecha o ciclo: o MCU lê um sensor, o Python decide, o MCU atua, e tudo se repete. Um laço de controle é isso, um ciclo contínuo de ler, decidir e agir. É a base da robótica, da automação industrial e dos veículos autônomos, e os princípios de temporização que você mede aqui valem em qualquer escala.

A métrica que importa é a **frequência do laço**: quantos ciclos completos por segundo. Para piscar um LED, 1 Hz basta. Para um robô que se equilibra, são necessários 100 Hz ou mais. Cada chamada à Bridge custa cerca de 7 ms, e um ciclo ingênuo faz três delas: ler, mover o servo, acender o LED. Isso limita o laço a uns 45

Hz antes de qualquer processamento. Você vai medir isso e depois medir o que acontece quando o ciclo inteiro vira uma única chamada.

Tão importante quanto a média é o **jitter**, a variação do tempo de ciclo. Um laço que faz 100 Hz na média mas às vezes cai para 60 Hz produz solavancos visíveis em um motor, mesmo com a média parecendo boa.

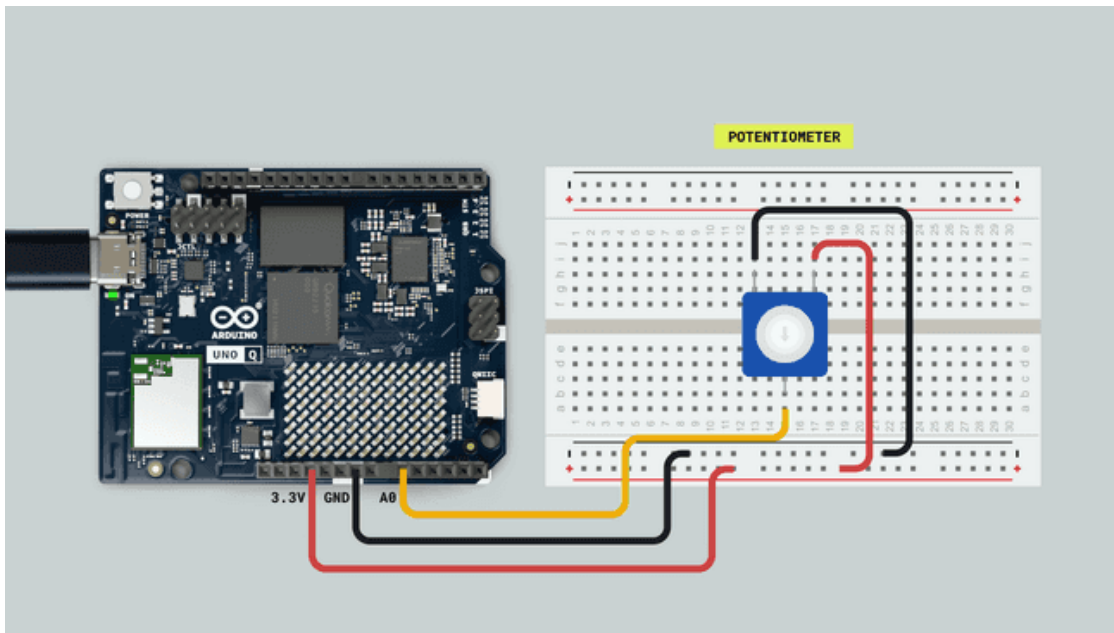


Figura 5.1: Potenciômetro ligado ao UNO Q, entrada analógica em A0

Procedimento

Parte A: Adicionar a biblioteca Servo

Passo 1: crie um app vazio chamado `control-loop`, pelo botão **+** ao lado de **Apps**.

Passo 2: o core do UNO Q não inclui a biblioteca Servo. Acima dos painéis da esquerda, clique no ícone de livro, **Add Sketch Library**, busque Servo e instale a versão 1.3.0 (autor Arduino). Ela aparece no painel “Sketch Libraries” e o App Lab a declara em `sketch/sketch.yaml`. Se a biblioteca ainda não estiver na placa, o App Lab a baixa na primeira compilação, e para isso a placa precisa estar no Wi-Fi, com internet.

Parte B: Laço de controle simples

Passo 3: substitua todo o conteúdo de `sketch/sketch.ino` por este sketch, com as funções de sensor e atuadores, mais uma função agrupada que faz o ciclo inteiro no MCU.

```
// sketch/sketch.ino – sensor, atuadores e uma versao agrupada do ciclo
#include <Arduino_RouterBridge.h>
#include <Servo.h>
```

```

Servo myServo;

int read_sensor_raw() {
    return analogRead(A0);           // 0 a 1023
}

void set_servo(int angle) {
    myServo.write(constrain(angle, 0, 180));
}

void set_led(bool state) {
    digitalWrite(LED_BUILTIN, state ? LOW : HIGH); // LED_BUILTIN e ativo em LOW
}

// Ciclo completo em uma unica chamada: le, decide, atua e devolve a leitura
int sense_and_act(int threshold) {
    int raw = analogRead(A0);
    myServo.write(constrain(raw * 180 / 1024, 0, 180)); // multiplica antes: divisao in
    teira
    digitalWrite(LED_BUILTIN, raw > threshold ? LOW : HIGH);
    return raw;
}

void setup() {
    pinMode(LED_BUILTIN, OUTPUT);
    pinMode(A0, INPUT);
    myServo.attach(9);                // servo em D9
    Bridge.begin();
    Bridge.provide("read_sensor_raw", read_sensor_raw);
    Bridge.provide("set_servo", set_servo);
    Bridge.provide("set_led", set_led);
    Bridge.provide("sense_and_act", sense_and_act);
}

void loop() {}

```

Se você não tem servo, não precisa mudar nada: o sketch roda igual sem o motor ligado, e o LED serve de atuador.

Passo 4: substitua todo o conteúdo de `python/main.py` por este código, que faz três chamadas por ciclo e mede o tempo.

```

# python/main.py - laco de controle com medicao de frequencia e jitter
from arduino.app_utils import App, Bridge
import time, statistics

loop_times = []

def control_cycle():
    # SENTIR: Ler o sensor no MCU
    raw = Bridge.call("read_sensor_raw")
    # PENSAR: decidir no MPU
    angle = int(raw / 1024 * 180)           # mapeia 0-1023 para 0-180 graus
    led_on = raw > 512
    # AGIR: comandar atuadores no MCU
    Bridge.call("set_servo", angle)
    Bridge.call("set_led", led_on)

```

```

    return raw, angle

print("Laco de controle: 200 iteracoes")
for i in range(200):
    t0 = time.perf_counter()
    raw, angle = control_cycle()
    loop_ms = (time.perf_counter() - t0) * 1000
    loop_times.append(loop_ms)
    if i % 20 == 0:
        print(f"Iteracao {i}: sensor={raw}, angulo={angle}, ciclo={loop_ms:.1f} ms ({1000/loop_ms:.0f} Hz)")

avg = statistics.mean(loop_times)
print(f"\nEstatisticas do laco:")
print(f"  Media:  {avg:.2f} ms ({1000/avg:.0f} Hz)")
print(f"  Min:    {min(loop_times):.2f} ms")
print(f"  Max:    {max(loop_times):.2f} ms")
print(f"  Jitter: {statistics.stdev(loop_times):.2f} ms")

App.run()

```

Passo 5: rode. Gire o potenciômetro e veja o servo acompanhar e o LED reagir. Registre a frequência e o jitter. Em uma placa de referência o ciclo de três chamadas ficou em 22,3 ms, ou 45 Hz, com jitter de 0,8 ms.

Parte C: Entender os limites de desempenho

Passo 6: meça o mesmo ciclo de três formas. Guarde os números do Passo 5 e substitua todo o conteúdo de `python/main.py` por:

```

# python/main.py - tres formas de fechar o mesmo laco
from arduino.app_utils import App, Bridge
import time, statistics

def run_loop(label, body, n=200):
    times = []
    for _ in range(n):
        t0 = time.perf_counter()
        body()
        times.append((time.perf_counter() - t0) * 1000)
    avg = statistics.mean(times)
    print(f"{label}: media {avg:.2f} ms ({1000/avg:.0f} Hz), "
          f"min {min(times):.2f}, max {max(times):.2f}, jitter {statistics.stdev(times):.2f} ms")

def three_calls():
    # A) tres chamadas por ciclo
    raw = Bridge.call("read_sensor_raw")
    Bridge.call("set_servo", int(raw / 1024 * 180))
    Bridge.call("set_led", raw > 512)

def one_call():
    # B) uma chamada agrupada, com retorno
    Bridge.call("sense_and_act", 512)

def one_notify():
    # C) uma chamada agrupada, sem esperar
    Bridge.notify("sense_and_act", 512)

```

```
run_loop("A) 3 x call", three_calls)
run_loop("B) 1 x call", one_call)
run_loop("C) 1 x notify", one_notify)
```

```
App.run()
```

Passo 7: rode e monte a sua tabela. Os valores de referência:

Forma	Tempo por ciclo	Frequência	Jitter
A) três call	22,4 ms	45 Hz	0,8 ms
B) um call agrupado	8,4 ms	120 Hz	0,6 ms
C) um notify agrupado	0,08 ms	o programa imprime mais de 12.000 Hz, que não é a frequência do laço	0,02 ms

Agrupar o ciclo em uma função do MCU dá quase três vezes mais frequência, porque elimina duas idas e voltas. Já o caso C engana: `notify` devolve o controle ao Python assim que a mensagem é enfileirada, sem esperar o MCU executar. O número mede a fila, não o laço fechado. O MCU continua limitado pelo tempo de executar cada mensagem, e se o Python enviar mais rápido do que isso, as mensagens se acumulam.

Passo 8: experimente. Acrescente `time.sleep(0.01)` como última linha da função `three_calls`, para simular um passo PENSAR de 10 ms, e veja a frequência do caso A cair para perto de 30 Hz. O `sleep` tem que ficar dentro da função: é ela que está sendo cronometrada. Depois troque o `sleep` por um `print("ciclo")` e meça de novo; o custo de um `print` por ciclo é quase nulo. O console recebe 200 linhas `ciclo` antes de cada resultado, então role até o fim para ler a medição. Anote os dois resultados.

Dica: em aplicações reais, o passo PENSAR costuma ser uma inferência de 30 a 400 ms. Aí o gargalo é o modelo, não a Bridge. Projete o sistema em torno do componente mais lento; o Laboratório 8 mede exatamente esses tempos.

Exemplo resolvido: um laço de estabilização de drone precisa de 200 Hz ou mais, ou seja, 5 ms por ciclo. Com três chamadas o seu laço faz 22 ms, quatro vezes mais lento. Agrupando em uma chamada ele faz 8 ms, ainda 60% acima do orçamento, e isso antes de qualquer cálculo. Conclusão: o laço de estabilização inteiro tem que viver no MCU; o MPU só ajusta os valores de referência (setpoints) de vez em quando. Reduzir pela metade o componente mais lento dobra a frequência; ajustar o mais rápido não muda quase nada.

Perguntas de checkpoint

P1: qual foi a frequência do laço com três chamadas? E com uma chamada agrupada? Alguma delas serve para estabilizar um drone a 200 Hz?

P2: o que é jitter e por que ele importa mais que a média em um sistema de controle?

P3: o caso C (notify) mostrou milhares de hertz. Por que esse número não representa a frequência real do laço?

P4: se o passo PENSAR fosse uma inferência de 50 ms, qual seria a frequência máxima do laço com uma chamada agrupada?

Principais lições

- Um sistema de controle é tão rápido quanto seu componente mais lento: tempo do laço = latência da Bridge vezes o número de chamadas, mais o tempo de decisão.
- Agrupar o ciclo em uma função do MCU quase triplica a frequência.
- `notify` não serve para medir laço fechado; ele mede só a fila de envio.
- Meça sempre a média, o pior caso (o máximo) e o jitter antes de declarar um sistema pronto.

Extensão opcional: monitorar o laço pelo celular

Adicione o Brick Arduino Cloud ao app e publique `sensor_value`, `servo_angle` e `loop_frequency` como variáveis da Cloud, atualizadas a cada 5 segundos. Monte um painel e acompanhe pelo app Arduino IoT Remote. O Laboratório 6 mostra como publicar valores da placa para a Cloud.

O que entregar

1. Seu `sketch/sketch.ino` e `python/main.py`.
2. Sua tabela A/B/C com frequência, mínimo, máximo e jitter, mais os resultados do Passo 8.
3. Respostas escritas para as perguntas de checkpoint.

Solução de problemas

Problema	Solução
Servo.h: No such file or directory	A biblioteca não foi adicionada. Use Add Sketch Library e instale Servo 1.3.0.
O servo vibra ou não se move	Confira a fiação: sinal em D9, 5 V, GND comum. Confira a fonte de 3 A.
Laço muito lento (menos de 20 Hz)	Procure <code>sleep</code> ou trabalho pesado dentro do ciclo e agrupe as chamadas. Um <code>print</code>

Problema	Solução
	por ciclo não é o problema.
Erros da Bridge em laços rápidos	Não misture <code>notify</code> em rajada com <code>call</code> ; a fila do MCU se enche. Use <code>call</code> ou intercale um <code>sleep</code> curto.
A frequência varia muito entre execuções	Feche outros apps na placa; a inferência de um app de visão em paralelo rouba CPU.

Notas

Espaço para suas anotações.

Laboratório 6: Sensores Modulino e painel na nuvem

Duração	Dificuldade	Pré-requisitos
90-120 min	Intermediário	Laboratórios 1, 2 e 4 concluídos. Não usa câmera

Objetivo

Ler sensores Modulino pelo MCU, construir um sistema de monitoramento com dois deles, processar as leituras localmente no UNO Q, enviar resumos para o Arduino Cloud e acompanhar um painel ao vivo pelo celular.

O que você vai aprender

1. Adicionar a biblioteca `Arduino_Modulino` a um sketch, ler sensores Modulino no MCU e entregar as leituras ao Python pela Bridge
2. Agregar dados localmente com média móvel e alertas por limiar
3. Publicar telemetria no Arduino Cloud com política periódica e por mudança
4. Decidir o que fica na borda e o que vai para a nuvem

Materiais necessários

Item	Detalhes
Arduino UNO Q 4GB	Conectado por USB ao PC ou por Wi-Fi
Modulino Thermo e Modulino Distance	Encadeados por cabo Qwiic. Modulino Pixels é opcional, para a variação da Parte A
Computador com Arduino App Lab	Logado na conta Arduino
Conta Arduino Cloud	Só para as Partes D e E. Placa já conectada pelo App Lab (Laboratório 1, Parte D)
Smartphone com app Arduino IoT Remote	Opcional, para o painel móvel

Visão geral do conceito

Modulinos são módulos de sensor ligados por um cabo de quatro fios chamado Qwiic: alimentação, terra e as duas linhas de dados do barramento I2C. Podem ser encadeados, um no outro. O conector Qwiic do UNO Q está identificado na placa e aparece na Figura 6.1. No sketch, esse barramento se chama `wire1`; o `wire`, sem número, é o I2C dos pinos do cabeçalho. Não há Brick nem pacote Python para Modulino: eles são lidos pelo **MCU**, com a biblioteca `Arduino_Modulino` no sketch, e as leituras chegam ao Python pela Bridge.

Os laboratórios anteriores trabalharam com uma entrada por vez. Sistemas reais combinam vários sensores e reportam para fora. Uma estufa inteligente lê temperatura, umidade e luz ao mesmo tempo, decide sozinha sobre irrigação, e ainda mostra um painel para o agricultor acompanhar de longe.

Este laboratório monta o padrão borda-para-nuvem. O UNO Q lê os sensores em alta frequência e decide localmente. A nuvem recebe apenas resumos: médias a cada 30 segundos e alertas quando um limite é ultrapassado. Enviar cada leitura bruta inundaria a rede sem acrescentar informação.

A divisão de trabalho na placa segue o padrão dos Laboratórios 4 e 5. O sketch no MCU fala com os Modulinos pelo barramento I2C e envia cada leitura ao Python com `Bridge.notify`. O Python recebe as leituras em um callback registrado com `Bridge.provide`, mantém a janela móvel, avalia os limiares e escreve nas variáveis da Cloud. A conexão com a Cloud é feita por um serviço do sistema; o código não guarda credenciais.

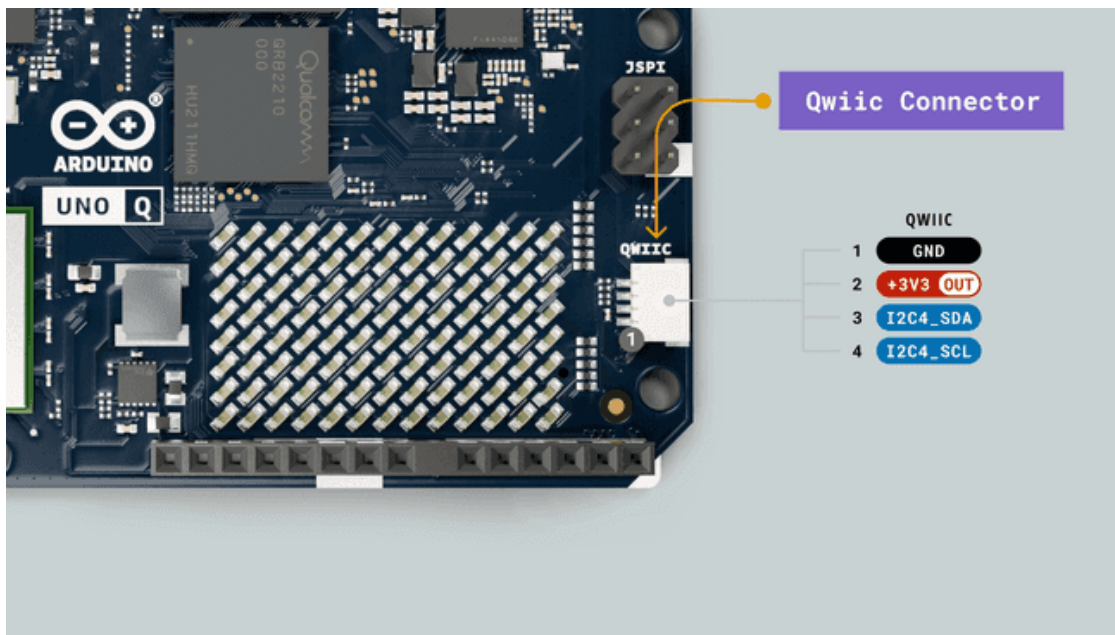


Figura 6.1: Conector Qwiic do UNO Q, interface I2C para os Modulinos

Procedimento

Parte A: O primeiro sensor

Passo 1: Conecte um Modulino Thermo ao conector Qwiic da placa.

Passo 2: Crie um app vazio chamado `modulino-test` pelo botão **+** ao lado de **Apps**, na barra lateral. Acima dos painéis da esquerda há três ícones; clique no do meio, **Add Sketch Library**, procure `Arduino_Modulino`, escolha a versão 0.7.0 e clique em **Install**.

A biblioteca não é adicionada sozinha: sem este passo a compilação para em `Arduino_Modulino.h`: No such file or directory. Ela também depende de outras sete bibliotecas, uma para cada tipo de sensor. Abra `sketch/sketch.yaml` e confira se ele ficou com a biblioteca e as sete dependências:

```
profiles:
  default:
    platforms:
      - platform: arduino:zephyr
    libraries:
      - Arduino_Modulino (0.7.0)
      - dependency: ArduinoGraphics (1.1.4)
      - dependency: Arduino_HS300x (1.0.0)
      - dependency: Arduino_LPS22HB (1.0.2)
      - dependency: Arduino_LSM6DSOX (1.1.2)
      - dependency: Arduino_LTR381RGB (1.0.0)
      - dependency: STM32duino_VL53L4CD (1.0.5)
      - dependency: STM32duino_VL53L4ED (1.0.1)
default_profile: default
```

Se faltar alguma linha, ou se você preferir o terminal, copie o arquivo pronto de um exemplo oficial que usa a mesma biblioteca. No terminal da placa:

```
cp /var/lib/arduino-app-cli/examples/inspirational/common/home-climate-monitoring-and-storage/sketch/sketch.yaml ~/ArduinoApps/modulino-test/sketch/
```

Passo 3: Substitua todo o conteúdo de `sketch/sketch.ino` por:

```
#include <Arduino_Modulino.h>
#include <Arduino_RouterBridge.h>

ModulinoThermo thermo;

unsigned long previousMillis = 0;
const long interval = 1000;           // uma leitura por segundo

void setup() {
  Bridge.begin();
  Modulino.begin(Wire1);               // barramento Qwiic do UNO Q
  thermo.begin();
}

void loop() {
  unsigned long now = millis();
  if (now - previousMillis >= interval) {
    previousMillis = now;
    float celsius = thermo.getTemperature();
    float humidity = thermo.getHumidity();
    // Envia as duas leituras ao Python, sem esperar resposta
    Bridge.notify("record_sensor_samples", celsius, humidity);
  }
}
```

Passo 4: Substitua todo o conteúdo de `python/main.py` por este código, que recebe as leituras e decide:

```

from arduino.app_utils import App, Bridge

def record_sensor_samples(celsius: float, humidity: float):
    # Chamado pelo sketch a cada segundo
    estado = "quente" if celsius > 28 else "ok"
    print(f"Temp: {celsius:.1f} C | Umidade: {humidity:.0f} % | {estado}")

Bridge.provide("record_sensor_samples", record_sensor_samples)

App.run()

```

Clique em Run. A primeira partida de um app que usa a biblioteca Modulino leva cerca de 5 minutos (medido: 4 min 40 s), porque a biblioteca e as dependências são compiladas junto com o sketch. O console imprime uma linha por segundo. Sentir (MCU lê o sensor), pensar (Python decide) e agir (imprimir, ou acionar algo): um ciclo completo, atravessando os dois processadores.

Dica: sem o sensor conectado o app roda do mesmo jeito, mas as leituras chegam como 0.0. Use isso para testar o código antes de ter o hardware.

Variação opcional: Distance e Pixels. Outros Modulinos seguem o mesmo padrão, trocando a classe e os métodos. O ModulinoDistance oferece available() e get(), que devolve a distância em milímetros. O ModulinoPixels acende os LEDs com set(indice, r, g, b, brilho) seguido de show(). Um semáforo de distância, verde longe, amarelo perto e vermelho muito perto, cabe inteiro no sketch:

```

#include <Arduino_Modulino.h>
#include <Arduino_RouterBridge.h>

ModulinoDistance distance;
ModulinoPixels leds;

void setup() {
    Bridge.begin();
    Modulino.begin(Wire1);
    distance.begin();
    leds.begin();
}

void loop() {
    if (distance.available()) {
        float cm = distance.get() / 10.0;           // get() devolve milímetros
        uint8_t r = cm < 10 ? 255 : (cm < 50 ? 255 : 0);
        uint8_t g = cm < 10 ? 0 : 255;
        for (int i = 0; i < 8; i++) leds.set(i, r, g, 0, 25);
        leds.show();
        Bridge.notify("record_distance", cm);       // e o Python recebe a leitura
    }
    delay(500);
}

```

O Python que recebe essa leitura segue o mesmo molde do Passo 4:

```

from arduino.app_utils import App, Bridge

def record_distance(cm: float):
    print(f"Distancia: {cm:.1f} cm")

Bridge.provide("record_distance", record_distance)

App.run()

```

Sem o Modulino Distance conectado, este sketch não envia nada e o console fica vazio: `available()` nunca é verdadeiro. Esses métodos foram conferidos na biblioteca `Arduino_Modulino` 0.7.0 instalada na placa, e o sketch compila. O código com Distance e Pixels não foi executado com os sensores nesta revisão.

Parte B: Leitura de dois sensores

Passo 5: Com o Thermo já no conector Qwiic da placa, ligue o Modulino Distance ao segundo conector do Thermo. Os Modulinos são encadeados: todos ficam no mesmo barramento I2C.

Passo 6: Pare o `modulino-test`. Crie um app vazio chamado `sensor-dashboard`, sem Bricks por enquanto, e adicione a biblioteca `Arduino_Modulino` como no Passo 2. Um app novo compila a biblioteca de novo: conte com outros 5 minutos na primeira partida dele. Pelo terminal, basta copiar o `sketch.yaml` do app anterior:

```
cp ~/ArduinoApps/modulino-test/sketch/sketch.yaml ~/ArduinoApps/sensor-dashboard/sketch/
```

Passo 7: Substitua todo o conteúdo de `sketch/sketch.ino` por:

```

#include <Arduino_Modulino.h>
#include <Arduino_RouterBridge.h>

ModulinoThermo thermo;
ModulinoDistance distance;

unsigned long previousMillis = 0;
const long interval = 1000; // uma leitura por segundo

void setup() {
    Bridge.begin();
    Modulino.begin(Wire1); // no UNO Q os Modulinos ficam no Wire1
    thermo.begin();
    distance.begin();
}

void loop() {
    unsigned long now = millis();
    if (now - previousMillis >= interval) {
        previousMillis = now;
        float celsius = thermo.getTemperature();
        float humidity = thermo.getHumidity();
        // get() devolve milímetros; convertemos para cm. Sem leitura válida, enviamos -1
        float cm = distance.available() ? distance.get() / 10.0 : -1;
        // Envia as três leituras ao Python de uma vez, sem esperar resposta
    }
}

```

```

        Bridge.notify("record_samples", celsius, humidity, cm);
    }
}

```

Passo 8: Substitua todo o conteúdo de `python/main.py` por:

```

from arduino.app_utils import App, Bridge

def record_samples(celsius: float, humidity: float, cm: float):
    print(f"Temp: {celsius:.1f} C | Umid: {humidity:.0f} % | Dist: {cm:.1f} cm")
    if celsius > 30:
        print(f"  ALERTA: temperatura {celsius:.1f} C acima de 30 C")
    if 0 <= cm < 10:
        print(f"  ALERTA: objeto a {cm:.1f} cm")

Bridge.provide("record_samples", record_samples)
App.run()

```

Passo 9: Clique em Run. O console deve mostrar uma linha por segundo. Aproxime a mão do Distance e aqueça o Thermo entre os dedos para ver os alertas.

Dica: sem os Modulinos conectados, o sketch compila e roda, e o Python recebe temperatura e umidade em 0.0 e distância em -1.0, o valor que o sketch usa para “sem leitura”. Isso é útil para testar a lógica antes de ter o hardware.

Parte C: Agregação local

O padrão de borda é: ler rápido, resumir localmente, enviar pouco. Aqui a agregação acontece dentro do callback, porque é nele que cada leitura chega.

Passo 10: Substitua todo o conteúdo de `python/main.py` por:

```

from arduino.app_utils import App, Bridge
from collections import deque
import statistics

# Janela das ultimas 30 Leituras (30 segundos a 1 Hz)
temp_window = deque(maxlen=30)
dist_window = deque(maxlen=30)
alert_count = 0
sample_count = 0

def record_samples(celsius: float, humidity: float, cm: float):
    global alert_count, sample_count
    sample_count += 1
    temp_window.append(celsius)
    if cm >= 0:                                     # -1 e "sem leitura valida" e nao entra na media
        dist_window.append(cm)

    if celsius > 30:
        alert_count += 1

    # Resumo a cada 5 Leituras
    if sample_count % 5 == 0:

```

```

        media_dist = f"{statistics.mean(dist_window):.1f} cm" if dist_window else "sem le
itura"
        print(f"Media temp: {statistics.mean(temp_window):.1f} C | "
              f"Media dist: {media_dist} | Alertas: {alert_count}")

```

```

Bridge.provide("record_samples", record_samples)
App.run()

```

Rode e confira: a cada 5 segundos sai uma linha de resumo. Repare no tratamento do valor -1: um valor que significa “não sei” não pode entrar em uma média.

Parte D: Envio para o Arduino Cloud

Passo 11: Pare o app sensor-dashboard. Adicione o Brick **Arduino Cloud** pelo primeiro dos três ícones acima do painel Bricks. O app.yaml ganha - arduino:arduino_cloud.

Passo 12: Confirme que a placa está conectada à Cloud: no App Lab, abra **Settings**, seção **Arduino Cloud**, e veja o status “Connected”. Se não estiver, siga a Parte D do Laboratório 1.

Passo 13: No site cloud.arduino.cc, crie uma Thing chamada sensor-dashboard, associe o seu dispositivo e crie as variáveis da tabela. A política de atualização é escolhida na própria janela de criação da variável, em **Variable Update Policy: On change** ou **Periodically**, com o período em segundos.

Nome	Tipo	Permissão	Atualização
avg_temperature	Float	Read only	Periodically, 30 s
avg_distance	Float	Read only	Periodically, 30 s
alert_count	Integer	Read only	On change
status	String	Read only	On change

Os nomes precisam ser idênticos aos usados no código.

Passo 14: Substitua todo o conteúdo de python/main.py por:

```

from arduino.app_utils import App, Bridge
from arduino.app_bricks.arduino_cloud import ArduinoCloud
from collections import deque
import statistics

cloud = ArduinoCloud()  # a conexao e feita pelo servico da placa; sem credenciais aqui

# interval=30: o valor atual e publicado a cada 30 s
cloud.register("avg_temperature", value=0.0, interval=30)
cloud.register("avg_distance", value=0.0, interval=30)
# sem interval: publicado quando muda
cloud.register("alert_count", value=0)
cloud.register("status", value="OK")

temp_window = deque(maxlen=30)
dist_window = deque(maxlen=30)
alert_count = 0

```

```

sample_count = 0

def record_samples(celsius: float, humidity: float, cm: float):
    global alert_count, sample_count
    sample_count += 1
    temp_window.append(celsius)
    if cm >= 0:                                     # -1 e "sem leitura valida" e nao entra na media
        dist_window.append(cm)

    if celsius > 30:
        alert_count += 1
        cloud.alert_count = alert_count
        cloud.status = f"ALERTA: {celsius:.1f} C"
    elif cloud.status != "OK":
        cloud.status = "OK"

    # As medias sao atualizadas a cada leitura; o Brick publica a cada 30 s
    if len(temp_window) >= 5:
        media_temp = round(statistics.mean(temp_window), 1)
        cloud.avg_temperature = media_temp
        if dist_window:
            cloud.avg_distance = round(statistics.mean(dist_window), 1)
        if sample_count % 10 == 0:
            print(f"Cloud: avg_temperature={media_temp} | alert_count={alert_count} | status={cloud.status}")

Bridge.provide("record_samples", record_samples)
App.run()

```

Repare na divisão: o código escreve nas variáveis a cada leitura, e o Brick decide quando publicar, a cada 30 segundos para as médias e imediatamente para os alertas. A linha `cloud: ...`, impressa a cada 10 leituras, serve para você confirmar no console o que está sendo escrito, antes de montar o painel.

Passo 15: Clique em Run. Se aparecer o diálogo “Provisioning required”, clique em “Connect device” e conclua o fluxo. O console deve mostrar a linha `cloud: ...` a cada 10 segundos, sem erros. Se a Thing acabou de ser criada, a associação pode levar alguns minutos.

Parte E: O painel

Passo 16: No site, em **Dashboards**, crie um painel Sensores com quatro widgets: um Gauge ligado a `avg_temperature`, um Value ligado a `avg_distance`, um Chart ligado a `avg_temperature` para o histórico, e um Value ligado a `status`.

Passo 17: Aqueça o Thermo entre os dedos por um minuto. O `status` deve mudar para “ALERTA” na hora, e a média deve subir na próxima publicação.

Passo 18: Abra o app Arduino IoT Remote no celular, com a mesma conta. O painel aparece com os mesmos widgets, atualizando a cada 30 segundos.

Exemplo resolvido: dois sensores a 1 Hz geram 2 leituras por segundo, 7.200 por hora. Com médias a cada 30 segundos, a nuvem recebe 2 valores

a cada 30 segundos, 240 por hora, 30 vezes menos. Com 200 placas em uma fábrica, a diferença é entre 1,4 milhão e 48 mil mensagens por hora.

Perguntas de checkpoint

P1: Por que um sensor Modulino é lido no sketch e não diretamente no Python?

P2: Por que o app publica médias a cada 30 segundos em vez de cada leitura? O que aconteceria com a rede e com a conta da Cloud se todas as leituras fossem enviadas?

P3: Se o Wi-Fi cair por 5 minutos, o monitoramento local para? Por quê? O que você faria para guardar os dados e enviá-los depois?

P4: Uma fábrica tem 200 placas UNO Q, cada uma lendo 5 sensores a 10 Hz. Calcule o total de leituras por minuto se tudo fosse enviado, e depois se cada placa enviasse um resumo por minuto.

Principais lições

- Sensores Modulino são lidos no MCU e chegam ao Python por `Bridge.notify` e `Bridge.provide`, não por polling em Python
- A biblioteca `Arduino_Modulino` precisa ser declarada no `sketch.yaml`, junto com as sete bibliotecas de que depende
- A borda processa em alta frequência e envia só resumos e alertas; isso reduz o tráfego em dezenas ou centenas de vezes
- No Arduino Cloud, o código escreve nas variáveis e o Brick decide quando publicar, por período ou por mudança
- A conexão com a Cloud pertence à placa, não ao app: nenhuma credencial fica no código

Extensão opcional

Adicione o Brick Web UI e uma página em `assets/index.html` que receba as leituras por `web_ui.send_message("sample", {...})` dentro do callback, com `web_ui = WebUI()` como no Laboratório 2. Assim você tem um painel local, que funciona mesmo sem internet, e o painel da Cloud para acompanhamento remoto.

O que entregar

1. O código completo (`sketch/sketch.ino` e `python/main.py`).
2. Uma captura de tela do painel do Arduino Cloud com dados ao vivo.
3. Respostas escritas para as perguntas de checkpoint.

Solução de problemas

Problema	Solução
Modulino não detectado, leituras em 0.0	Confira o cabo Qwiic nas duas pontas.

Problema	Solução
ou -1.0	Teste um sensor por vez. Confirme <code>Modulino.begin(Wire1)</code> no sketch.
Arduino_Modulino.h: No such file or directory	A biblioteca não está declarada no <code>sketch.yaml</code> . Refaça o Passo 2.
vl53l4cd_class.h: No such file or directory	A biblioteca foi declarada sem as dependências. Copie o <code>sketch.yaml</code> do exemplo, como no Passo 2.
“Provisioning required” ao clicar em Run	A placa não está conectada à Cloud. Clique em “Connect device” e conclua o fluxo em Settings.
Variáveis não atualizam no painel	Os nomes na Thing precisam ser exatamente os do <code>register(...)</code> . Confira também se a Thing está associada ao seu dispositivo.
Aviso “no thing assigned yet” no console	A Thing ainda não foi associada ou a associação está propagando. Aguarde alguns minutos ou reinicie o serviço com <code>sudo systemctl restart arduino-cloud-connector</code> .
App IoT Remote sem painel	Entre com a mesma conta Arduino. Puxe a lista para baixo para atualizar.

Notas

Espaço para suas anotações.

Laboratório 7: Noções básicas de câmera e captura de imagem

Duração	Dificuldade	Pré-requisitos
60-90 min	Intermediário	Laboratório 3 concluído; Wi-Fi configurado no Laboratório 0

Objetivo

Conectar uma webcam ao UNO Q, capturar imagens de dentro de um app do App Lab e entender os fundamentos da imagem digital: resolução, espaços de cor e o custo computacional de processar dados visuais.

Por que isso importa: a visão é a entrada que mais consome dados em Edge AI. Um único quadro de câmera pode ser de 100 a 1.000 vezes maior que uma leitura de sensor. Entender os dados de imagem agora prepara você para medir o custo dos modelos (Laboratório 8), construir visão em tempo real (Laboratório 9) e otimizar modelos (Laboratórios 10 a 12).

O que você vai aprender

1. Montar a placa com hub USB-C e webcam, acessando-a pela rede
2. Verificar se a câmera foi detectada pelo Linux da placa
3. Capturar e salvar imagens usando o periférico Camera do App Lab
4. Medir a taxa de captura e calcular o volume de dados por segundo
5. Calcular os requisitos de memória para diferentes resoluções e espaços de cor

Materiais necessários

Item	Detalhes
Arduino UNO Q 4GB	Acessado pela rede (Wi-Fi), não pelo USB do PC
Hub USB-C com Power Delivery	Fonte de 5 V/3 A na entrada PD do hub, webcam em uma porta USB-A do hub. Obrigatório: a placa tem uma única porta USB-C
Webcam USB (UVC)	Qualquer webcam padrão. A maioria das webcams modernas é UVC (USB Video Class). Alternativa: módulo de câmera no conector MIPI-CSI
Computador com Arduino App Lab	App Lab em modo Rede, ou terminal SSH

Visão geral do conceito

Os Laboratórios 3 a 5 cobriram a arquitetura da placa e a comunicação entre os processadores. Os Laboratórios 7 a 9 passam para visão, a carga de trabalho mais exigente da Edge AI. Pipelines de visão em veículos autônomos, imagem médica e inspeção industrial começam pelos mesmos fundamentos de câmera que você vai ver aqui.

Uma imagem digital é uma grade de pixels. Cada pixel guarda a cor em três valores, vermelho, verde e azul (RGB), cada um de 0 a 255. Uma imagem RGB de 640x480 tem $640 \times 480 \times 3 = 921.600$ bytes, quase 1 MB de dados brutos. A 30 quadros por segundo, são 26 MB por segundo passando pela memória da placa. Neste manual, 1 KB vale 1.024 bytes e 1 MB vale 1.024 KB, a mesma conta que os scripts fazem.

Resolução	Pixels	Tamanho bruto (RGB)	A 30 FPS
160x120 (QQVGA)	19.200	56 KB	1,6 MB/s
320x240 (QVGA)	76.800	225 KB	6,6 MB/s
640x480 (VGA)	307.200	900 KB	26 MB/s
1280x720 (HD)	921.600	2,6 MB	79 MB/s
1920x1080 (Full HD)	2.073.600	5,9 MB	178 MB/s

É por isso que a visão é a tarefa mais cara em Edge AI, e por isso a escolha de resolução e de modelo importa tanto quanto a escolha do hardware.

Espaços de cor. O OpenCV, biblioteca padrão de processamento de imagem em Python, guarda as cores na ordem BGR (azul, verde, vermelho) em vez de RGB. É um resquício de hardware antigo de câmera. Lembre disso ao inspecionar pixels manualmente. O periférico `Camera` do App Lab entrega quadros nesse mesmo formato. Outros espaços úteis: HSV (matiz, saturação, valor) para rastrear um objeto pela cor independente da iluminação, e escala de cinza, que descarta a cor e mantém só o brilho, reduzindo a memória em dois terços.

O caminho do quadro. Cada quadro passa por várias etapas antes de chegar ao seu código: captura no sensor, transferência USB, decodificação (muitas webcams comprimem em MJPEG, uma sequência de imagens JPEG) e só então o processamento. Cada etapa leva tempo, e as configurações que você escolhe (resolução, formato, taxa de quadros) afetam todas elas.

Onde o código roda. No UNO Q, o `main.py` do seu app executa dentro de um container Linux que já traz o OpenCV, o NumPy e o periférico `Camera`. O Python que você acessa pelo terminal SSH não tem essas bibliotecas. Por isso, neste laboratório, todo código de captura roda como um app do App Lab, não como script solto no terminal.

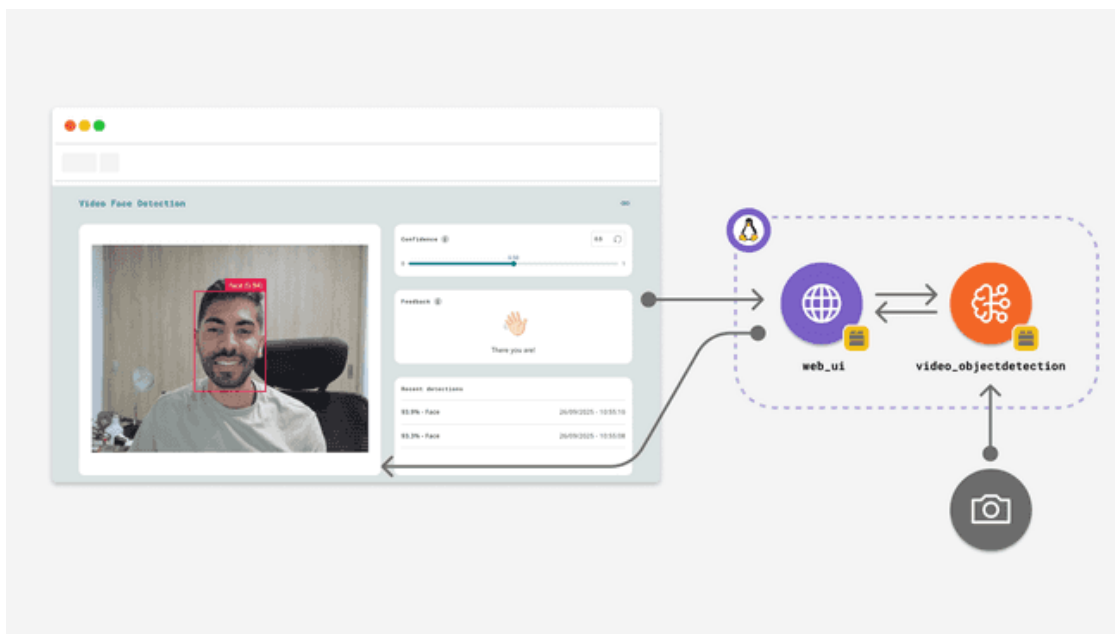


Figura 7.1: Detecção facial em vídeo, captura de câmera e processamento em tempo real

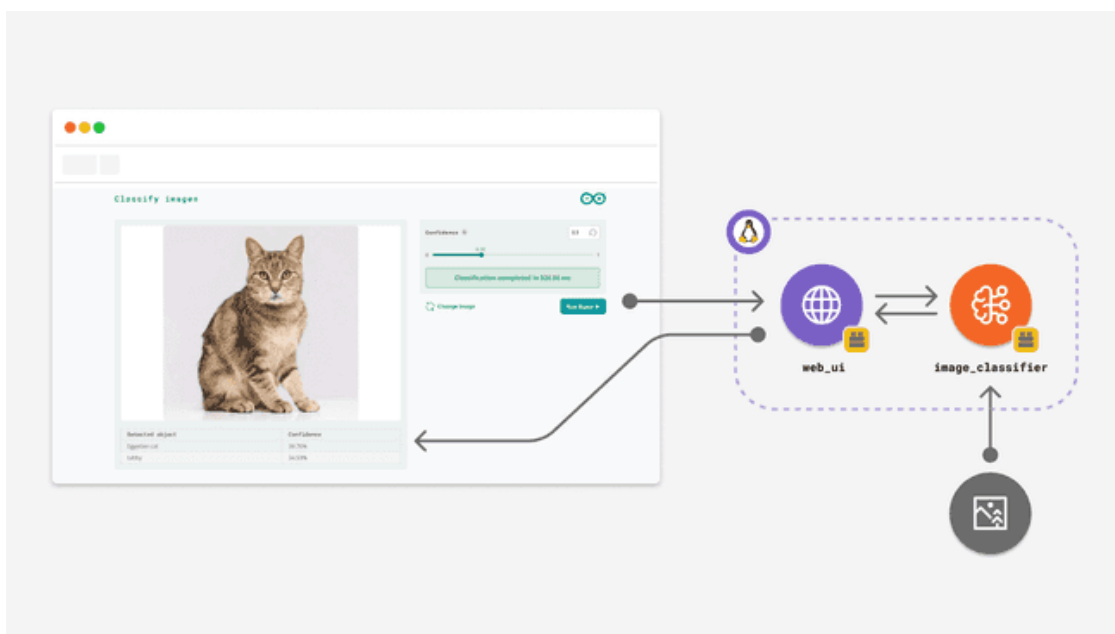


Figura 7.2: Exemplo “Classify images” do App Lab: uma imagem enviada pelo navegador passa pelo Brick de classificação

Procedimento

Parte A: Montando a placa com a webcam

A placa tem uma única porta USB-C. Até aqui ela estava ligada ao computador por esse conector. Para usar uma webcam, o conector passa a ser ocupado por um hub, e o acesso à placa passa a ser pela rede Wi-Fi.

Passo 1: Pare qualquer app em execução no App Lab e desconecte o cabo USB-C do computador.

Passo 2: Ligue a fonte USB-C na entrada PD do hub, conecte a webcam em uma porta USB-A do hub e ligue o hub na placa. A placa pode reiniciar nessa troca; espere cerca de 30 segundos.

Passo 3: No App Lab, a placa deve reaparecer em modo Rede: o nome dela volta a aparecer no canto inferior esquerdo, agora com um ícone de Wi-Fi ao lado. Se o App Lab voltar para a tela de seleção de placa, escolha a placa pelo nome; se ele pedir uma senha, é a senha do usuário `arduino` que você definiu no Laboratório 0. Se a placa não aparecer, confira se o computador e a placa estão na mesma rede Wi-Fi e feche e abra o App Lab.

Aviso: um adaptador USB-C simples sem Power Delivery não alimenta a placa. Se a placa apagar ao trocar o cabo, o hub não está entregando energia. Alguns splitters “PD + fone” para celular funcionam, mas hubs com PD e portas USB-A são a opção segura.

Parte B: Verificando a câmera

Passo 4: Abra um terminal SSH na placa:

```
ssh arduino@<nome-da-placa>.local
```

Passo 5: Liste os dispositivos de vídeo pelo nome, não só pelo nó de dispositivo:

```
v4l2-ctl --list-devices
```

Saída esperada, com a webcam presente (o nome da câmera e os números dos nós mudam de uma montagem para outra):

```
Qualcomm Venus video decoder (platform:qcom-venus):  
  /dev/video2  
  /dev/video3
```

```
See3CAM_CU30 (usb-xhci-hcd.2.auto-1):  
  /dev/video0  
  /dev/video1
```

Aviso: `ls /dev/video*` mostra dois dispositivos mesmo sem câmera nenhuma. Eles pertencem ao decodificador de vídeo Venus do processador Qualcomm, não a uma câmera. Use sempre `v4l2-ctl --list-devices` para

saber o que é câmera. A numeração dos nós muda conforme a ordem em que os dispositivos aparecem no boot; o caminho estável fica em `/dev/v4l/by-id/`, uma pasta que só existe enquanto há uma câmera conectada.

Passo 6: Veja os formatos, resoluções e taxas que a sua webcam oferece. No lugar de `/dev/video0`, use o **primeiro nó listado embaixo do nome da sua webcam** no Passo 5. Se você apontar para um nó do Venus, o comando responde sem erro, mas com os formatos do decodificador (NV12, QCOM Compressed), não os da câmera.

```
v4l2-ctl --list-formats-ext -d /dev/video0
```

Anote as combinações disponíveis para 640x480. Uma webcam típica oferece formato bruto (UYVY ou YUYV, dois jeitos de guardar brilho e cor sem compressão) a 30 FPS e MJPEG a 30 ou 60 FPS nessa resolução. Só é possível pedir o que a câmera oferece; o driver negocia a taxa mais próxima.

Parte C: Capturando imagens com o periférico Camera

Passo 7: No App Lab, crie um app vazio chamado `camera-test`, pelo botão **+** ao lado de **Apps**, sem Bricks. O periférico `Camera` faz parte do App Lab e não precisa de Brick.

Passo 8: Substitua o conteúdo de `python/main.py` por este script de captura única:

```
from arduino.app_utils import App
from arduino.app_peripherals.camera import Camera
from arduino.app_utils.image import compress_to_jpeg

# A primeira camera encontrada (USB antes de CSI) e usada. Sem camera, Levanta excecao.
camera = Camera(resolution=(640, 480), fps=30)
camera.start()

frame = camera.capture()          # numpy array (altura, largura, canais), ordem BGR
print("Formato:", frame.shape)    # (480, 640, 3)
print("Tipo:", frame.dtype)       # uint8
print(f"Tamanho em memoria: {frame.nbytes} bytes ({frame.nbytes/1024:.0f} KB)")

jpeg = compress_to_jpeg(frame=frame, quality=90)
with open("/app/captured_image.jpg", "wb") as f:  # /app e a pasta do app na placa
    f.write(jpeg.tobytes())
print(f"JPEG salvo: {jpeg.nbytes} bytes")

camera.stop()
App.run()  # mantem o app aberto ate voce clicar em Stop
```

Passo 9: Clique em Run e leia o console. O script faz o trabalho uma vez e o app fica aberto, sem imprimir mais nada, até você clicar em Stop. Saída esperada:

```
Formato: (480, 640, 3)
Tipo: uint8
Tamanho em memoria: 921600 bytes (900 KB)
JPEG salvo: 74895 bytes
```

O arquivo aparece na pasta do app na placa, em `/home/arduino/ArduinoApps/camera-test/captured_image.jpg`. Para copiá-lo ao computador:

```
scp arduino@<nome-da-placa>.local:/home/arduino/ArduinoApps/camera-test/captured_image.jpg .
```

Repare na diferença: 900 KB de pixels brutos viraram 75 KB de JPEG. Compressão é o que torna viável transmitir vídeo pela rede, mas o modelo de IA sempre trabalha com os pixels descomprimidos.

Passo 10: Meça o desempenho da captura. Substitua o `main.py` por:

```
from arduino.app_utils import App
from arduino.app_peripherals.camera import Camera
import time, statistics

def medir(fps_pedido, n=100):
    cam = Camera(resolution=(640, 480), fps=fps_pedido)
    cam.start()
    frame = cam.capture() # descarta o primeiro quadro (aquecimento)
    tempos = []
    for _ in range(n):
        t0 = time.perf_counter()
        frame = cam.capture() # bloqueia ate o proximo quadro, respeitando o fps
    tempos.append((time.perf_counter() - t0) * 1000)
    cam.stop()
    media = statistics.mean(tempos)
    fps_real = 1000 / media
    taxa_mb = frame.nbytes * fps_real / 1024 / 1024
    print(f"fps pedido {fps_pedido}: media {media:.1f} ms, {fps_real:.1f} FPS, "
          f"min {min(tempos):.1f}, max {max(tempos):.1f}, {taxa_mb:.1f} MB/s")

medir(30)
medir(60)
medir(10)
App.run()
```

Valores medidos em uma webcam UVC comum:

```
fps pedido 30: media 33.6 ms, 29.7 FPS, min 29.9, max 62.1, 26.1 MB/s
fps pedido 60: media 33.3 ms, 30.0 FPS, min 31.6, max 35.8, 26.4 MB/s
fps pedido 10: media 100.3 ms, 10.0 FPS, min 68.7, max 133.4, 8.8 MB/s
```

Observe o console: ao pedir 60 FPS o driver avisa FPS set to 30 instead of requested 60. A câmera só entrega as taxas que anuncia, e esta não anuncia 60. Ao pedir 10, a câmera é configurada na taxa mais próxima que ela tem (15 nesta webcam), e o `capture()` espera o tempo que falta para entregar só 10 quadros por segundo ao seu código. Por isso a medição mostra 10,0 FPS. Anote os seus três resultados.

Parte D: Entendendo os dados da imagem

Passo 11: Explore os pixels e os espaços de cor. Clique em Stop e substitua mais uma vez todo o `main.py` por:

```

from arduino.app_utils import App
from arduino.app_peripherals.camera import Camera
import cv2

camera = Camera(resolution=(640, 480), fps=30)
camera.start()
frame = camera.capture()
camera.stop()

altura, largura = frame.shape[:2]
pixel = frame[altura // 2, largura // 2]          # pixel central
print("Pixel central (BGR):", pixel)
print(f" Azul: {pixel[0]} Verde: {pixel[1]} Vermelho: {pixel[2]}")

cinza = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY) # 3 canais -> 1 canal
print(f"Colorida: {frame.shape} ({frame.nbytes} bytes)")
print(f"Cinza: {cinza.shape} ({cinza.nbytes} bytes)")
print(f"Economia: {(1 - cinza.nbytes / frame.nbytes) * 100:.0f}%")
App.run()

```

Saída esperada: a imagem em cinza tem 307.200 bytes, 67% a menos. Converter para escala de cinza é uma otimização comum em modelos de Edge AI que não dependem da cor.

Exemplo resolvido: tamanho do quadro = largura × altura × canais. Um quadro RGB 640×480 tem $640 \times 480 \times 3 = 921.600$ bytes (900 KB). Um quadro RGB 1280×720 tem $1280 \times 720 \times 3 = 2.764.800$ bytes (2,6 MB). Em escala de cinza, $640 \times 480 \times 1 = 307.200$ bytes (300 KB). Reduzido para 96×96 em cinza, um tamanho de entrada comum no Edge Impulse, $96 \times 96 \times 1 = 9.216$ bytes, cerca de 100 vezes menor que o quadro colorido original.

Perguntas de checkpoint

P1: Quantos bytes ocupa um único quadro RGB de 640x480? E um de 1280x720?

P2: Qual taxa de captura você mediu com `fps=30`? E por que pedir `fps=60` não mudou o resultado na sua câmera?

P3: Por que um modelo de Edge AI usaria imagens em escala de cinza de 96x96 em vez de coloridas de 640x480? Calcule a diferença de memória.

P4: Sem câmera conectada, `ls /dev/video*` ainda lista dois dispositivos. O que são, e como você confirma que uma webcam de verdade foi detectada?

Principais lições

- Um quadro RGB de 640×480 tem 900 KB. A 30 FPS, são 26 MB por segundo passando pela memória da placa.
- Escala de cinza reduz os dados em dois terços, e reduzir a resolução pela metade reduz os dados quatro vezes. A resolução tem efeito quadrático.

- A câmera só entrega as taxas que anuncia, e `capture()` respeita o fps configurado. Meça antes de assumir 30 FPS.
- No UNO Q, código de câmera roda como app do App Lab, dentro do container que já traz OpenCV e o periférico `Camera`. A placa fica no hub e é acessada pela rede.

Alternativa sem webcam: o celular como câmera

Se você não tem hub nem webcam, o smartphone pode fazer o papel da câmera nos laboratórios de visão ao vivo. O vídeo vai do celular para a placa pela rede Wi-Fi, e a placa pode continuar ligada ao computador pelo cabo USB-C. Você precisa do app **Arduino IoT Remote** no celular (Android ou iOS), de uma conta Arduino e do celular na mesma rede Wi-Fi da placa.

1. No App Lab, abra o exemplo “Detect Objects on Smartphone Camera”, que fica em **Inspirations**, na barra lateral, e clique em Run. Este app inicia mesmo sem nenhuma câmera ligada à placa.
2. Abra a interface web da placa em `http://<nome-da-placa>.local:7000`. Ela mostra um código QR.
3. No celular, abra o Arduino IoT Remote e entre com a sua conta Arduino. Em **Devices**, toque no + e escolha **Stream phone camera to UNO Q**.
4. Aponte o celular para o código QR da página. Quando o pareamento termina, a página troca o QR pelo vídeo do celular, com as caixas de detecção.

O código de IA é o mesmo do exemplo com webcam. A única diferença está na origem do vídeo: O `main.py` cria uma `WebSocketCamera` e a entrega ao Brick, `VideoObjectDetection(camera, ...)`. As Partes B, C e D deste laboratório (formatos da webcam, captura e pixels) só valem para a webcam USB; com o celular, leia o Laboratório 8 sem executá-lo (ele exige a webcam) e faça o Laboratório 9 na versão B.

O que entregar

1. A saída de `v4l2-ctl --list-devices` e os formatos de 640x480 da sua webcam.
2. A saída das três medições de captura (fps 30, 60 e 10).
3. A saída da exploração de pixels e a imagem `captured_image.jpg`.
4. Respostas escritas para as perguntas de checkpoint.

Solução de problemas

Problema	Solução
A placa não aparece no App Lab depois de trocar para o hub	Espere 30 s (a placa pode ter reiniciado). Confira se PC e placa estão na mesma rede Wi-Fi. Tente <code>ssh arduino@<nome>.local</code> ou pelo IP mostrado no App Lab.
A placa apaga ao conectar o hub	O hub não tem Power Delivery ou a fonte

Problema	Solução
	é fraca. Use hub com PD e fonte de 5 V/3 A.
<code>v4l2-ctl --list-devices</code> só mostra o Venus	A webcam não enumerou. Troque de porta no hub, teste outro cabo, e veja as linhas com <code>usb</code> na saída do comando <code>dmesg</code> . Alguns splitters só passam áudio.
CameraOpenError: No available cameras found ao rodar o app	A mensagem cobre dois casos. Ou a webcam não foi detectada (confira com <code>v4l2-ctl --list-devices</code>), ou outro app está usando a câmera (pare todos os apps no App Lab e rode de novo).
O app aparece como parado ou <code>failed</code> logo depois de imprimir a saída FPS muito baixo (< 10)	O script terminou sem chamar <code>App.run()</code> . Confira se a última linha está lá. Confira os formatos com <code>v4l2-ctl --list-formats-ext</code> . Reduza a resolução. Webcams baratas têm taxa nativa baixa em formato bruto.
ModuleNotFoundError: cv2 no terminal	Você rodou o script no Python do host via SSH. Rode como app do App Lab, onde o container já tem OpenCV.

Notas

Espaço para suas anotações.

Laboratório 8: O custo do modelo: tamanho, resolução e latência

Duração	Dificuldade	Pré-requisitos
90 min	Intermediário-Avançado (um pouco de matemática nova; vá passo a passo)	Laboratório 7 concluído; hub USB-C com webcam; acesso pela rede

Objetivo

Rodar quatro modelos de IA prontos no mesmo hardware, medir o tempo de inferência de cada um, e entender por que a escolha do modelo e da resolução de entrada define se um sistema de visão consegue trabalhar em tempo real.

O que você vai aprender

1. Medir o tempo de inferência de um modelo rodando no UNO Q por dois métodos independentes
2. Trocar o modelo embutido de um Brick pelo `app.yaml`
3. Relacionar resolução de entrada, número de classes e tipo de tarefa com a latência
4. Calcular a taxa máxima de quadros a partir do tempo de inferência e decidir se um modelo atende a uma meta de tempo real
5. Entender o papel de aceleradores de hardware e o que o UNO Q oferece hoje

Materiais necessários

Item	Detalhes
Arduino UNO Q 4GB	No hub USB-C com PD, acessado pela rede
Webcam USB	Do Laboratório 7. Este laboratório exige a webcam; com o celular como câmera, leia-o e use os números de referência
Computador com Arduino App Lab	E um terminal SSH

Visão geral do conceito

O Laboratório 7 mostrou o custo de mover pixels. Este laboratório mostra o custo de interpretá-los. Uma rede neural é uma enorme sequência de multiplicações e somas sobre grades de números. Quanto maior a imagem de entrada e quanto mais camadas e classes o modelo tem, mais operações por quadro. Esse custo aparece como tempo de inferência, e o tempo de inferência define o teto da taxa de quadros:

$$\text{FPS máximo} = 1000 \div \text{tempo de inferência em milissegundos}$$

Aceleradores de hardware. Todo chip moderno de Edge AI oferece alguma unidade dedicada a esse tipo de matemática: DSPs, GPUs ou NPUs. O processador Qualcomm QRB2210 do UNO Q traz uma GPU Adreno 702 e um DSP Hexagon. Não há uma NPU separada. Aceleradores executam centenas de multiplicações em paralelo e costumam entregar de 3 a 10 vezes mais velocidade que a CPU, com menos energia. Na versão atual do App Lab (Bricks 0.12.0), porém, os Bricks de visão executam o modelo em um único núcleo da CPU: o runner de modelos roda em um container sem acesso ao DSP nem à GPU. O que você mede neste laboratório é o desempenho de CPU. Isso não invalida nada: em produção, a primeira alavanca de desempenho é sempre a escolha do modelo, e é isso que você vai medir.

Os quatro modelos. O App Lab traz modelos prontos para os Bricks de visão. Quatro deles servem para uma comparação direta:

Modelo (model: no app.yaml)	Tarefa	Entrada	Classes
yolox-object-detection	Detecção de objetos (caixas)	416×416×3	80 (COCO)
mobilenet-image-classification	Classificação de imagem	224×224×3	1000 (ImageNet)
face-detection	Detecção de rostos (caixas)	320×256×1	1
person-classification	Classificação pessoa / não pessoa	224×224×3	2

Os dois primeiros são os padrões dos Bricks Video Object Detection e Video Image Classification. Os outros dois são escolhidos no `app.yaml`. Repare que eles variam em três eixos ao mesmo tempo: tamanho da entrada, tipo de tarefa (detecção é mais cara que classificação) e número de classes.

Como o Brick roda o modelo. Quando o app inicia, o App Lab sobe dois containers: o do seu `main.py` e um runner de modelos do Edge Impulse. A câmera é lida pelo seu container, os quadros vão por TCP para o runner, o runner roda o modelo e devolve os resultados por WebSocket, e o Brick chama o seu callback. O runner também publica uma página de preview em `http://<nome-da-placa>.local:4912` que mostra o vídeo, as detecções e o tempo de cada inferência. É esse número que você vai coletar.

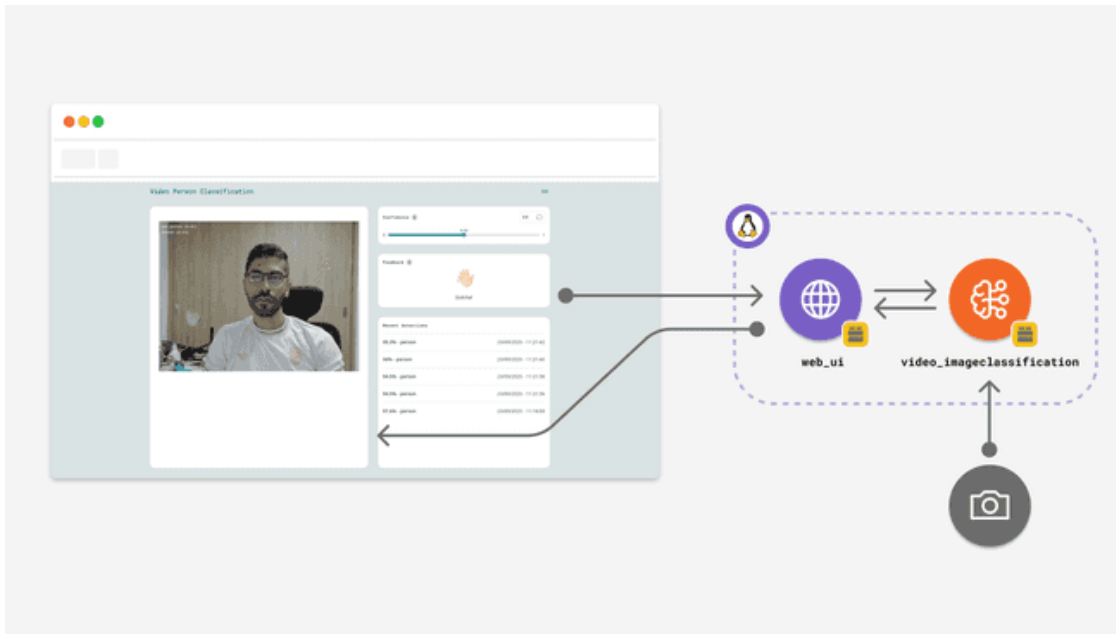


Figura 8.1: Classificação de pessoas em vídeo, inferência em tempo real no UNO Q

Procedimento

Parte A: Linha de base com o modelo padrão de detecção

Passo 1: No App Lab, abra **Inspirations**, na barra lateral, encontre “Detect Objects on Camera” e clique em **Copy and edit app**. Dê o nome `model-cost`. A lista de Bricks no `app.yaml` fica assim:

```
bricks:
  - arduino:video_object_detection
  - arduino:web_ui
```

O App Lab pode gravar cada linha com `: {}` no fim (`- arduino:web_ui: {}`); é a mesma coisa. Sem a chave `model:`, o Brick usa o YOLO-X nano.

Passo 2: Clique em Run. Espere o app iniciar; o console mostra `App started` no fim. O `main.py` deste exemplo não imprime as detecções no console: ele as envia para a página web em `:7000`.

Passo 3: Abra `http://<nome-da-placa>.local:4912` no navegador. Fique na frente da câmera. Abaixo do vídeo aparece “Time per inference: NNN ms”. Observe por 20 segundos e anote o valor típico. A frequência com que os resultados chegam você vai medir na Parte C.

Valor de referência medido: cerca de 370 ms por inferência.

Parte B: Trocando o modelo

Passo 4: Pare o app e troque o modelo do Brick pelo de rostos, que é uma detecção mais leve. No painel **Bricks**, clique em **Video Object Detection**, abra a aba **All models** e use o botão **Brick Configuration** para escolher o modelo de detecção de rostos. A escolha fica gravada no `app.yaml`, na chave `model:`, aninhada embaixo do Brick. Abra o arquivo e confira; a lista de Bricks tem de ficar assim (as linhas `name`, `icon` e `description` continuam como estavam):

```
bricks:
  - arduino:video_object_detection:
      model: face-detection
  - arduino:web_ui
```

Se preferir, edite o `app.yaml` direto, pelo App Lab ou pelo terminal SSH (`nano ~/ArduinoApps/model-cost/app.yaml`; salve com `Ctrl+O` e `Enter`, saia com `Ctrl+X`). Rode de novo e repita a leitura na página :4912. Referência: cerca de 100 ms. O rótulo detectado passa a ser `face`.

Passo 5: Agora troque o Brick de detecção pelo de classificação. Pare o app `model-cost`: só um app usa a câmera por vez. Crie um app vazio chamado `model-cost-cls`, pelo botão **+** ao lado de **Apps**, e adicione o Brick **Video Image Classification** pelo primeiro dos três ícones acima do painel Bricks. O `app.yaml` fica assim:

```
name: model-cost-cls
icon: 📷
description: "Custo do modelo, classificação"
bricks:
  - arduino:video_image_classification
```

Substitua todo o conteúdo de `python/main.py` por:

```
from arduino.app_utils import App
from arduino.app_bricks.video_imageclassification import VideoImageClassification
import time
```

```
stream = VideoImageClassification(confidence=0.3, debounce_sec=0.0)
contagem = 0
inicio = time.time()
```

```
def ao_classificar(resultados: dict):
    # resultados: {"rotulo": confianca, ...}, so os acima do limiar
    global contagem
    contagem += 1
    if contagem <= 3:
        print("Resultado:", resultados)
    if contagem % 30 == 0:
        taxa = contagem / (time.time() - inicio)
        print(f"{taxa:.1f} resultados/s")
```

```
stream.on_detect_all(ao_classificar)
App.run()
```

Rode e leia a página :4912. Este é o MobileNet-v2 de 1000 classes. Referência: cerca de 130 ms. Repare nos rótulos: um modelo de ImageNet tenta encaixar a cena em 1000 categorias e pode dar respostas curiosas.

Passo 6: Pare o app e troque para o modelo de pessoa, do mesmo jeito que no Passo 4. O app.yaml inteiro fica assim:

```
name: model-cost-cls
icon: 🤖
description: "Custo do modelo, classificação"
bricks:
  - arduino:video_image_classification:
      model: person-classification
```

Rode e leia. Referência: cerca de 27 ms, com resultados chegando cerca de 10 vezes por segundo (o console mostra a taxa a cada 30 resultados). O rótulo é person com você na frente da câmera e non person sem ninguém.

Parte C: Medindo pelo WebSocket

A página :4912 é prática, mas ler um número na tela não é um benchmark. O runner publica cada resultado, com o tempo de inferência, em um WebSocket. Um script que escute esse canal por 20 segundos dá média, mínimo e máximo.

Passo 7: Crie o arquivo ws_probe.py na pasta do app, na raiz, ao lado do app.yaml. O jeito mais direto é pelo terminal SSH da placa: abra o editor com o comando abaixo, cole o código, salve com Ctrl+O e Enter e saia com Ctrl+X.

```
nano ~/ArduinoApps/model-cost-cls/ws_probe.py
```

Depois copie o mesmo arquivo para o outro app:

```
cp ~/ArduinoApps/model-cost-cls/ws_probe.py ~/ArduinoApps/model-cost/
```

A sonda roda dentro do container do app, que já tem a biblioteca websockets e enxerga a pasta do app como /app:

```
# ws_probe.py: escuta o runner de modelos por 20 s e resume o tempo de inferencia
import asyncio, json, time, sys
```

```
URL = sys.argv[1] if len(sys.argv) > 1 else "ws://ei-video-obj-detection-runner:4912"
```

```
async def main():
    import websockets
    async with websockets.connect(URL, max_size=None) as ws:
        inicio = time.time()
        tempos = []
        n = 0
        while time.time() - inicio < 20:
            try:
                msg = await asyncio.wait_for(ws.recv(), timeout=5)
            except asyncio.TimeoutError:
                print("nenhuma mensagem em 5 s (ha alguem na frente da camera?)")
                continue
```

```

n += 1
dado = json.loads(msg)
if "timeMs" in dado:
    inferencia
    tempos.append(dado["timeMs"])
duracao = time.time() - inicio
print(f"{n} mensagens em {duracao:.1f} s = {n / duracao:.2f} resultados/s")
if tempos:
    print(f"inferencia: media {sum(tempos) / len(tempos):.1f} ms, "
          f"min {min(tempos)} ms, max {max(tempos)} ms (n={len(tempos)})")

asyncio.run(main())

```

Passo 8: Com o app rodando, execute a sonda de dentro do container do app, pelo terminal SSH da placa. O nome do container é o **nome da pasta do app** em ~/ArduinoApps/ seguido de -main-1; ele não vem do campo name: do app.yaml. Para ver o nome exato, rode `docker ps --format '{{.Names}}'` com o app em execução. O endereço do runner depende do Brick:

```

# app com video_object_detection
docker exec model-cost-main-1 python3 /app/ws_probe.py ws://ei-video-obj-detection-runner:4912

# app com video_image_classification
docker exec model-cost-cls-main-1 python3 /app/ws_probe.py ws://ei-video-classification-runner:4912

```

Saída de referência para o modelo de pessoa:

```

202 mensagens em 20.1 s = 10.04 resultados/s
inferencia: media 27.0 ms, min 19 ms, max 34 ms (n=201)

```

Passo 9: Repita a sonda para os quatro modelos e preencha a tabela. Para cada modelo: pare o app, troque o modelo, rode, fique na frente da câmera e execute a sonda.

Parte D: Compare e analise

Passo 10: Tabela de comparação. Preencha com os seus valores; a coluna de referência mostra o que foi medido em uma placa UNO Q com webcam, com a câmera aberta pelo Brick no padrão dele: 640x480 a 10 quadros por segundo.

Modelo	Entrada	Inferência medida (ms)	Referência (ms)	FPS máx. teórico (1000 ÷ ms)	Resultados/ s observados
YOLO-X nano	416×416×3		370	2,7	1,6
MobileNet- v2	224×224×3		130	7,7	3,1
ImageNet					
Face	320×256×1		101	9,9	5,4

Modelo	Entrada	Inferência medida (ms)	Referência (ms)	FPS máx. teórico (1000 ÷ ms)	Resultados/ s observados
detection					
Person	224×224×3		27	37	10,0
classification					

Passo 11: Compare a coluna “FPS máximo teórico” com “resultados/s observados”. O teórico considera só a inferência. O observado inclui captura, transferência entre containers, redimensionamento e entrega do resultado: é o custo do pipeline. E há um teto que não depende do modelo: o Brick abre a câmera com `Camera()`, cujo padrão é `fps=10`. Como você viu no Laboratório 7, o `capture()` respeita esse valor, então chegam ao modelo no máximo 10 quadros por segundo. Por isso o classificador de pessoa, que daria 37 FPS, para em 10 resultados por segundo.

Para confirmar que o teto é da câmera, e não do modelo, entregue ao Brick uma câmera a 30 FPS no `main.py` do app `model-cost-cls` e meça de novo. A primeira linha abaixo vai junto dos outros imports, no topo do arquivo; as outras duas substituem a linha `stream = VideoImageClassification(confidence=0.3, debounce_sec=0.0)`:

```
from arduino.app_peripherals.camera import Camera

camera = Camera(resolution=(640, 480), fps=30)
stream = VideoImageClassification(camera=camera, confidence=0.3, debounce_sec=0.0)
```

Exemplo resolvido: suponha que a inferência do YOLO-X nano leve 370 ms e a do classificador de pessoa leve 27 ms. FPS máximo = $1000 \div \text{ms}$ por quadro. YOLO-X: $1000 \div 370 = 2,7$ FPS. Classificador: $1000 \div 27 = 37$ FPS. Uma câmera de segurança que precise de 10 FPS não consegue usar o YOLO-X neste hardware, e consegue usar o classificador com folga, ainda que na prática a câmera a 10 quadros por segundo limite a taxa a 10 resultados por segundo. Fator entre os dois modelos: $370 \div 27 = 13,7$ vezes. É esse número, o fator de velocidade entre alternativas no mesmo hardware, que engenheiros reportam ao escolher um modelo.

Passo 12: Discuta: o que o YOLO-X entrega que o classificador de pessoa não entrega? (Caixas, 80 classes, várias instâncias por quadro.) Quando vale pagar 370 ms por isso?

Perguntas de checkpoint

P1: Qual foi o fator de velocidade entre o modelo mais lento e o mais rápido que você mediu? Ele bateu com a referência de cerca de 14 vezes?

P2: Um modelo leva 130 ms por quadro. Qual é a taxa máxima teórica? Ele atende uma aplicação que precisa de 10 FPS? E de 5 FPS?

P3: O modelo de pessoa tem a mesma entrada (224×224×3) que o MobileNet-v2 de ImageNet, mas roda cerca de 5 vezes mais rápido. O que explica a diferença, se a resolução é igual?

P4: Por que a taxa observada de resultados por segundo fica abaixo de 1000 ÷ tempo de inferência? E por que o modelo de pessoa não passa de 10 resultados por segundo?

Principais lições

- O tempo de inferência define o teto de FPS. Medir esse tempo é o primeiro passo de qualquer projeto de visão embarcada.
- No mesmo hardware, a escolha do modelo muda a latência em mais de dez vezes. Resolução de entrada, tipo de tarefa e número de classes são as três alavancas.
- Os Bricks de visão do App Lab rodam o modelo em um núcleo da CPU. Aceleradores como DSP e GPU existem no chip, mas não estão expostos nesta versão; a comparação justa hoje é entre modelos, não entre unidades de processamento.
- O pipeline (captura, transferência, pós-processamento) tem custo próprio, e a taxa da câmera é um teto: com o padrão de 10 quadros por segundo, nenhum modelo entrega mais de 10 resultados por segundo. Meça o sistema inteiro, não só o modelo.

O que entregar

1. Os `app.yaml` das quatro configurações (um por modelo) e o `main.py` de classificação.
2. A tabela da Parte D preenchida com as suas medições, pelos dois métodos (página :4912 e sonda).
3. Respostas escritas para as perguntas de checkpoint.

Solução de problemas

Problema	Solução
[ERROR] No Camera Device Found ao clicar em Run, e o app nem inicia	A placa não está vendo nenhuma câmera. Os Bricks de vídeo conferem isso antes de subir o app. Verifique o hub e a webcam com <code>v4l2-ctl --list-devices</code> (Laboratório 7).
A página :4912 não abre	O app precisa estar rodando. Espere o container do runner ficar “Healthy” (aparece no log de start). Use o IP da placa se <code>.local</code> falhar.
“Time per inference” não aparece ou fica	Não há nada acima do limiar de

Problema	Solução
parado	confiança na cena. Fique na frente da câmera; para o modelo de rostos, aproxime o rosto.
<code>docker exec</code> responde “No such container”	Confira o nome com <code>docker ps --format '{{.Names}}'</code> . É o nome da pasta do app seguido de <code>-main-1</code> . O app precisa estar rodando.
<code>python3: can't open file '/app/ws_probe.py'</code>	O arquivo não está na pasta deste app. Copie-o para <code>~/ArduinoApps/<pasta-do-app>/</code> .
A sonda imprime “nenhuma mensagem em 5 s”	O runner só publica quando há resultado acima do limiar. Coloque algo detectável na frente da câmera. Confira o endereço do runner (detecção vs classificação).
Resultados/s muito abaixo da referência	Outro app pode estar rodando e disputando a CPU. Pare tudo e rode um app por vez. Confira a temperatura (Laboratório 13).
Modelo trocado no <code>app.yaml</code> não faz efeito	Pare e inicie o app de novo. O runner só carrega o modelo no start.

Notas

Espaço para suas anotações.

Laboratório 9: Aplicação de visão em tempo real

Duração	Dificuldade	Pré-requisitos
90-120 min	Intermediário-Avançado	Versão A: Laboratório 8 concluído, hub USB-C com webcam e acesso pela rede. Versão B: Laboratório 8 lido (ele exige webcam) e celular com o app Arduino IoT Remote

Objetivo

Construir uma aplicação completa de visão que processa continuamente os quadros da câmera, detecta objetos, mostra o resultado em uma interface web e aciona o microcontrolador, tudo rodando no UNO Q.

O que você vai aprender

1. Montar uma aplicação de detecção em tempo real a partir do exemplo oficial do App Lab
2. Combinar o Brick Video Object Detection com o Brick Web UI
3. Medir a taxa de resultados sustentada em operação prolongada
4. Ligar o resultado da inferência a um atuador físico pela Bridge
5. Escolher entre o modelo de detecção geral e um modelo leve conforme a meta de tempo real

Materiais necessários

Item	Detalhes
Arduino UNO Q 4GB	No hub USB-C com PD, acessado pela rede
Webcam USB (versão A)	Do Laboratório 7
Celular com Arduino IoT Remote (versão B)	No lugar da webcam e do hub; veja “Alternativa sem webcam” no Laboratório 7
Computador com Arduino App Lab	E navegador para a interface web

Visão geral do conceito

Um sistema de visão em tempo real junta tudo o que você fez até aqui: captura de câmera (Laboratório 7), inferência com um modelo medido (Laboratório 8), interface web (Laboratório 2) e comando do MCU pela Bridge (Laboratório 4). Um pipeline de

detecção funcionando com saída física é o entregável que muitos projetos de Edge AI levam meses para alcançar. Este laboratório comprime esse caminho em uma sessão, porque o App Lab já traz o pipeline pronto no Brick Video Object Detection.

O que significa “tempo real” aqui. O Laboratório 8 mostrou que o modelo padrão de detecção, o YOLO-X nano, leva cerca de 370 ms por quadro neste hardware, o que resulta em cerca de 1,5 resultado por segundo. Isso basta para reagir a uma pessoa que entra numa sala, mas não para acompanhar um objeto em movimento. O classificador de pessoa, a 10 resultados por segundo, dá uma resposta fluida, ao custo de não dizer onde a pessoa está nem reconhecer outros objetos. Você vai construir com o primeiro e depois trocar para o segundo, e medir os dois.

O caminho de um quadro. Câmera → container do app → runner de modelos → resultado por WebSocket → seu callback → `send_message` para o navegador e `Bridge.call` para o MCU. Cada etapa soma latência. O objetivo do laboratório é fechar esse ciclo e medir quanto ele custa.

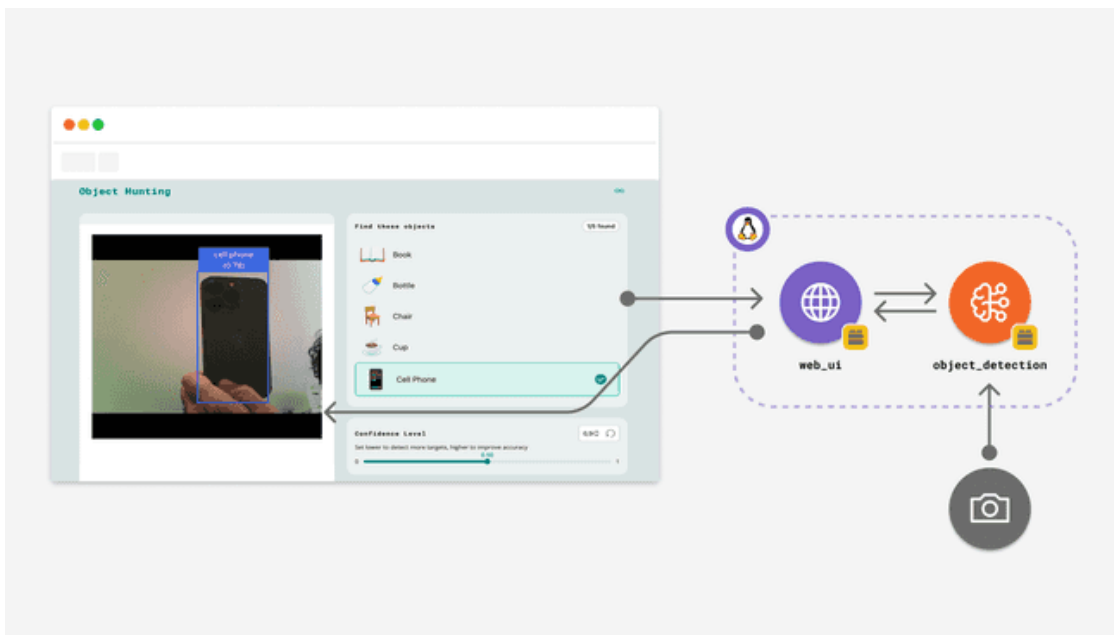


Figura 9.1: Exemplo “Object Hunting” do App Lab: a câmera alimenta o Brick de detecção e a interface web mostra as caixas

Procedimento

Este laboratório tem duas versões de entrada de vídeo. O código de IA é o mesmo nas duas.

Versão	Entrada de vídeo	Exemplo de partida	Montagem
A	Webcam USB	“Detect Objects on Camera”	Placa no hub USB-C com PD, acesso pela rede

Versão	Entrada de vídeo	Exemplo de partida	Montagem
B	Celular com o app Arduino IoT Remote	“Detect Objects on Smartphone Camera”	(Laboratório 7) Placa no cabo do computador ou na rede; celular na mesma rede Wi-Fi

Siga a versão A se você tem webcam. Os passos valem para as duas; onde há diferença, o texto avisa.

Parte A: Partindo do exemplo oficial

O Brick Web UI exige um `assets/index.html`, e uma página com vídeo, caixas e lista de detecções não é trivial de escrever. O exemplo oficial já traz tudo isso.

Passo 1: No App Lab, abra **Inspirations**, na barra lateral, encontre o exemplo da sua versão (“Detect Objects on Camera” na A, “Detect Objects on Smartphone Camera” na B) e clique em **Copy and edit app**. Chame o app de `vision-app`.

Passo 2: Examine os arquivos. O `app.yaml` declara os dois Bricks:

```
bricks:
  - arduino:video_object_detection
  - arduino:web_ui
```

O `python/main.py` do exemplo cria os dois Bricks, registra um callback que reenvia cada detecção para o navegador e liga o controle de confiança da página ao método `override_threshold`. Na versão A ele é assim (o arquivo real não tem os comentários e usa os nomes `key` e `values` no lugar de `label` e `instances`):

```
from arduino.app_utils import App
from arduino.app_bricks.web_ui import WebUI
from arduino.app_bricks.video_objectdetection import VideoObjectDetection
from datetime import datetime, UTC
```

```
ui = WebUI()
detection_stream = VideoObjectDetection(confidence=0.5, debounce_sec=0.0)
```

```
# O controle deslizante da pagina envia "override_th"; repassamos ao Brick
ui.on_message("override_th", lambda sid, threshold: detection_stream.override_threshold(threshold))
```

```
def send_detections_to_ui(detections: dict):
    # detections: {"person": [{"confidence": 0.94, "bounding_box_xyxy": (x1, y1, x2, y2)}], ...}
    for label, instances in detections.items():
        for det in instances:
            ui.send_message("detection", message={
                "content": label,
                "confidence": det.get("confidence"),
                "timestamp": datetime.now(UTC).isoformat(),
            })
```

```
detection_stream.on_detect_all(send_detections_to_ui)
```

```
App.run()
```

A pasta `assets/` traz o `index.html`, o `app.js` e os estilos da página.

Na versão B o arquivo tem a mesma função `send_detections_to_ui` e mais algumas linhas no começo, que criam a câmera remota e o código de pareamento. O Brick recebe essa câmera e fica na variável `detection`, no lugar de `detection_stream`:

```
camera = WebSocketCamera(secret=secret, encrypt=True, resolution=resolution)
detection = VideoObjectDetection(camera, confidence=0.5, debounce_sec=0.0)
```

Passo 3: Clique em Run. Abra `http://<nome-da-placa>.local:7000`.

- **Versão A:** você deve ver o vídeo ao vivo com caixas coloridas sobre os objetos, um controle de confiança e a lista “Recent detections” à direita.
- **Versão B:** a página mostra um código QR e, embaixo dele, uma senha de seis dígitos (Password). O QR já leva essa senha; se o app do celular pedir, digite a que está na página. No celular, abra o Arduino IoT Remote, vá em **Devices**, toque no +, escolha **Stream phone camera to UNO Q** e aponte para o QR. A página troca o QR pelo vídeo do celular, com as caixas.

Aponte a câmera para pessoas e objetos comuns e observe a lista.

Dica: o parâmetro `debounce_sec` é o intervalo mínimo entre dois disparos do mesmo rótulo. Com `0.0`, o callback recebe todos os resultados. Com `2.0`, uma pessoa parada na frente da câmera gera um evento a cada 2 segundos, o que é útil quando o callback faz algo caro, como enviar uma notificação.

Parte B: Medindo o desempenho de ponta a ponta

Passo 4: Adicione um contador de resultados ao callback. Clique em Stop. Na versão A, substitua todo o conteúdo de `python/main.py` por este arquivo. As mudanças em relação ao exemplo são o `import time`, as duas variáveis `contagem` e `inicio` e o trecho do contador dentro da função:

```
# python/main.py - deteccao ao vivo com contador de resultados (versao A, webcam)
from arduino.app_utils import App
from arduino.app_bricks.web_ui import WebUI
from arduino.app_bricks.video_objectdetection import VideoObjectDetection
from datetime import datetime, UTC
import time
```

```
ui = WebUI()
detection_stream = VideoObjectDetection(confidence=0.5, debounce_sec=0.0)
```

```
ui.on_message("override_th", lambda sid, threshold: detection_stream.override_threshold(threshold))
```

```

contagem = 0
inicio = time.time()

def send_detections_to_ui(detections: dict):
    global contagem
    contagem += 1

    for label, instances in detections.items():
        for det in instances:
            ui.send_message("detection", message={
                "content": label,
                "confidence": det.get("confidence"),
                "timestamp": datetime.now(UTC).isoformat(),
            })

    if contagem % 30 == 0:
        decorrido = time.time() - inicio
        print(f"Desempenho: {contagem / decorrido:.1f} resultados/s "
              f"({1000 * decorrido / contagem:.0f} ms por resultado) | total {contagem}")

detection_stream.on_detect_all(send_detections_to_ui)

App.run()

```

Na versão B, substitua todo o conteúdo de `python/main.py` por este arquivo. Ele é o exemplo do celular com as mesmas mudanças do contador; não tente colar as linhas à mão no arquivo original, que usa outro recuo (2 espaços) e recusa linhas com 4:

```

# python/main.py - deteccao ao vivo com contador de resultados (versao B, celular)
import secrets
import string
import time
from datetime import datetime, UTC

from arduino.app_utils import App
from arduino.app_bricks.web_ui import WebUI
from arduino.app_bricks.video_objectdetection import VideoObjectDetection
from arduino.app_peripherals.camera import WebSocketCamera

# Senha de seis digitos do pareamento; ela vai dentro do codigo QR
secret = ''.join(secrets.choice(string.digits) for _ in range(6))

ui = WebUI()
camera = WebSocketCamera(secret=secret, encrypt=True, resolution=(480, 640)) # video em
pe, como o celular
camera.on_status_changed(lambda evt_type, data: ui.send_message(evt_type, data))

detection_stream = VideoObjectDetection(camera, confidence=0.5, debounce_sec=0.0)

# Quando o navegador abre a pagina, ela recebe os dados para desenhar o codigo QR
ui.on_connect(lambda sid: ui.send_message("welcome", {
    "client_name": camera.name, "secret": secret, "status": camera.status,
    "protocol": camera.protocol, "ip": camera.ip, "port": camera.port}))
ui.on_message("override_th", lambda sid, threshold: detection_stream.override_threshold(threshold))

```

```

contagem = 0
inicio = time.time()

def send_detections_to_ui(detections: dict):
    global contagem
    contagem += 1

    for label, instances in detections.items():
        for det in instances:
            ui.send_message("detection", {
                "content": label,
                "confidence": det.get("confidence"),
                "timestamp": datetime.now(UTC).isoformat(),
            })

    if contagem % 30 == 0:
        decorrido = time.time() - inicio
        print(f"Desempenho: {contagem / decorrido:.1f} resultados/s "
              f"({1000 * decorrido / contagem:.0f} ms por resultado) | total {contagem}")

detection_stream.on_detect_all(send_detections_to_ui)

App.run()

```

Passo 5: Rode por pelo menos 100 resultados com alguém na frente da câmera e anote a taxa sustentada que o console imprime. Referência com o YOLO-X nano e webcam: cerca de 1,3 a 1,6 resultados por segundo. O callback só é chamado quando há detecção acima do limiar, então a cena precisa ter algo detectável.

Parte C: Fechando o ciclo com o MCU

Os exemplos de visão não têm sketch: a cópia que você fez veio só com `app.yaml`, `assets/` e `python/`. Para comandar o LED, o app precisa de uma pasta `sketch/` com **dois** arquivos, `sketch.ino` e `sketch.yaml`. Se faltar um deles, o app não inicia.

Passo 6: Copie a pasta `sketch/` do exemplo Blink para o seu app. Clique em Stop e, em um terminal na placa (SSH, como no Laboratório 7, ou `adb shell` se a placa estiver no cabo), rode:

```

cp -r /var/lib/arduino-app-cli/examples/core-and-foundational/01-led-blink/03-blinking-an-
led-from-python/sketch ~/ArduinoApps/vision-app/
ls ~/ArduinoApps/vision-app/sketch

```

O `ls` deve mostrar `sketch.ino` e `sketch.yaml`. No App Lab, feche e abra o app `vision-app` para a pasta aparecer no painel **Files**. Abra `sketch/sketch.ino` e substitua o conteúdo por este sketch, que expõe o LED com o nome `set_led_state`:

```

#include <Arduino_RouterBridge.h>

void set_led_state(bool on) {
    digitalWrite(LED_BUILTIN, on ? LOW : HIGH);    // LED_BUILTIN acende em LOW
}

void setup() {

```

```

pinMode(LED_BUILTIN, OUTPUT);
digitalWrite(LED_BUILTIN, HIGH);           // começa apagado
Bridge.begin();
Bridge.provide("set_led_state", set_led_state);
}

```

```
void loop() {}
```

Passo 7: No python/main.py, importe a Bridge e acione o LED quando houver uma pessoa. São duas mudanças: Bridge entra na linha from arduino.app_utils import App, que passa a ser from arduino.app_utils import App, Bridge, e quatro linhas novas entram no callback, logo depois de contagem += 1. Na versão A o arquivo completo fica assim:

```

# python/main.py - deteccao ao vivo, contador e LED (versao A, webcam)
from arduino.app_utils import App, Bridge
from arduino.app_bricks.web_ui import WebUI
from arduino.app_bricks.video_objectdetection import VideoObjectDetection
from datetime import datetime, UTC
import time

ui = WebUI()
detection_stream = VideoObjectDetection(confidence=0.5, debounce_sec=0.0)

ui.on_message("override_th", lambda sid, threshold: detection_stream.override_threshold(threshold))

contagem = 0
inicio = time.time()

def send_detections_to_ui(detections: dict):
    global contagem
    contagem += 1

    pessoa = "person" in detections
    Bridge.call("set_led_state", pessoa)           # AGIR: LED acende com pessoa na cena
    if pessoa:
        print("PESSOA DETECTADA - LED ligado")

    for label, instances in detections.items():
        for det in instances:
            ui.send_message("detection", message={
                "content": label,
                "confidence": det.get("confidence"),
                "timestamp": datetime.now(UTC).isoformat(),
            })

    if contagem % 30 == 0:
        decorrido = time.time() - inicio
        print(f"Desempenho: {contagem / decorrido:.1f} resultados/s "
              f"({1000 * decorrido / contagem:.0f} ms por resultado) | total {contagem}")

detection_stream.on_detect_all(send_detections_to_ui)

App.run()

```

Na versão B, faça as mesmas duas mudanças à mão no arquivo que você colocou no Passo 4: a linha do import e as quatro linhas depois de `contagem += 1`, com o mesmo recuo de quatro espaços.

Clique em Run. A primeira partida com o sketch novo demora mais, porque o App Lab compila e grava o MCU. Teste: entre e saia do campo da câmera. O LED vermelho da placa (LED3) deve acompanhar, com o atraso de cerca de um resultado.

Dica: `Bridge.call` espera a resposta do MCU (cerca de 7 ms). O Laboratório 4 avisou para nunca chamar `Bridge.call` dentro de um callback RPC. Aquela regra vale para as funções que o Python registra com `Bridge.provide` e que o MCU chama. O callback de um Brick é outra coisa: quem o chama é o próprio Python, quando chega um resultado do modelo, e nele o `Bridge.call` funciona (medido: cerca de 8 ms por chamada). Se a aplicação não precisa da resposta, `Bridge.notify("set_led_state", pessoa)` faz o mesmo sem esperar. Note que, quando não há nenhuma detecção, o callback não é chamado, então o LED só apaga no próximo resultado sem `person`. O Laboratório 14 resolve isso com um agente que mantém estado.

Parte D: Trocando para o modelo leve (versão A)

Esta parte usa o exemplo “Person Classifier on Camera”, que só existe para webcam. Com o celular, leia os passos e compare com os números de referência.

Passo 8: Pare o `vision-app`. Crie um segundo app, `vision-app-fast`, copiando o exemplo “Person Classifier on Camera”, também em **Inspirations**. Ele já vem com o `app.yaml` apontando para o modelo de pessoa:

```
bricks:
- arduino:video_image_classification:
    model: person-classification
- arduino:web_ui
```

Copie a pasta do sketch do app anterior, que já está pronta, pelo terminal da placa:

```
cp -r ~/ArduinoApps/vision-app/sketch ~/ArduinoApps/vision-app-fast/
```

O callback de classificação recebe um formato diferente do de detecção: um dicionário de rótulo para confiança, sem caixas. Com o modelo de pessoa chega `{"person": 0.90}` quando há alguém e `{"non person": 0.97}` quando não há. Substitua todo o conteúdo de `python/main.py` por:

```
# python/main.py – classificador de pessoa, contador e LED
from arduino.app_utils import App, Bridge
from arduino.app_bricks.web_ui import WebUI
from arduino.app_bricks.video_imageclassification import VideoImageClassification
from datetime import datetime, UTC
import json
import time
```

```
ui = WebUI()
```

```

detection_stream = VideoImageClassification(confidence=0.5, debounce_sec=0.0)

ui.on_message("override_th", lambda sid, threshold: detection_stream.override_threshold(threshold))

contagem = 0
inicio = time.time()

def send_detections_to_ui(classifications: dict):
    # classifications: {"person": 0.90} ou {"non person": 0.97}
    global contagem
    contagem += 1

    pessoa = "person" in classifications
    Bridge.call("set_led_state", pessoa)

    entradas = [{"content": label, "confidence": confidence,
                  "timestamp": datetime.now(UTC).isoformat()}
                 for label, confidence in classifications.items()]
    ui.send_message("classifications", message=json.dumps(entradas))

    if contagem % 30 == 0:
        decorrido = time.time() - inicio
        print(f"Desempenho: {contagem / decorrido:.1f} resultados/s | total {contagem}")

detection_stream.on_detect_all(send_detections_to_ui)

App.run()

```

Rode e abra a página :7000. Referência: cerca de 10 resultados por segundo. Compare a resposta do LED com a da Parte C: agora ele apaga logo que você sai da cena, porque o resultado non person chega dez vezes por segundo.

Exemplo resolvido: os limiares de “tempo real” dependem da aplicação. Uma câmera de segurança precisa de uns 15 FPS para acompanhar uma pessoa andando; análise esportiva pede 25 ou mais; inspeção industrial de alta velocidade, 60 ou mais. Com o YOLO-X nano a 1,5 resultados por segundo, você detecta presença, mas não acompanha movimento. Com o classificador de pessoa a 10 resultados por segundo, você chega perto da barra de uma câmera de segurança, ao custo de perder as caixas e os outros 79 rótulos. Esses 10 por segundo são o teto da câmera, que o Brick abre a 10 quadros por segundo (Laboratório 8); o modelo sozinho daria 37. Pergunte sempre: para onde vai o tempo? Divida o ciclo em captura, inferência e ação. A maior fatia é onde se otimiza primeiro; no YOLO-X, é a inferência.

Perguntas de checkpoint

P1: Qual taxa sustentada você obteve com o YOLO-X nano e com o classificador de pessoa? Qual dos dois atende uma câmera de segurança que precise de 10 resultados por segundo?

P2: O que faz o parâmetro `debounce_sec`, e em que aplicação você o colocaria acima de zero?

P3: Na Parte C, o LED só apaga quando chega um resultado sem `person`. O que acontece se a pessoa sair da cena e nada mais for detectado? Como você resolveria?

P4: Descreva duas maneiras de aumentar a taxa de resultados se ela estiver baixa demais para a sua aplicação. Considere o modelo e a taxa da câmera.

Principais lições

- É neste laboratório que as peças convergem: câmera, inferência, interface web, decisão e atuação física em um pipeline contínuo na placa.
- Parta do exemplo oficial: a interface web com vídeo e caixas já existe, e o Brick Web UI exige um `index.html`.
- O modelo define a taxa: cerca de 1,5 resultados por segundo com detecção geral, cerca de 10 com classificação de pessoa. Escolha pelo requisito da aplicação, não pelo modelo mais completo.
- Teste sustentado importa tanto quanto o pico: um sistema que rende bem por um minuto e cai depois de cinco, por aquecimento, não serve para implantação. O Laboratório 13 mede isso.

O que entregar

1. O `main.py` completo com contador e ação no LED (Passo 7), e o `sketch.ino`.
2. As taxas sustentadas medidas com os dois modelos (100 resultados ou mais cada).
3. Uma captura de tela da interface web mostrando detecções ao vivo.
4. Respostas escritas para as perguntas de checkpoint.

Solução de problemas

Problema	Solução
<code>[ERROR] No Camera Device Found</code> ao clicar em Run, e o app nem inicia	A placa não está vendo nenhuma câmera (versão A). Confira o hub e a webcam com <code>v4l2-ctl --list-devices</code> .
<code>sketch folder is incomplete: both sketch.ino and sketch.yaml are required</code>	A pasta <code>sketch/</code> do app está sem um dos dois arquivos. Refaça a cópia do Passo 6.
A interface web não mostra vídeo	Versão A: confira se a câmera está detectada (<code>v4l2-ctl --list-devices</code>). Versão B: refaça o pareamento pelo código QR e confira se o celular está na mesma rede Wi-Fi. Espere o runner ficar “Healthy” no log.
<code>RuntimeError</code> sobre <code>index.html</code> ao iniciar	O Brick Web UI precisa de

Problema	Solução
	<code>assets/index.html</code> . Copie o exemplo em vez de criar o app do zero.
Nenhum resultado no console	O callback só dispara com detecção acima do limiar. Fique na frente da câmera ou reduza <code>confidence</code> .
Taxa muito baixa (< 1 por segundo)	Pare outros apps. Verifique a temperatura da CPU (Laboratório 13). Considere o modelo leve da Parte D.
Function not found ao chamar <code>set_led_state</code>	O nome em <code>Bridge.provide</code> do sketch tem que ser idêntico ao de <code>Bridge.call</code> . Confira se o sketch compilou (log do Run).
Detecções imprecisas	O modelo padrão é genérico. Um modelo próprio, treinado no Edge Impulse (Laboratório 11), resolve casos específicos.

Notas

Espaço para suas anotações.

Laboratório 10: Entendendo o tamanho e as restrições do modelo

Duração	Dificuldade	Pré-requisitos
45-60 min	Intermediário-Avançado (um pouco de matemática nova; vá passo a passo)	Laboratório 8 concluído. Não usa câmera

Objetivo

Entender por que modelos de aprendizado de máquina são grandes, calcular os requisitos de memória de arquiteturas comuns e descobrir por que os modelos precisam ser otimizados antes de ir para um dispositivo de borda como o UNO Q.

Mudança de marcha: os Laboratórios 1 a 9 usaram modelos de IA como ferramentas prontas. Os Laboratórios 10 a 12 abrem a caixa: por que os modelos são grandes, como treinar o seu, como reduzi-lo e qual é o custo disso. Você vai trabalhar com números, cálculos e análise de compromissos, mais do que com aplicações rodando. Esse raciocínio analítico é o que separa quem implanta IA de quem apenas a usa.

O que você vai aprender

1. Calcular os requisitos de memória de arquiteturas comuns de redes neurais
2. Comparar o tamanho de modelos em FP32 e em INT8
3. Entender a relação entre resolução de entrada e uso de memória
4. Avaliar quais modelos cabem em dispositivos de borda com diferentes restrições de memória
5. Ler o tamanho real dos modelos instalados na sua placa

Materiais necessários

Item	Detalhes
Arduino UNO Q 4GB	Conectada por USB ou pela rede
Computador com terminal	<code>adb shell</code> ou SSH na placa

Visão geral do conceito

Uma rede neural é feita de camadas, e cada camada contém “pesos”: números que o modelo ajusta durante o treino para acertar mais. Um modelo de imagem típico tem milhões desses pesos. Por padrão eles são armazenados como números de ponto flutuante de 32 bits (FP32), que dão alta precisão mas custam 4 bytes cada.

Uma conta simples mostra o problema:

- MobileNetV2: 3,4 milhões de parâmetros × 4 bytes = 13,6 MB (FP32)
- ResNet-50: 25,6 milhões de parâmetros × 4 bytes = 102 MB (FP32)
- YOLOv8-nano: 3,2 milhões de parâmetros × 4 bytes = 12,8 MB (FP32)

O UNO Q tem cerca de 3,6 GiB de RAM disponíveis para o Linux, então esses modelos cabem, mas consomem recursos que fazem falta ao sistema, aos buffers de câmera e à sua aplicação. Em dispositivos de borda menores, com dezenas ou centenas de kilobytes de RAM, esses modelos são impossíveis sem otimização.

De onde vêm os milhões. As primeiras camadas aprendem a reconhecer coisas simples, como bordas e texturas. As intermediárias combinam isso em formas. As finais juntam formas em uma classificação. Cada camada tem seu conjunto de pesos. Uma única camada convolucional com filtros 3×3, 64 canais de entrada e 128 de saída tem $3 \times 3 \times 64 \times 128 = 73.728$ parâmetros. Empilhe dezenas dessas camadas, como no ResNet-50, e você chega aos milhões.

Pesos não são tudo. Enquanto o modelo roda, cada camada produz um resultado intermediário, chamado ativação, que a próxima camada consome. Essas ativações ficam na RAM até serem usadas. Para uma entrada 224×224 passando por uma camada de 64 filtros, o mapa de ativação tem $224 \times 224 \times 64 = 3,2$ milhões de valores. A memória total para executar um modelo é: pesos + buffer de entrada + maior ativação intermediária + buffer de saída. Por isso a otimização para dispositivos de borda ataca todos esses termos, não só a contagem de pesos.

O arquivo que roda na placa. Os modelos que o App Lab executa chegam como arquivos .eim do Edge Impulse. Um .eim não é só o modelo: é um executável que traz o runtime de inferência embutido. Isso explica por que até modelos pequenos aparecem com mais de 13 MB no disco, como você vai ver na Parte C.

Procedimento

Parte A: Calculando os requisitos de memória

Passo 1: Escreva um script para analisar tamanhos de modelo. Ele é Python puro e roda em qualquer lugar: no seu computador, no terminal da placa ou em um app do App Lab. No terminal da placa, crie o arquivo com `nano tamanho.py`, cole o código, salve com `Ctrl+O` e `Enter`, saia com `Ctrl+X` e rode com `python3 tamanho.py`. Os dois scripts deste laboratório estão prontos em `scripts/lab10/`, no repositório.

```
# Analise de tamanho de modelos: parametros x bytes por parametro
modelos = {
    "MobileNetV2": {"params": 3_400_000, "input": (224, 224, 3)},
    "MobileNetV1-0.25": {"params": 470_000, "input": (128, 128, 3)},
    "EfficientNet-B0": {"params": 5_300_000, "input": (224, 224, 3)},
    "ResNet-50": {"params": 25_600_000, "input": (224, 224, 3)},
    "YOLOv8-nano": {"params": 3_200_000, "input": (640, 640, 3)},
    "Tiny-YOLO": {"params": 6_900_000, "input": (416, 416, 3)},
}
```

```

print(f"{'Modelo':<20} {'Parametros':>12} {'FP32 (MB)':>10} {'INT8 (MB)':>10} {'Economia':>9}")
print("-" * 66)
for nome, info in modelos.items():
    fp32_mb = info["params"] * 4 / 1_000_000 # 4 bytes por peso; MB = 1 milhao de bytes
    int8_mb = info["params"] * 1 / 1_000_000 # 1 byte por peso
    economia = (1 - int8_mb / fp32_mb) * 100
    print(f"{'nome':<20} {'info['params']':>12,} {'fp32_mb:>10.1f} {'int8_mb:>10.1f} {'economia:>8.0f}%")

print("\nRAM visivel no UNO Q (free -h): ~3,6 GiB")
print("Sistema + containers do App Lab em repouso: ~0,7 GiB")
print("Disponivel para modelo e aplicacao: ~2,9 GiB")

```

Passo 2: Rode e confira a tabela. A economia de INT8 sobre FP32 é sempre 75%, porque cada peso passa de 4 bytes para 1.

Parte B: Impacto da resolução de entrada

Passo 3: Calcule o tamanho do buffer de entrada para várias resoluções. Salve como `resolucao.py` e rode do mesmo jeito:

```

# Tamanho do buffer de entrada por resolucao
resolucoes = [(48, 48), (96, 96), (160, 160), (224, 224), (320, 320), (416, 416)]

base = 224 * 224 # referencia: a entrada do MobileNetV2

print(f"{'Resolucao':<12} {'RGB (KB)':>10} {'Cinza (KB)':>11} {'vs 224x224':>11}")
print("-" * 47)
for w, h in resolucoes:
    rgb = w * h * 3 / 1024
    cinza = w * h * 1 / 1024
    print(f"{'{w}x{h}':<12} {'rgb:>10.1f} {'cinza:>11.1f} {'w * h / base:>10.2f}x")

```

A última coluna compara cada resolução com 224×224. Reduzir a resolução tem efeito quadrático: metade do lado, um quarto da memória. De 224 para 96, o lado cai 2,3 vezes e a memória cai 5,4 vezes (0,18x). E o custo de computação acompanha, porque cada camada convolucional trabalha sobre todos os pixels. É por isso que, no Laboratório 8, o modelo de 416×416 foi o mais lento de todos.

Parte C: Os modelos que já estão na sua placa

Passo 4: Liste os modelos embutidos do App Lab e o tamanho dos arquivos. No terminal da placa:

```

arduino-app-cli model list
docker run --rm --entrypoint ls ghcr.io/arduino/app-bricks/ei-models-runner:0.12.1 -la /models/ootb/ei/

```

Tamanhos medidos nesta versão:

Arquivo .eim	Modelo (Laboratório 8)	Entrada	Tamanho no disco
--------------	---------------------------	---------	------------------

Arquivo .eim	Modelo (Laboratório 8)	Entrada	Tamanho no disco
yolo-x-nano.eim	Detecção de objetos, 80 classes	416×416×3	18,3 MB
mobilenet-v2-224px.eim	Classificação ImageNet, 1000 classes	224×224×3	27,9 MB
lw-face-det.eim	Detecção de rostos	320×256×1	17,6 MB
person-classification.eim	Classificação pessoa / não pessoa	224×224×3	14,4 MB

Repare que o menor dos quatro tem 14,4 MB, embora um classificador de duas classes tenha poucos pesos. A listagem traz outros modelos ainda menores, todos perto de 13 MB, e nenhum abaixo disso. O .eim carrega o runtime junto, cerca de 13 MB fixos. Para comparar modelos pelo tamanho, subtraia esse piso; para comparar pelo custo de execução, use o tempo de inferência do Laboratório 8, que é o que importa para o tempo real.

Passo 5: Compare a diferença entre o MobileNet-v2 de ImageNet (27,9 MB) e o classificador de pessoa (14,4 MB). Ambos têm a mesma entrada 224×224×3. A diferença entre os dois arquivos é de 13,5 MB, quase os 13,6 MB que a Parte A calculou para os pesos do MobileNetV2 em FP32. O classificador de pessoa usa uma rede bem menor: o arquivo dele fica só 1,4 MB acima do piso do runtime. No Laboratório 8, esse mesmo par mostrou 130 ms contra 27 ms de inferência.

Perguntas de checkpoint

Exemplo resolvido: o YOLOv8-nano tem 3,2 milhões de parâmetros. FP32: $3.200.000 \times 4 = 12,8$ MB. INT8: $3.200.000 \times 1 = 3,2$ MB. Economia: 9,6 MB (75%).

P1: Aplique o mesmo cálculo ao MobileNetV2 (3,4 milhões de parâmetros). Quanta memória se economiza ao passar de FP32 para INT8?

P2: Se você tivesse um dispositivo com apenas 256 KB de RAM, como um microcontrolador típico, algum dos modelos da Parte A caberia depois da quantização INT8? Se nenhum couber, quantos parâmetros um modelo poderia ter, no máximo, para caber? Lembre que o buffer de entrada e as ativações também precisam de espaço.

P3: Por que reduzir a resolução de entrada de 224×224 para 96×96 ajuda não só na memória, mas também na velocidade de inferência?

P4: O `person-classification.eim` tem 14,4 MB e o `yolo-x-nano.eim` tem 18,3 MB, uma diferença de só 27%. Mas a inferência de um é 14 vezes mais rápida que a do outro. O que o tamanho do arquivo não conta?

Principais lições

- Cada parâmetro FP32 ocupa 4 bytes; em INT8, 1 byte. A quantização economiza 75% do espaço dos pesos.
- O MobileNetV2 em FP32 precisa de 13,6 MB só para os pesos; a memória de execução soma entrada, ativações e saída.
- Reduzir a resolução de entrada tem efeito quadrático na memória e na computação.
- Os 3,6 GiB do UNO Q são generosos para um dispositivo de borda, mas o tempo de inferência, não a memória, é o que limita o tempo real neste hardware.
- Um `.eim` inclui o runtime (~13 MB); compare modelos pelo tempo de inferência, não pelo tamanho do arquivo.

O que entregar

1. A saída do script de análise de tamanho de modelos.
2. A saída da análise de resolução de entrada.
3. A listagem dos arquivos `.eim` da sua placa, com os tamanhos.
4. Respostas escritas para as perguntas de checkpoint.

Solução de problemas

Problema	Solução
Os cálculos parecem errados	FP32 = 4 bytes e INT8 = 1 byte por parâmetro. Os tamanhos de modelo usam MB = 1.000.000 bytes (3,4 milhões × 4 = 13,6 MB). Os buffers de imagem usam KB = 1.024 bytes, como no Laboratório 7 (640×480×3 = 900 KB).
<code>docker run</code> falha com “permission denied”	O usuário <code>arduino</code> já pertence ao grupo <code>docker</code> na imagem padrão. Se falhar, use <code>sudo docker ...</code>
<code>docker run</code> responde “Unable to find image” e tenta baixar	A tag da imagem mudou com uma atualização do App Lab. Veja a tag instalada na linha <code>ei-models-runner</code> da saída de <code>docker images</code> e use-a no comando.

Notas

Espaço para suas anotações.

Laboratório 11: Treine o seu modelo

Duração	Dificuldade	Pré-requisitos
90-120 min	Intermediário-Avançado	Laboratório 9 concluído; conta Edge Impulse (Laboratório 0)

Objetivo

Até aqui você usou modelos prontos. Neste laboratório o modelo é seu: você coleta imagens, treina um detector de objetos no Edge Impulse, implanta o modelo no UNO Q e o executa ao vivo sobre o vídeo da câmera, no lugar do modelo embutido do Laboratório 9.

O que você vai aprender

1. Criar um projeto de detecção de objetos no Edge Impulse Studio e entender o pipeline de aprendizado de máquina.
2. Coletar e rotular imagens de treinamento com caixas delimitadoras.
3. Projetar e treinar um detector FOMO leve, adequado a dispositivos de borda.
4. Exportar o modelo treinado como arquivo .eim e importá-lo em um Brick do App Lab.
5. Executar inferência em tempo real sobre a câmera e reagir às detecções em Python.

Materiais necessários

Item	Detalhes
Arduino UNO Q 4GB	Ligado à rede Wi-Fi configurada no Laboratório 0
Hub USB-C com Power Delivery	Fonte 5 V/3 A na entrada PD, webcam na porta USB-A, como no Laboratório 7. Sem o hub, use o celular como câmera: faça as Partes A a C e depois siga a seção “Extensão opcional: o celular como câmera”, no lugar dos Passos 14 a 17
Webcam USB (UVC) ou smartphone	Para coleta de dados e inferência
Computador com Arduino App Lab	Na mesma rede Wi-Fi da placa (a porta USB-C da placa fica ocupada pelo hub)
Conta Edge Impulse	Criada no Laboratório 0
Smartphone	Recomendado, para fotografar as amostras de treinamento pelo Studio. Sem ele, envie fotos do computador.

Visão geral do conceito

Todo sistema de Edge AI segue o mesmo padrão de três etapas:

- **SENTIR:** capturar dados do mundo real (câmera, microfone, sensor).
- **PENSAR:** executar um modelo de aprendizado de máquina que interpreta os dados (inferência).
- **AGIR:** reagir ao resultado (acender um LED, enviar um alerta, mover um motor).

O Edge Impulse é a plataforma de treinamento que se integra ao App Lab. Ela cobre o pipeline inteiro: coleta, rotulagem, treinamento, otimização e exportação em um formato que o UNO Q executa. O modelo exportado, um arquivo .eim, é carregado por um Brick de visão do App Lab, que cuida da câmera, do runner de inferência e das chamadas ao seu código Python.

Neste laboratório o modelo é um **detector de objetos**: ele responde “onde está o objeto e com que confiança”, devolvendo uma caixa por objeto encontrado. Escolhemos detecção porque é o mesmo tipo de modelo do Brick Video Object Detection, que você usou no Laboratório 9 e vai usar de novo no Laboratório 13. A arquitetura será o **FOMO** (Faster Objects, More Objects), uma rede pequena do Edge Impulse feita para rodar rápido em dispositivos de borda, com entrada de 96x96 pixels.

Alguns termos de aprendizado de máquina aparecem neste laboratório:

Termo	Significado
Ciclo de treinamento	Uma passada completa da rede por todas as imagens de treino. Mais ciclos, mais ajuste. Também chamado de época.
Matriz de confusão	Tabela que cruza o que o modelo respondeu com a resposta certa. Mostra quais classes ele troca.
F1 score	Nota de 0 a 100% que resume acertos e erros de um detector. Combina quantos objetos ele achou com quantos alarmes falsos deu.
float32 e int8	Duas formas de guardar os números do modelo. A versão int8, chamada quantizada, é quatro vezes menor e mais rápida. O Laboratório 12 compara as duas.
Runner	O programa que carrega o modelo e

Termo	Significado
	executa a inferência. No App Lab ele roda em um container próprio.

Dica: o Edge Impulse Studio adapta a interface ao tipo de dado do projeto. Um projeto criado para detecção mostra só os blocos de aprendizado compatíveis com caixas delimitadoras; um projeto de classificação mostra outros. Se um bloco “sumir”, é isso. O link “Show all blocks anyway” exhibe todos.

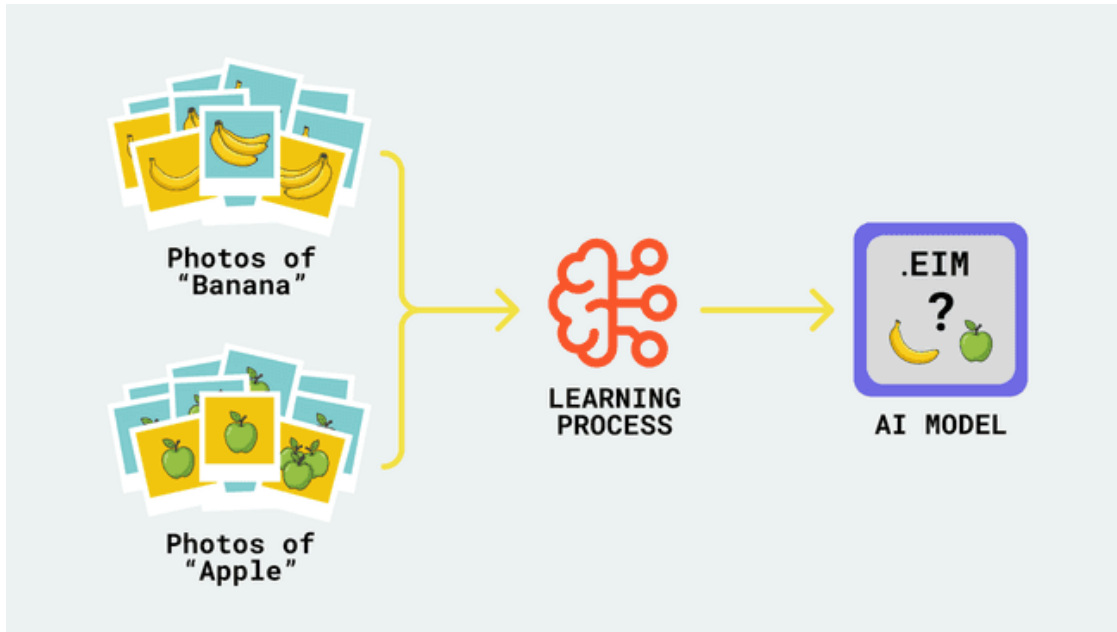


Figura 11.1: Pipeline de Edge AI, dos dados à implantação

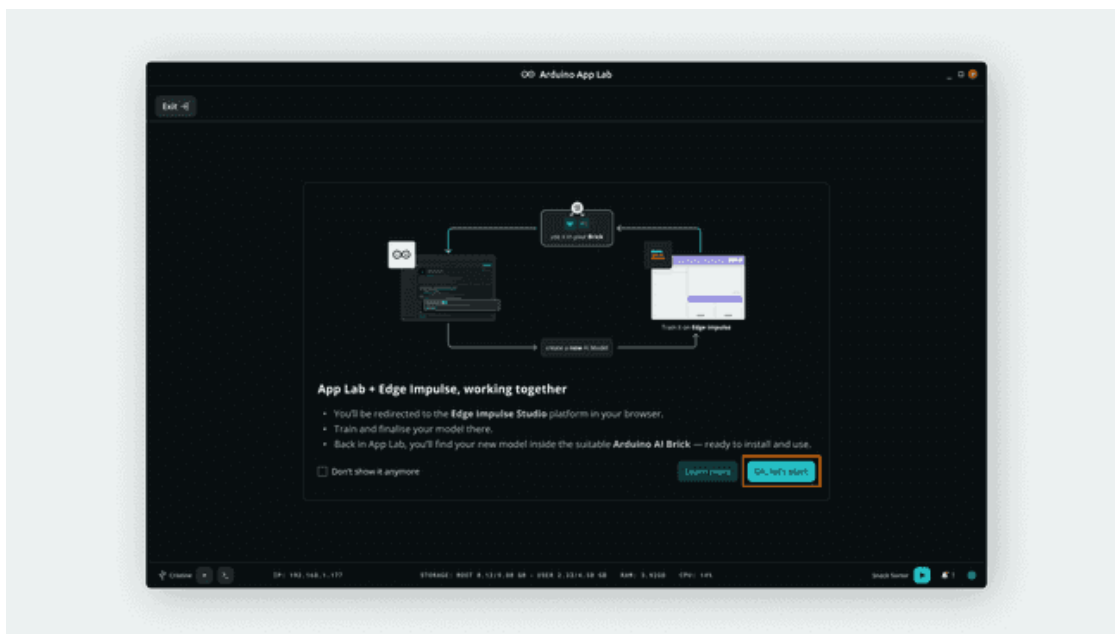


Figura 11.2: Integração do Edge Impulse com o Arduino App Lab

Procedimento

Parte A: Criar o projeto no Edge Impulse

Passo 1: abra studio.edgeimpulse.com e entre na sua conta.

Passo 2: clique em “Create new project”, dê um nome como `unoq-detector` e confirme.

Passo 3: quando o Studio perguntar o tipo de projeto, escolha **Images** e, na pergunta seguinte, **Classify multiple objects (object detection)**. Essa escolha define que as imagens receberão caixas delimitadoras e que os blocos de detecção ficarão disponíveis.

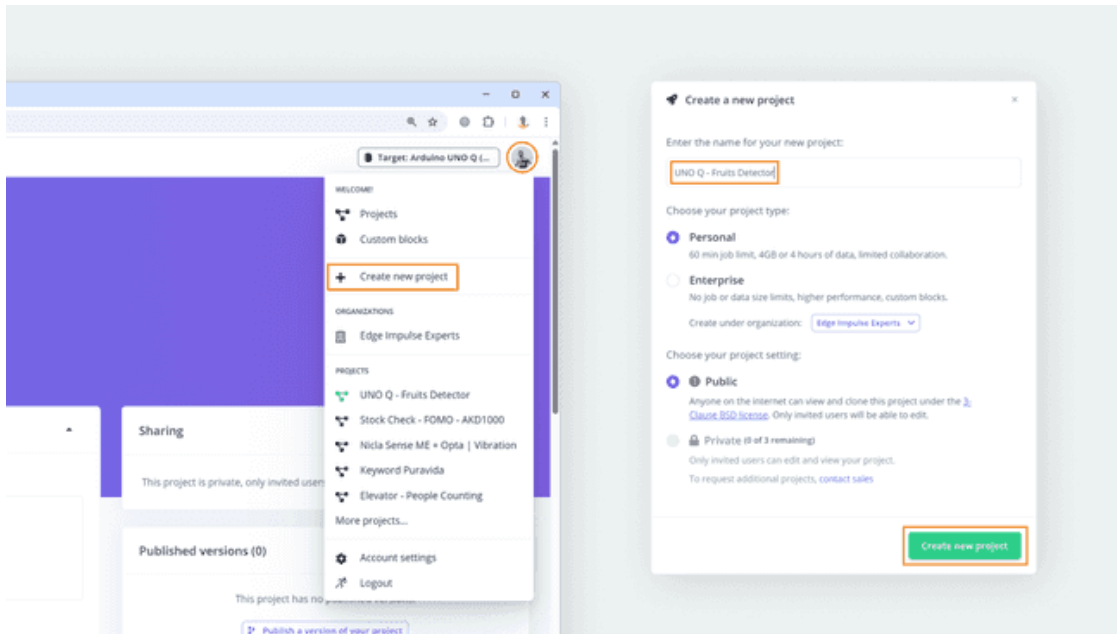


Figura 11.3: Criando um novo projeto no Edge Impulse

Parte B: Coletar e rotular os dados

Passo 4: decida o que detectar. Escolha dois objetos bem diferentes entre si e fáceis de segurar, por exemplo uma caneca e um mouse, ou dois gestos de mão. Você precisa de 30 a 40 imagens por classe para um primeiro modelo.

Passo 5: no menu lateral, abra **Data acquisition**. No quadro “Collect new data”, escolha a opção de usar o celular. O Studio mostra um código QR; ao lê-lo, o navegador do celular abre uma página de captura ligada ao projeto. Fotografe cada objeto variando ângulo, distância, fundo e iluminação. Você também pode subir imagens do computador com “Upload data”, no mesmo quadro.

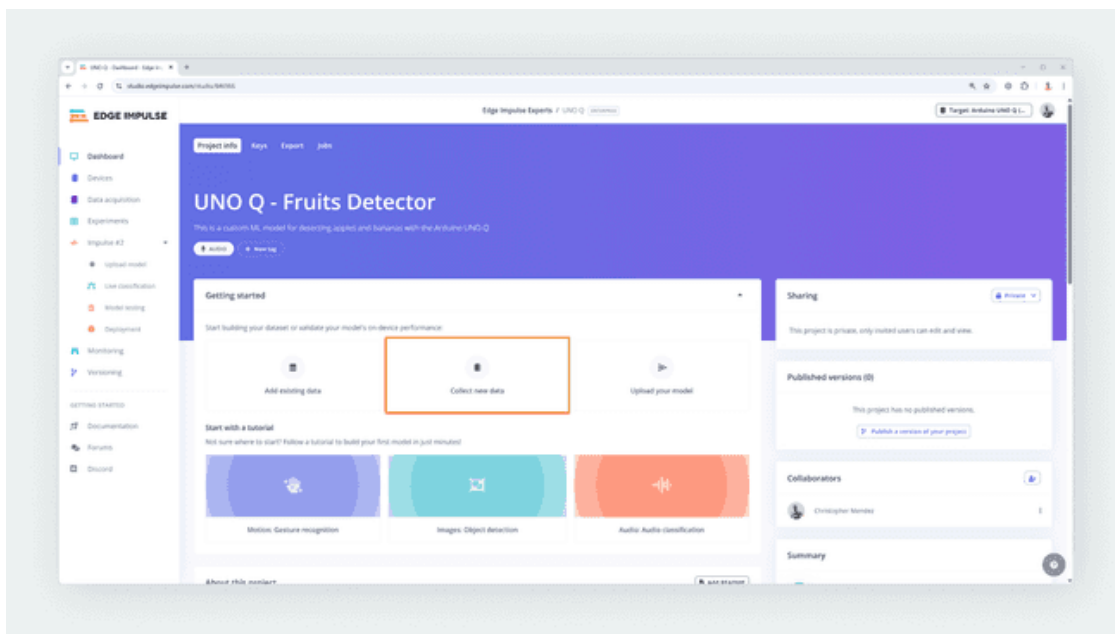


Figura 11.4: Painel do projeto no Edge Impulse, com a coleta de dados em destaque

Passo 6: rotule as imagens. Em um projeto de detecção, cada imagem precisa de ao menos uma caixa desenhada em volta do objeto, com o nome da classe. A **Labeling queue** (fila de rotulagem) é uma aba no topo da página Data acquisition, com o número de imagens pendentes. Abra-a, desenhe a caixa, escolha o rótulo e avance. Imagens sem caixa não entram no treinamento.

Passo 7: confira, no topo da página Data acquisition, a divisão entre Train e Test. O padrão é cerca de 80/20. Se todas as imagens caíram em um só lado, refaça a divisão: no **Dashboard** do projeto, perto do fim da página, há a opção “Perform train / test split”.

Dica: qualidade importa mais do que quantidade. Imagens representativas do que a câmera verá na prática, com fundos e luz variados, rendem mais que centenas de fotos iguais.

Parte C: Projetar e treinar o modelo

Passo 8: vá em **Create impulse**. Em Image data, deixe largura e altura em 96 e o modo de redimensionamento em “Fit shortest axis”. Adicione o bloco de processamento **Image** e o bloco de aprendizado **Object Detection (Images)**. Salve o impulse.

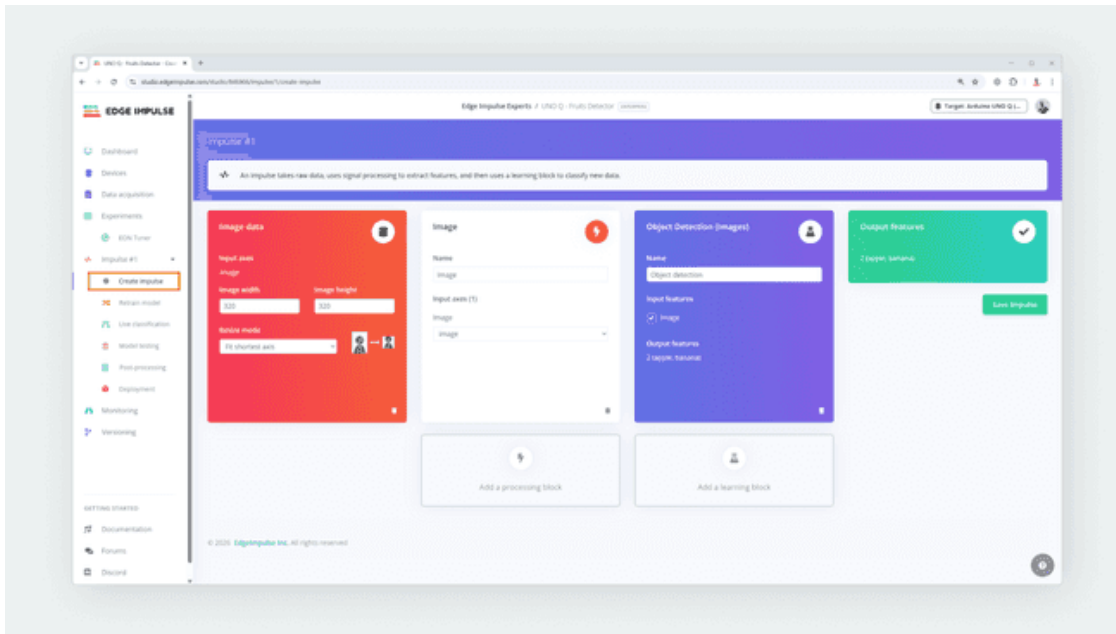


Figura 11.5: Projeto do impulse com blocos de processamento e aprendizado

Passo 9: abra a página **Image** no menu lateral, salve os parâmetros e clique em **Generate features**. Essa etapa converte todas as imagens no formato que a rede neural consome. Espere o job terminar.

Aviso: se aparecer o erro `input.json does not exist`, o conjunto de treino está vazio ou as imagens ainda não têm caixas. Volte à Parte B, rotule a fila e confira a divisão Train/Test.

Passo 10: abra a página **Object detection**. Em “Neural network architecture” escolha **FOMO (Faster Objects, More Objects) MobileNetV2 0.35**. Defina 60 ciclos de treinamento e clique em **Start training**. O treino leva alguns minutos nos servidores do Edge Impulse.

Passo 11: ao terminar, anote o **F1 score** e a matriz de confusão. Anote também o quadro “On-device performance”, que estima latência e memória; repare para qual dispositivo a estimativa foi feita, porque ela não é a do UNO Q. Mire em F1 acima de 80% na primeira tentativa.

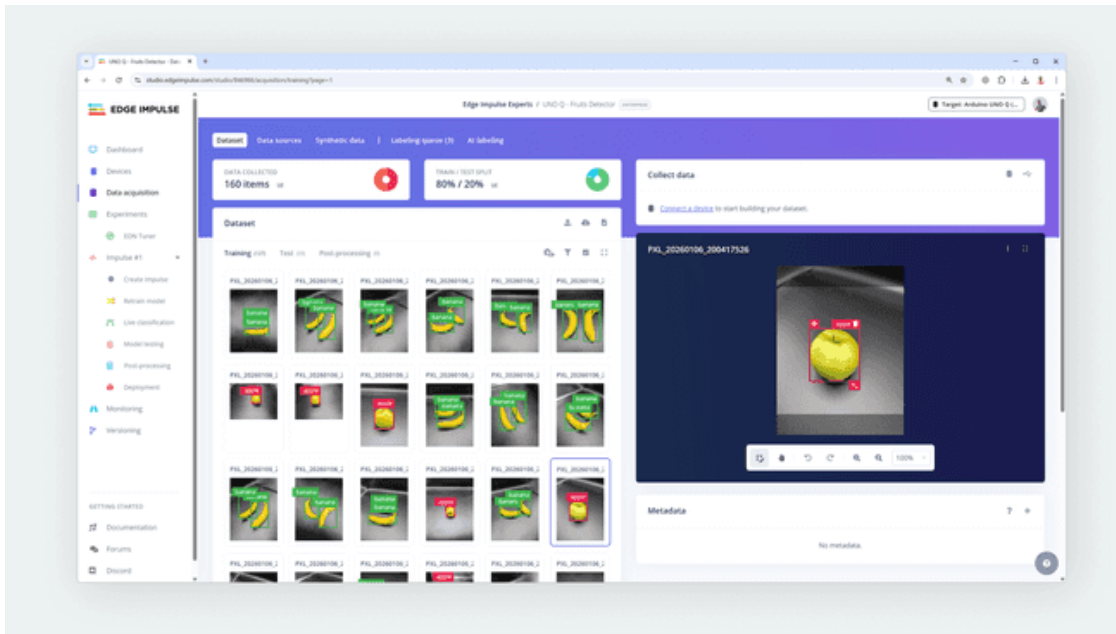


Figura 11.6: Resultados de treinamento com métricas de precisão

Parte D: Implantar no UNO Q

Passo 12: vá em **Deployment**. Na busca de alvos, procure “Arduino UNO Q”. Se o alvo existir, selecione-o. Se não existir, selecione **Linux (AARCH64)**: AARCH64 é a arquitetura do processador da placa, a mesma que o comando `uname -a` mostrou no Laboratório 0.

Passo 13: se o Studio oferecer a escolha entre modelo quantizado (int8) e não otimizado (float32), escolha **Quantized (int8)**. Clique em **Build**. O navegador baixa um arquivo `.eim` para o seu computador. Guarde-o com um nome que indique a versão, por exemplo `unoq-detector-int8.eim`. O Laboratório 12 volta a este ponto para comparar as duas versões.

Passo 14: no App Lab, abra **Inspirations**, na barra lateral, localize **Detect Objects on Camera** e use **Copy and edit app**, com o nome `meu-detector`. É o mesmo exemplo de partida do Laboratório 9: ele já traz o Brick Video Object Detection, o Brick Web UI e uma página web com o vídeo e as detecções.

Passo 15: clique no Brick Video Object Detection no painel lateral e abra a aba **AI models**, a mesma em que você trocou de modelo no Laboratório 8. Use a opção de importar modelo personalizado e selecione o `.eim` que está no seu computador. O App Lab copia o arquivo para a placa; você não precisa transferi-lo à mão.

Parte E: Escrever e rodar a aplicação de inferência

Passo 16: abra `python/main.py` e substitua todo o conteúdo pelo código abaixo. Ele mantém o que o exemplo já fazia, que é alimentar a página web, e acrescenta a

impressão de cada detecção no console. O Brick abre a câmera sozinho, envia os quadros ao runner do modelo e chama a sua função a cada resultado.

```
# python/main.py – inferência em tempo real com o modelo treinado
from arduino.app_utils import App
from arduino.app_bricks.web_ui import WebUI
from arduino.app_bricks.video_objectdetection import VideoObjectDetection
from datetime import datetime, UTC

ui = WebUI() # serve a página do exemplo em :7000; sem esta linha a página não abre

# confidence=0.5: só reporta detecções em que o modelo tem mais de 50% de certeza.
# Experimente 0.3 (mais detecções, mais alarmes falsos) ou 0.8 (menos, mais confiáveis).
# debounce_sec=0.0: entrega todos os resultados, sem intervalo mínimo entre dois iguais.
detection_stream = VideoObjectDetection(confidence=0.5, debounce_sec=0.0)

# O controle de confiança da página envia a mensagem "override_th"
ui.on_message("override_th", lambda sid, threshold: detection_stream.override_threshold(threshold))

def on_detections(detections: dict):
    # Formato: {"caneca": [{"confidence": 0.91, "bounding_box_xyxy": (x1, y1, x2, y2)}], ...}
    for label, instances in detections.items():
        for det in instances:
            print(f"{label}: {round(det['confidence'] * 100)} % caixa={det['bounding_box_xyxy']}")
    ui.send_message("detection", message={
        "content": label,
        "confidence": det.get("confidence"),
        "timestamp": datetime.now(UTC).isoformat(),
    })

detection_stream.on_detect_all(on_detections)

App.run()
```

Passo 17: clique em **Run**. Aponte a câmera para os objetos treinados e observe o console: cada linha traz o rótulo, a confiança e a caixa. Abra `http://<nome-da-placa>.local:7000` no navegador para ver o vídeo com as caixas desenhadas.

Dica: o runner de modelos também publica um preview em `http://<nome-da-placa>.local:4912`, com o vídeo e o campo “Time per inference”. Esse número é o tempo real de inferência do seu modelo no UNO Q. Compare-o com os quatro modelos que você mediu no Laboratório 8; o Laboratório 12 volta a ele.

Nota, variante com classificação: se o seu projeto foi criado como classificação de imagem única (“Classify a single object”), o modelo não devolve caixas. Nesse caso troque o Brick por Video Image Classification: no `app.yaml` use `arduino:video_image_classification`, importe de `arduino.app_bricks.video_imageclassification` import `VideoImageClassification`, e

o dicionário do callback passa a ser {"caneca": 0.91}, rótulo e confiança direto, sem lista de caixas.

Perguntas de checkpoint

P1: por que a qualidade dos dados importa mais que a quantidade ao treinar um modelo de borda? Dê um exemplo de foto que atrapalharia o seu detector.

P2: o que o arquivo .eim contém, e por que ele roda no UNO Q sem conexão com a nuvem?

P3: o seu modelo teve F1 de 90% no Edge Impulse, mas na placa ele erra bastante. Cite duas causas prováveis e como testar cada uma.

P4: o que mudaria no código se o modelo fosse de classificação em vez de detecção?

Principais lições

- Todo pipeline de Edge AI tem três etapas: sentir, pensar com um modelo e agir com o resultado.
- O Edge Impulse cuida da etapa mais difícil: coleta, rotulagem, treinamento e exportação de um .eim que roda direto na placa.
- Em detecção de objetos, rotular é desenhar caixas; sem caixas não há treino.
- O Brick de visão encapsula câmera, runner e callbacks. Seu código só decide o que fazer com cada detecção.

Extensão opcional: o celular como câmera

Sem hub ou webcam, use o smartphone como câmera sem fio, como na versão B do Laboratório 9. Esta seção substitui os Passos 14 a 17:

1. No App Lab, abra o seu vision-app da versão B do Laboratório 9. Se você não o tem, copie o exemplo **Detect Objects on Smartphone Camera** e cole nele o main.py da versão B (Laboratório 9, Passo 4). Não use o main.py do Passo 16 deste laboratório: ele não cria a câmera remota.
2. Importe o seu .eim no Brick Video Object Detection, como no Passo 15, e clique em Run. A interface web em `http://<nome-da-placa>.local:7000` mostra um código QR.
3. No celular, abra o app Arduino IoT Remote, vá em **Devices**, toque no +, escolha **Stream phone camera to UNO Q** e aponte para o QR.
4. Aponte o celular para os objetos. As detecções aparecem na interface web em tempo real.
5. Para ver cada detecção também no console, acrescente ao main.py, dentro do laço `for det in instances:`, a linha `print(f"{label}: {round(det.get('confidence') * 100)} %")`, com o mesmo recuo da linha `ui.send_message` que vem logo abaixo.

O vídeo nunca sai da sua rede: o celular e a placa se falam por WebSocket, com o quadro comprimido em JPEG. Para usar o seu .eim nesse exemplo, importe-o no Brick Video Object Detection do mesmo jeito da Parte D.

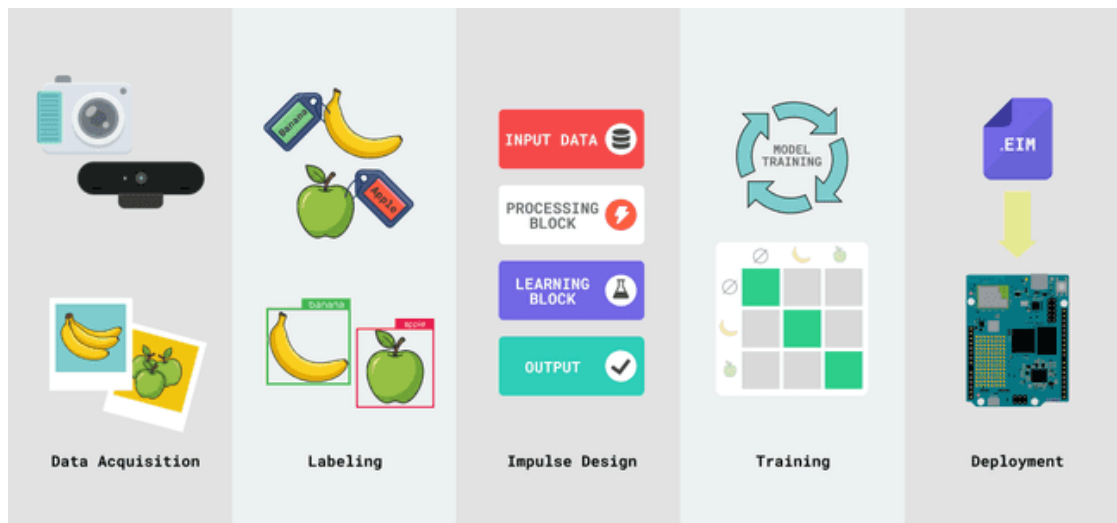


Figura 11.7: Detecção de objetos com a câmera do smartphone

O que entregar

1. Uma captura de tela dos resultados do treinamento no Edge Impulse (F1 e matriz de confusão).
2. Uma captura de tela do console do App Lab mostrando detecções do seu modelo.
3. O seu `python/main.py`.
4. Um parágrafo descrevendo o que o modelo detecta, o tempo por inferência mostrado em :4912 e como ele se saiu.

Solução de problemas

Problema	Solução
O bloco de aprendizado que você queria não aparece	O Studio esconde blocos incompatíveis com os dados do projeto. Confirme o tipo do projeto (detecção ou classificação) ou clique em “Show all blocks anyway”.
Generate features falha com <code>input.json</code> does not exist	Conjunto de treino vazio ou imagens sem caixa. Rotule a Labeling queue e confira a divisão Train/Test.
F1 muito baixo (abaixo de 50%)	Colete imagens mais variadas, confira os rótulos, aumente os ciclos de treino.
Não encontro o alvo Arduino UNO Q no Deployment	Use Linux (AARCH64), que gera o mesmo .eim.

Problema	Solução
O App Lab não aceita o arquivo	Confirme que a importação foi feita na aba de modelos do Brick correto (detecção para modelos com caixas, classificação para os sem).
Inferência lenta	Veja o “Time per inference” em :4912. Modelos maiores que 96x96 ou arquiteturas além do FOMO custam mais, como você mediu no Laboratório 8.
O celular não conecta no exemplo do smartphone	Celular e placa precisam estar na mesma rede Wi-Fi. O QR é lido de dentro do app Arduino IoT Remote (Devices, +, Stream phone camera to UNO Q), não pela câmera do celular.
O app não inicia e o console mostra No Camera Device Found	A placa não vê nenhuma câmera. Confira o hub e a webcam, ou use o exemplo do smartphone (Extensão opcional).
A página :7000 não abre	O <code>main.py</code> precisa criar o <code>WebUI()</code> . Confira se copiou o código do Passo 16 inteiro.

Notas

Espaço para suas anotações.

Laboratório 12: Quantização, encolhendo o modelo

Duração	Dificuldade	Pré-requisitos
90-120 min	Avançado	Laboratórios 11, 8 e 10 concluídos, conta Edge Impulse com projeto treinado

Objetivo

Exportar o mesmo modelo treinado no Edge Impulse em duas versões, uma em ponto flutuante de 32 bits (float32) e outra quantizada em inteiros de 8 bits (INT8), comparar precisão, tamanho e tempo de inferência, e medir as duas versões rodando no UNO Q.

O que você vai aprender

1. Aplicar quantização pós-treinamento usando o Edge Impulse Studio
2. Comparar precisão, tamanho e latência estimada entre float32 e INT8
3. Medir o tempo real de inferência de cada versão na placa
4. Tomar uma decisão de implantação com base em dados, não em suposição

Materiais necessários

Item	Detalhes
Arduino UNO Q 4GB	Alimentado por hub USB-C com Power Delivery, acesso por Wi-Fi
Webcam USB (UVC)	Conectada ao hub. Este laboratório foi escrito para a webcam; com o celular, a medição do Passo 11 vale do mesmo jeito no app da versão B do Laboratório 9
Computador com Arduino App Lab	Conectado à mesma rede Wi-Fi da placa
Conta Edge Impulse	Com o projeto treinado no Laboratório 11

Visão geral do conceito

O Laboratório 10 mostrou por que os modelos precisam ser menores. Este laboratório aplica a técnica principal para isso.

Quantização é converter os pesos do modelo de números de alta precisão (ponto flutuante de 32 bits, como 0,0023456789) para inteiros de baixa precisão (8 bits, que só admitem 256 valores diferentes). O modelo fica quatro vezes menor e, em geral, mais rápido, porque unidades inteiras são mais simples e consomem menos energia do que unidades de ponto flutuante.

O preço é a precisão. Arredondar cada peso para um inteiro perde detalhe. Para a maioria das aplicações a queda é pequena, tipicamente entre 1 e 3 pontos percentuais, mas você só sabe medindo com os seus dados.

Formato	Bits por peso	Tamanho dos pesos (modelo de 3,4 milhões de parâmetros)	Impacto típico na precisão
float32 (original)	32	13,6 MB	Linha de base
float16	16	6,8 MB	Desprezível
INT8 (quantizado)	8	3,4 MB	Pequeno (1-3 pontos)

Como funciona: para cada camada, a quantização encontra o intervalo dos pesos (por exemplo, de -1,0 a +1,0) e mapeia esse intervalo nos 256 valores possíveis de um inteiro de 8 bits, de -128 a 127. Dois pesos próximos, como 0,5000 e 0,5030, viram o mesmo inteiro, 64. Não é exato, mas é próximo o suficiente.

Existem duas abordagens. A quantização pós-treinamento (PTQ) treina o modelo normalmente e converte no fim. O treinamento consciente de quantização (QAT) incorpora a quantização ao treino, e o modelo aprende a conviver com a precisão menor; dá resultados melhores, mas é mais trabalhoso. O Edge Impulse usa PTQ e oferece as duas versões na hora de exportar.

Uma nota sobre o UNO Q: nesta versão dos Bricks, os modelos rodam na CPU do processador Qualcomm, sem usar o DSP ou a GPU. Mesmo assim, a versão INT8 costuma ser mais rápida, porque a CPU também tem instruções vetoriais para inteiros. O quanto ela é mais rápida, você vai medir.

Aviso: o tamanho do arquivo .eim que o Edge Impulse gera inclui o runtime de inferência, cerca de 13 MB fixos, além dos pesos do modelo. Para comparar tamanhos, use os números da página do modelo no Studio, não o tamanho do arquivo.

Procedimento

Parte A: Registre a linha de base (float32)

Passo 1: Abra o projeto do Laboratório 11 no Edge Impulse Studio.

Passo 2: Abra a página do bloco de aprendizado (Object detection ou Transfer learning, conforme o seu projeto) e leia os resultados do último treinamento. Registre:

Métrica	Valor float32
F1 no conjunto de validação (projeto de detecção) ou acurácia (projeto de	_____

Métrica	Valor float32
classificação)	
Tamanho estimado do modelo	_____ KB
Tempo de inferência estimado	_____ ms
Dispositivo usado na estimativa	_____

Dica: a estimativa de latência do Studio é feita para um dispositivo de referência escolhido nas configurações do projeto. Ela serve para comparar versões entre si, não como previsão do tempo no UNO Q. O tempo real você mede na Parte C.

Parte B: Exporte as duas versões

Passo 3: Abra a página Deployment.

Passo 4: Na busca de alvos, escolha “Arduino UNO Q” se existir, ou “Linux (AARCH64)”. Os dois geram um arquivo .eim.

Passo 5: Antes de clicar em Build, procure a opção de otimização. O Studio oferece “Quantized (int8)” e “Unoptimized (float32)”, com a precisão estimada de cada uma ao lado. Registre a precisão mostrada para a versão INT8.

Passo 6: Faça o build da versão INT8 e baixe o arquivo. Renomeie para algo como meu-modelo-int8.eim. Se você guardou o unoq-detector-int8.eim do Laboratório 11 e não retreinou o modelo desde então, pode usá-lo no lugar deste build.

Passo 7: Repita o build com “Unoptimized (float32)” e renomeie para meu-modelo-float32.eim.

Passo 8: Preencha a comparação de estimativas:

Métrica	float32	INT8	Mudança
Precisão ou F1			
Tamanho estimado (KB)			
Latência estimada (ms)			

Parte C: Meça na placa

Passo 9: No App Lab, abra o app meu-detector do Laboratório 11. Se você não o tem mais, copie o exemplo “Detect Objects on Camera” com esse nome; nesse caso as detecções aparecem só na página :7000, não no console. Clique no Brick Video Object Detection, abra a aba de modelos, importe meu-modelo-int8.eim e selecione-o pelo botão **Brick Configuration**. Confira no app.yaml se a chave model: aponta para ele; sem isso você mede o modelo anterior sem perceber.

Passo 10: Clique em Run e aponte a câmera para os objetos que o modelo conhece.

Passo 11: Abra `http://<nome-da-placa>.local:4912` no navegador. Essa é a página do runner de modelos do Edge Impulse que o Brick usa por baixo. Ela mostra o vídeo, as detecções e a linha “Time per inference: N ms”. Anote o valor depois de 30 segundos de operação, com a placa aquecida.

Passo 12: Pare o app, importe `meu-modelo-float32.eim` do mesmo jeito e selecione-o no Brick, pelo botão **Brick Configuration** da aba de modelos. Confira no `app.yaml` qual modelo ficou na chave `model:` e repita os passos 10 e 11.

Passo 13: Se quiser uma medição mais rigorosa do que a leitura visual, use a sonda do Laboratório 8, que lê o WebSocket do runner e calcula média, mínimo e máximo de `timeMs` em 20 segundos. A sonda precisa estar na pasta do app que está rodando. No terminal SSH da placa, copie o arquivo e execute:

```
cp ~/ArduinoApps/model-cost/ws_probe.py ~/ArduinoApps/meu-detector/
docker exec meu-detector-main-1 python3 /app/ws_probe.py ws://ei-video-obj-detection-runner:4912
```

Passo 14: Preencha a tabela final com valores medidos:

Métrica	float32	INT8	Mudança
Tempo de inferência medido (ms)			
Resultados por segundo			
Qualidade das detecções (avaliação qualitativa)			

Exemplo resolvido: suponha que o modelo float32 tenha 91,5 % de precisão e o INT8 89,8 %. A queda é de 1,7 ponto percentual. Isso é aceitável? Para um exercício que classifica gestos, sim. Para uma ferramenta de diagnóstico médico, provavelmente não. A resposta depende da aplicação, e é por isso que a decisão de implantação precisa dos dois números, precisão e latência, lado a lado.

Perguntas de checkpoint

P1: Qual foi a diferença de precisão entre as versões float32 e INT8 do seu modelo? Você a consideraria aceitável para a sua aplicação? Justifique.

P2: Por qual fator a quantização reduziu o tamanho estimado do modelo? Chegou perto da redução teórica de 4 vezes? Por que o arquivo `.eim` não encolheu na mesma proporção?

P3: Qual foi o ganho de velocidade medido na placa entre float32 e INT8? Ele foi maior ou menor do que a diferença de latência estimada pelo Studio?

Principais lições

- Quantização INT8 reduz os pesos do modelo em 4 vezes, com perda de precisão tipicamente pequena, mas que deve ser medida com os seus dados

- O Edge Impulse gera as duas versões na exportação, o que permite comparar sem retreinar
- A estimativa do Studio serve para comparar versões; o número que vale é o medido no UNO Q
- O arquivo `.eim` carrega o runtime junto, então o tamanho do arquivo não reflete o tamanho do modelo

O que entregar

1. As duas tabelas de comparação preenchidas (estimativas do Studio e medições na placa).
2. Os dois arquivos `.eim` exportados.
3. Respostas escritas para as perguntas de checkpoint.
4. Um parágrafo: você implantaria a versão float32 ou INT8 na sua aplicação, e por quê?

Solução de problemas

Problema	Solução
A precisão cai muito (mais de 10 pontos)	O conjunto de dados é pequeno ou pouco variado. Colete mais dados e retreine antes de quantizar.
O modelo INT8 não é mais rápido	Em modelos muito pequenos, o custo de converter entrada e saída pode anular o ganho. Compare também o tempo total por quadro na página :4912.
O build falha no Edge Impulse	Confirme que a arquitetura é suportada para o alvo escolhido. Tente uma arquitetura mais simples, como FOMO MobileNetV2 0.35.
O App Lab não aceita o <code>.eim</code>	Confirme que o alvo de exportação foi Linux AARCH64 ou Arduino UNO Q, não um alvo de microcontrolador.
[ERROR] No Camera Device Found ao clicar em Run	A placa não está vendo nenhuma câmera. Confira o hub e a webcam com <code>v4l2-ctl --list-devices</code> .
A página :4912 não abre	Confirme que o app está rodando e que o PC está na mesma rede da placa. Tente pelo IP em vez do nome <code>.local</code> .

Notas

Espaço para suas anotações.

Laboratório 13: Implantando e medindo seu sistema

Duração	Dificuldade	Pré-requisitos
90-120 min	Avançado	Laboratórios 9 e 12 concluídos

Objetivo

Montar a aplicação completa, com câmera, inferência, interface web e acionamento do MCU, monitorar CPU, memória e temperatura durante uma execução longa e produzir um relatório estruturado de desempenho.

O que você vai aprender

1. Integrar câmera, modelo, interface web e atuador em um único app
2. Monitorar os recursos do sistema Linux enquanto a inferência roda
3. Identificar limitação térmica e o efeito dela no desempenho
4. Escrever um relatório de desempenho para uma implantação de Edge AI

Materiais necessários

Item	Detalhes
Arduino UNO Q 4GB	Alimentado por hub USB-C com Power Delivery, acesso por Wi-Fi
Webcam USB (UVC)	Conectada ao hub. Com o celular como câmera, pule a Parte A: o seu vision-app da versão B do Laboratório 9 já tem câmera, modelo, página web, contador e LED. Faça as Partes B a E com ele
Computador com Arduino App Lab	Na mesma rede Wi-Fi; um terminal SSH aberto
Modelo	O modelo embutido do Brick ou o .eim do Laboratório 12
Dissipador de calor	Opcional, recomendado para testes longos

Visão geral do conceito

O Laboratório 12 comprimiu um modelo. Este laboratório coloca o modelo em uma aplicação completa e mede se ele se comporta bem por tempo prolongado.

Implantar em um dispositivo de borda não é só fazer caber; é fazer funcionar bem, por horas, sem intervenção.

Uma lista de verificação de implantação cobre cinco perguntas:

1. O modelo cabe na memória?

2. A inferência é rápida o suficiente para a aplicação?
3. A precisão é aceitável depois da otimização?
4. Como o sistema se comporta sob carga sustentada? A temperatura sobe? O desempenho cai?
5. Quanto de CPU e memória sobra para o resto do sistema?

O UNO Q tem 3,6 GiB de RAM visíveis para o Linux e quatro núcleos. O runner de modelos usa um núcleo por inferência. Isso deixa folga para a interface web, a câmera e a lógica da aplicação, mas a temperatura sob carga sobe. Você vai medir quanto.

Procedimento

Parte A: A aplicação completa

Passo 1: No App Lab, copie o exemplo “Detect Objects on Camera”, em **Inspirations**, com **Copy and edit app** e dê o nome `deploy-final`. Ele já traz os Bricks `arduino:video_object_detection` e `arduino:web_ui`, e a página web em `assets/`. Como você viu no Laboratório 9, o exemplo vem sem a pasta `sketch/`.

Passo 2: Se você quiser usar o modelo do Laboratório 12, importe o `.eim` na aba de modelos do Brick. Para usar o modelo embutido, não mude nada.

Passo 3: Dê ao app a pasta `sketch/`, com os dois arquivos que ela precisa ter (`sketch.ino` e `sketch.yaml`). No terminal SSH da placa, copie a do exemplo Blink:

```
cp -r /var/lib/arduino-app-cli/examples/core-and-foundational/01-led-blink/03-blinking-an-led-from-python/sketch ~/ArduinoApps/deploy-final/
```

Se você fez o Laboratório 9, pode copiar a pasta do seu app: `cp -r ~/ArduinoApps/vision-app/sketch ~/ArduinoApps/deploy-final/`. Feche e abra o app no App Lab para a pasta aparecer em **Files**, e confira que `sketch/sketch.ino` tem este conteúdo (substitua se for diferente):

```
#include <Arduino_RouterBridge.h>

// LED_BUILTIN acende em LOW
void set_led_state(bool state) {
    digitalWrite(LED_BUILTIN, state ? LOW : HIGH);
}

void setup() {
    pinMode(LED_BUILTIN, OUTPUT);
    digitalWrite(LED_BUILTIN, HIGH);           // começa apagado
    Bridge.begin();
    Bridge.provide("set_led_state", set_led_state);
}

void loop() {}
```

Passo 4: Substitua todo o conteúdo de `python/main.py` por:

```

from arduino.app_utils import App, Bridge
from arduino.app_bricks.web_ui import WebUI
from arduino.app_bricks.video_objectdetection import VideoObjectDetection
from datetime import datetime, UTC
import time

ui = WebUI()
detection_stream = VideoObjectDetection(confidence=0.5, debounce_sec=0.0)
ui.on_message("override_th", lambda sid, th: detection_stream.override_threshold(th))

frame_count = 0
start_time = time.time()
detections_log = []
led_on = False

def on_detections(detections: dict):
    global frame_count, led_on
    frame_count += 1
    timestamp = time.time() - start_time

    # Registra cada detecção e envia para a página web
    for class_name, instances in detections.items():
        for det in instances:
            detections_log.append({"time": timestamp, "class": class_name,
                                   "confidence": det.get("confidence", 0)})
            ui.send_message("detection", message={"content": class_name,
                                                  "confidence": det.get("confidence"),
                                                  "timestamp": datetime.now(UTC).isoformat()})

t())

# Aciona o LED do MCU quando há pessoa na cena
person = "person" in detections
if person != led_on:
    led_on = person
    Bridge.call("set_led_state", led_on)

# Relatório periódico
if frame_count % 50 == 0:
    fps = frame_count / (time.time() - start_time)
    print(f"Quadros: {frame_count} | {fps:.1f} resultados/s | Detecções: {len(detections_log)}")

detection_stream.on_detect_all(on_detections)
App.run()

```

O callback só chama a Bridge quando o estado do LED muda. Isso evita uma chamada de 7 ms a cada resultado e mantém o callback leve.

Passo 5: Clique em Run. Abra `http://<nome-da-placa>.local:7000` para ver o vídeo com as caixas, e confira que o LED acende quando você entra na cena.

Parte B: Monitoramento do sistema

Passo 6: Abra uma sessão SSH na placa em um segundo terminal:

```
ssh arduino@<nome-da-placa>.local
```

Passo 7: Com o app rodando, execute cada comando e anote:

```
# Uso de CPU por processo. O runner do modelo aparece com o nome do arquivo .eim
top -bn2 -d1 | grep -E "Cpu|python|eim|gst" | tail -8
```

```
# Memoria
free -h
```

```
# Quais zonas termicas existem e qual e qual
for z in /sys/class/thermal/thermal_zone*; do echo "$(basename $z) $(cat $z/type) $(cat $z/temp)"; done
```

No UNO Q, as zonas 3 e 4 (cpuss0-thermal e cpuss1-thermal) são a CPU e a zona 7 (gpu-thermal) é a GPU. Os valores estão em milésimos de grau: 50700 significa 50,7 °C.

```
# Temperatura da CPU, em graus, com uma casa decimal
awk '{printf "%.1f\n", $1/1000}' /sys/class/thermal/thermal_zone3/temp
```

Dica: iostat não vem instalado na placa, e este laboratório não precisa dele. Em uma placa compartilhada, não instale pacotes com apt sem combinar com o instrutor.

Passo 8: Observe a linha %Cpu(s) do top. O runner de modelos ocupa um núcleo inteiro, o que aparece como cerca de 25 % de uso total em uma placa de quatro núcleos, mais o consumo do Python e do GStreamer.

Parte C: Teste sustentado

Passo 9: Deixe o app rodando por pelo menos 5 minutos com uma pessoa ou objeto na cena. A cada minuto, anote a temperatura da CPU e a taxa de resultados por segundo. A taxa aparece no console do App Lab a cada 50 resultados, o que dá uma linha a cada meio minuto com o YOLO-X. Para a temperatura, deixe este laço rodando no terminal SSH; ele imprime a hora e a temperatura uma vez por minuto, e você o interrompe com Ctrl+C:

```
while true; do echo "$(date +%T) $(awk '{printf "%.1f", $1/1000}' /sys/class/thermal/thermal_zone3/temp) C"; sleep 60; done
```

Minuto	Temperatura CPU (°C)	Resultados/s
--------	----------------------	--------------

0		
1		
2		
3		
4		
5		

Dica: em testes de referência, a CPU do UNO Q fica entre 33 e 40 °C em repouso, conforme o ambiente, e sobe para cerca de 50 °C com o modelo YOLO-X rodando continuamente, sem dissipador. O processador

Qualcomm reduz a frequência quando a temperatura ultrapassa o limite térmico. Se a taxa cair com o tempo enquanto a temperatura sobe, essa é a causa.

Passo 10: Se você tiver um dissipador, repita o teste com ele e compare as duas curvas.

Parte D: Alternativa, monitoramento de dentro do app

Em vez de um segundo terminal, o próprio app pode registrar CPU e memória com a biblioteca `psutil`. Ela não vem no container dos apps, e esta é uma boa hora para conhecer o mecanismo de instalar bibliotecas Python em um app: o arquivo `python/requirements.txt`. Cada linha dele é o nome de um pacote, e o App Lab instala todos na partida do app. A placa precisa de internet nessa primeira partida.

Crie um app vazio chamado `monitor-app`, pelo botão **+** ao lado de **Apps**. No terminal da placa, crie o arquivo de requisitos:

```
echo "psutil" > ~/ArduinoApps/monitor-app/python/requirements.txt
```

Substitua todo o conteúdo de `python/main.py` por esta versão mínima e clique em Run. O exemplo oficial “System Resources Logger” faz o mesmo, grava as amostras em um banco de séries temporais e mostra um gráfico na web.

```
from arduino.app_utils import App
import psutil, time

def monitor_loop():
    cpu = psutil.cpu_percent(interval=1)
    mem = psutil.virtual_memory().percent
    print(f"CPU {cpu:.0f}% | RAM {mem:.0f}%")
    time.sleep(4)
```

```
App.run(user_loop=monitor_loop)
```

Saída de referência, com a placa em repouso: CPU 1% | RAM 19%, uma linha a cada 5 segundos. A temperatura não está disponível pelo `psutil` dentro do container; para ela, use o terminal SSH. Para medir o sistema de visão, faça três coisas no app `deploy-final`: acrescente `import psutil` junto dos outros imports do `main.py`, cole a função `monitor_loop` acima da última linha e troque essa linha por `App.run(user_loop=monitor_loop)`; e crie o mesmo `requirements.txt` na pasta `python/` dele.

Parte E: Relatório de desempenho

Passo 11: Escreva o relatório com estas seis seções:

1. **Resumo do modelo:** arquitetura, resolução de entrada, quantização, tamanho.
2. **Desempenho de inferência:** tempo por inferência (página :4912), resultados por segundo no callback, mínimo e máximo.

3. **Avaliação de precisão:** avaliação qualitativa da detecção nos seus objetos de teste.
4. **Recursos do sistema:** CPU, memória e temperatura ao longo dos 5 minutos.
5. **Comportamento térmico:** a taxa caiu? A partir de que temperatura?
6. **Recomendações:** qual é o gargalo e o que você mudaria primeiro.

Exemplo resolvido: com o modelo YOLO-X nano embutido, a inferência leva cerca de 370 ms, o que limita o sistema a menos de 2 resultados por segundo, e a CPU chega a 50 °C. Com o modelo `person-classification`, a inferência cai para 27 ms e o sistema entrega 10 resultados por segundo. Com o YOLO-X, o gargalo não está na câmera, que o Brick abre a 10 quadros por segundo, nem na Bridge, que responde em 7 ms. Está no modelo. A recomendação segue daí: trocar o modelo, ou reduzir a resolução de entrada no Edge Impulse. Com o classificador de pessoa a conta muda: o modelo daria 37 resultados por segundo e o sistema entrega 10, então o gargalo passou a ser a taxa da câmera (Laboratório 8).

Perguntas de checkpoint

P1: O seu sistema mostrou queda de desempenho durante o teste de 5 minutos? Se sim, a que temperatura ela começou e de quanto foi a queda?

P2: Quanto de memória o sistema inteiro usa com o app rodando, de acordo com `free -h`? Quanto sobra?

P3: Se este sistema fosse instalado em uma fábrica para operar 24 horas por dia, quais três mudanças você faria para garantir confiabilidade?

Principais lições

- Implantação é ajuste, velocidade, precisão e confiabilidade ao longo do tempo, não só “o modelo roda”
- O runner de modelos ocupa um núcleo; a folga de CPU e memória do UNO Q é grande, mas a temperatura sobe sob carga contínua
- Meça a temperatura na zona certa; no UNO Q, a CPU são as zonas 3 e 4
- Um relatório de desempenho que separa câmera, modelo e comunicação aponta o gargalo sem adivinhação

Extensão opcional

Adicione o Brick Arduino Cloud ao app, registre variáveis `fps`, `detection_count` e `cpu_temperature`, atualize-as no callback de detecção e acompanhe pelo app IoT Remote no celular. O Laboratório 6 mostra o código da Cloud.

O que entregar

1. O código completo (`python/main.py` e `sketch/sketch.ino`).

2. O relatório de desempenho com as seis seções.
3. Respostas escritas para as perguntas de checkpoint.

Solução de problemas

Problema	Solução
A taxa de resultados cai com o tempo	Limitação térmica. Adicione um dissipador ou reduza a carga trocando o modelo ou a resolução.
O sistema fica sem memória	Feche outros apps na placa (<code>arduino-app-cli app list</code> e <code>stop</code>). Reduza a resolução da câmera.
Não encontro a temperatura da CPU	Liste as zonas com o laço <code>for</code> do Passo 7 e procure <code>cpuss0-thermal</code> .
[ERROR] No Camera Device Found ao clicar em Run	A placa não está vendo nenhuma câmera. Confira o hub e a webcam com <code>v4l2-ctl --list-devices</code> .
sketch folder is incomplete: both sketch.ino and sketch.yaml are required	Falta um dos dois arquivos da pasta <code>sketch/</code> . Refaça a cópia do Passo 3.
NameError: name 'psutil' is not defined	Faltou a linha <code>import psutil</code> no topo do <code>main.py</code> .
ModuleNotFoundError: No module named 'psutil'	Falta o arquivo <code>python/requirements.txt</code> com a linha <code>psutil</code> , ou a placa estava sem internet na partida.
O LED não acompanha a detecção	Confira no console se aparece erro da Bridge. O sketch precisa ter <code>Bridge.provide("set_led_state", ...)</code> .
O vídeo não aparece em :7000	O app precisa da pasta <code>assets/</code> do exemplo. Copie o exemplo de novo em vez de criar um app vazio.

Notas

Espaço para suas anotações.

Laboratório 14: Construindo um agente de borda

Duração	Dificuldade	Pré-requisitos
90-120 min	Intermediário-Avançado	Laboratório 9 concluído

Objetivo

Transformar um app de detecção que reage quadro a quadro em um agente que lembra o que viu, raciocina sobre o tempo, mantém estado e decide sozinho quando agir, tudo rodando localmente no UNO Q.

O que você vai aprender

1. Distinguir inferência (um resultado por quadro, sem memória) de agência (decisão com estado)
2. Implementar uma máquina de estados com os estados IDLE, ALERT e COOLDOWN
3. Adicionar filtro de confiança, detecção sustentada e tempo de espera entre alertas
4. Projetar um padrão de falha segura para quando o modelo está incerto

Materiais necessários

Item	Detalhes
Arduino UNO Q 4GB	Alimentado por hub USB-C com Power Delivery, acesso por Wi-Fi
Webcam USB (UVC)	Conectada ao hub. Este laboratório exige a webcam: o modelo de pessoa não tem versão para o celular neste manual
Computador com Arduino App Lab	Na mesma rede Wi-Fi
Modelo	person-classification, embutido no App Lab

Visão geral do conceito

Nos Laboratórios 9, 11 e 13, cada quadro foi processado, o resultado impresso, e o quadro esquecido. Isso é inferência pura: um sistema reativo que responde “o que estou vendo agora?”.

Um agente responde a outras perguntas: “o que venho vendo nos últimos segundos? Isso é um evento novo ou o mesmo de antes? Devo agir agora ou esperar mais evidência? O que faço se não tenho certeza?”. Um agente tem memória, raciocina sobre padrões no tempo e toma decisões com consequências.

Pense em uma câmera de segurança. A inferência pura diz: “quadro 1, pessoa (85 %); quadro 2, nada; quadro 3, pessoa (62 %); quadro 4, pessoa (91 %)”. Um sistema ingênuo dispararia três alertas. Um agente reconhece um único evento, ignora a falha do quadro 2, e só dispara quando a presença se sustenta por alguns segundos.

Neste laboratório o modelo não muda. O que muda é a camada de decisão em volta dele. Isso é o núcleo da agência de borda: software inteligente sobre hardware capaz.

O modelo escolhido é o `person-classification`, um classificador binário que responde “pessoa” ou “não pessoa” para o quadro inteiro. Ele roda em cerca de 27 ms e entrega 10 resultados por segundo no UNO Q, o que dá ao agente uma amostragem fina para decidir. O detector YOLO-X usado no Laboratório 9 entregaria menos de 2 resultados por segundo e tornaria o agente lento para reagir.

O callback de classificação recebe um dicionário simples, com a confiança de cada classe acima do limiar do Brick. Este modelo tem duas classes: com alguém na cena chega `{ 'person': 0.90 }`, e sem ninguém chega `{ 'non person': 0.97 }`.

Procedimento

Parte A: Da inferência ao rastreamento com estado

Passo 1: Crie um app vazio chamado `smart-agent`, pelo botão **+** ao lado de **Apps**, e adicione o Brick **Video Image Classification** pelo primeiro dos três ícones acima do painel Bricks. Escolha o modelo de pessoa, como no Laboratório 8: clique no Brick, abra a aba **AI models** e use **Brick Configuration**, ou edite o `app.yaml`. A lista de Bricks tem de ficar assim:

```
bricks:
  - arduino:video_image_classification:
      model: person-classification
```

Passo 2: Substitua todo o conteúdo de `sketch/sketch.ino` pelo sketch do LED, o mesmo dos Laboratórios 9 e 13:

```
#include <Arduino_RouterBridge.h>

// LED_BUILTIN acende em LOW
void set_led_state(bool state) {
  digitalWrite(LED_BUILTIN, state ? LOW : HIGH);
}

void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
  digitalWrite(LED_BUILTIN, HIGH);           // começa apagado
  Bridge.begin();
  Bridge.provide("set_led_state", set_led_state);
}

void loop() {}
```

Passo 3: Substitua todo o conteúdo de python/main.py por:

```
from arduino.app_utils import App, Bridge
from arduino.app_bricks.video_imageclassification import VideoImageClassification
import time

stream = VideoImageClassification(confidence=0.3, debounce_sec=0.0)

class DetectionTracker:
    """Memoria e regras de decisao do agente."""

    CONFIDENCE_THRESHOLD = 0.6 # confianca minima para contar como pessoa
    SUSTAIN_SECONDS = 3.0      # presenca continua necessaria para alertar
    CLEAR_SECONDS = 2.0        # ausencia necessaria para encerrar o alerta
    COOLDOWN_SECONDS = 10.0    # espera minima entre dois alertas
    HISTORY_SECONDS = 10.0     # quanto tempo de historico e mantido

    def __init__(self):
        self.history = []      # lista de (timestamp, pessoa_presente: bool)
        self.state = "IDLE"    # IDLE, ALERT ou COOLDOWN
        self.presence_start = None # quando a presenca atual comecou
        self.last_seen = 0.0    # ultima vez que uma pessoa foi vista
        self.alert_start = None
        self.cooldown_start = 0.0
        self.total_alerts = 0

    def update(self, results: dict) -> str:
        now = time.time()
        present = results.get("person", 0.0) >= self.CONFIDENCE_THRESHOLD

        # Memoria: guarda a observacao e descarta o que ficou velho
        self.history.append((now, present))
        self.history = [(t, p) for t, p in self.history if now - t < self.HISTORY_SECONDS]

        if present:
            # Uma presenca nova comeca depois de uma ausencia de CLEAR_SECONDS
            if self.presence_start is None or now - self.last_seen > self.CLEAR_SECONDS:
                self.presence_start = now
                self.last_seen = now

        return self._evaluate(now)

    def _ratio(self, now, seconds):
        """Fracao de observacoes recentes com pessoa presente."""
        recent = [p for t, p in self.history if now - t < seconds]
        return (sum(recent) / len(recent)) if recent else 0.0

    def _evaluate(self, now):
        absent_for = now - self.last_seen

        if self.state == "IDLE":
            # Dispara so quando a presenca ja dura SUSTAIN_SECONDS
            # e a pessoa aparece em mais de 60 % das observacoes desse periodo
            sustained = (self.presence_start is not None
                        and absent_for <= self.CLEAR_SECONDS
                        and now - self.presence_start >= self.SUSTAIN_SECONDS
                        and self._ratio(now, self.SUSTAIN_SECONDS) > 0.6)
```

```

    if sustained:
        self.state = "ALERT"
        self.alert_start = now
        self.total_alerts += 1
        return "ALERT_NEW"
    return "MONITORING"

    if self.state == "ALERT":
        # Encerra quando ninguem e visto ha CLEAR_SECONDS
        if absent_for >= self.CLEAR_SECONDS:
            duration = now - self.alert_start
            self.state = "COOLDOWN"
            self.cooldown_start = now
            return f"ALERT_CLEARED ({duration:.1f}s)"
        return "ALERT_ACTIVE"

    if self.state == "COOLDOWN":
        # A espera conta a partir do fim do alerta
        if now - self.cooldown_start > self.COOLDOWN_SECONDS:
            self.state = "IDLE"
            return "COOLDOWN"

    return "MONITORING"

```

A classe é a memória e a política do agente. A função `update` recebe o resultado do modelo e devolve o que aconteceu. Ela nunca aciona hardware; isso fica com quem a chama.

Parte B: Conectando o agente às ações

Passo 4: Acrescente ao final de `python/main.py`, depois da classe e sem recuo:

```

tracker = DetectionTracker()
led_on = False
events = 0

def set_led(state: bool):
    global led_on
    if state != led_on:
        led_on = state
        Bridge.call("set_led_state", state)

def on_results(results: dict):
    global events
    events += 1
    status = tracker.update(results)

    if status == "ALERT_NEW":
        set_led(True)
        print(f">>> ALERTA #{tracker.total_alerts}: presença sustentada")
    elif status.startswith("ALERT_CLEARED"):
        set_led(False)
        print(f"<<< Alerta encerrado. {status}")
    elif status == "ALERT_ACTIVE":
        pass # LED continua aceso
    else:
        set_led(False) # padrao seguro: LED apagado

```

```

if events % 50 == 0:
    print(f"Estado: {tracker.state} | Alertas: {tracker.total_alerts} | "
          f"Historico: {len(tracker.history)} observacoes")

stream.on_detect_all(on_results)
App.run()

```

Aviso: o Brick chama `on_results` a cada resultado do modelo, cerca de 10 vezes por segundo, desde que alguma classe passe do limiar do Brick. Sem ninguém na cena, o que chega é `{"non person": 0.97}`; a chave `person` não existe, `results.get("person", 0.0)` devolve zero e o agente conta a observação como ausência. Isso só funciona porque o modelo tem uma classe para “não pessoa”. Com um modelo de uma classe só, ou com um detector, o callback simplesmente não seria chamado quando a cena está vazia, e o agente ficaria parado no último estado. O Laboratório 15 trata desse caso com um watchdog.

Passo 5: Clique em Run. Este app não tem página web própria; para ver o enquadramento da câmera, abra a página do runner em `http://<nome-da-placa>.local:4912`. Teste os quatro cenários:

1. Fique parado na frente da câmera. O LED deve acender depois de cerca de 3 segundos e o console mostrar “ALERTA #1”.
2. Saia da cena. O LED deve apagar depois de cerca de 2 segundos, com a duração do alerta.
3. Passe rápido pela câmera. Não deve haver alerta: a presença não se sustentou.
4. Volte para a cena logo depois de um alerta encerrado. Nos 10 segundos de espera, o agente fica em COOLDOWN e não dispara. Se você continuar na cena, o alerta dispara assim que a espera termina.

Dica: `CONFIDENCE_THRESHOLD`, `SUSTAIN_SECONDS` e `COOLDOWN_SECONDS` são a personalidade do agente. Um agente nervoso usa limiar baixo e sustentação curta. Um agente calmo usa limiar alto e sustentação longa. Não há valor certo; há o valor certo para a aplicação.

Parte C: Analisando o comportamento

Passo 6: Registre os eventos com horário. Dentro de `on_results`, cole estas duas linhas logo depois da linha `status = tracker.update(results)`, com o mesmo recuo dela:

```

if status == "ALERT_NEW" or status.startswith("ALERT_CLEARED"):
    print(f"EVENTO {time.strftime('%H:%M:%S')}: {status}")

```

Passo 7: Experimente com os parâmetros e anote o comportamento em cada caso:

Parâmetro	Valor	O que mudou
<code>CONFIDENCE_THRESHOLD</code>	0.3	

Parâmetro	Valor	O que mudou
CONFIDENCE_THRESHOLD	0.8	
SUSTAIN_SECONDS	1.0	
SUSTAIN_SECONDS	5.0	
COOLDOWN_SECONDS	2.0	

Uma pessoa andando de um lado para o outro na frente da câmera é o melhor teste para o COOLDOWN_SECONDS.

Exemplo resolvido: o agente exige presença em 60 % das observações dos últimos 3 segundos. A 10 resultados por segundo, são 30 observações, e 18 delas precisam ter pessoa. Se o modelo errar 5 dessas observações por causa de um reflexo, 25 ainda têm pessoa, bem acima de 18, e o alerta dispara. Com 1,6 resultados por segundo, como no detector do Laboratório 9, seriam apenas 5 observações em 3 segundos, e um único quadro perdido mudaria a decisão. A taxa de amostragem do modelo define quão robusto o agente pode ser.

Perguntas de checkpoint

P1: Qual é a diferença central entre o sistema de detecção do Laboratório 9 e o agente deste laboratório? Dê um exemplo concreto de uma situação em que eles agem de forma diferente.

P2: O agente exige 60 % de presença ao longo de 3 segundos. A 10 resultados por segundo, quantas observações com pessoa são necessárias para disparar um alerta? E se o modelo entregasse apenas 2 resultados por segundo?

P3: Por que existe um tempo de espera entre alertas? Qual problema do mundo real ele resolve?

P4: Se a câmera for desconectada ou o modelo travar, o callback deixa de ser chamado. O que acontece com o LED? Isso é um padrão seguro? Como você melhoraria?

Principais lições

- Um agente tem memória, estado e raciocínio temporal; a inferência sozinha não tem memória
- Filtro de confiança, detecção sustentada e tempo de espera eliminam alarmes falsos sem mudar o modelo
- Os parâmetros do agente definem o comportamento e precisam ser ajustados para cada aplicação
- Quando o agente não sabe, o sistema deve ir para o estado seguro, com o LED apagado, e não travar aceso

A resposta da P4 é o gancho do próximo laboratório: se o callback para de ser chamado, ninguém apaga o LED. O Laboratório 15 adiciona um watchdog para isso.

O que entregar

1. O `python/main.py` completo com a classe `DetectionTracker`.
2. A tabela de experimentos com os parâmetros preenchida.
3. O registro de eventos de uma sessão de 5 minutos.
4. Respostas escritas para as perguntas de checkpoint.

Solução de problemas

Problema	Solução
<code>[ERROR] No Camera Device Found</code> ao clicar em Run	A placa não está vendo nenhuma câmera. Confira o hub e a webcam com <code>v4l2-ctl --list-devices</code> .
O agente nunca dispara	Baixe <code>CONFIDENCE_THRESHOLD</code> para 0.4. Para ver o que o modelo está entregando, acrescente <code>print(results)</code> como primeira linha de <code>on_results</code> e confira se a chave <code>person</code> aparece quando você está na cena.
O agente dispara demais	Aumente <code>SUSTAIN_SECONDS</code> e <code>COOLDOWN_SECONDS</code> . Aumente <code>CONFIDENCE_THRESHOLD</code> .
O LED fica preso aceso	O callback parou de ser chamado. Veja a P4 e o watchdog do Laboratório 15.
A chave do dicionário não é <code>person</code>	O modelo escolhido no <code>app.yaml</code> não é o <code>person-classification</code> . Confira o <code>model:</code> e reinicie o app.
O app demora para iniciar	O runner do modelo sobe em um container próprio e precisa ficar “Healthy” antes do app. É normal levar alguns segundos a mais.

Notas

Espaço para suas anotações.

Laboratório 15: Benchmarking e confiabilidade

Duração	Dificuldade	Pré-requisitos
90-120 min	Intermediário-Avançado	Laboratórios 13 e 14 concluídos

Objetivo

Medir a aplicação de Edge AI com método, registrar a saúde do sistema ao longo do tempo, provocar falhas de câmera e de rede para ver como o sistema reage, implementar um watchdog com estado seguro e produzir um relatório de referência.

O que você vai aprender

1. Projetar um benchmark justo e reproduzível, com aquecimento e estatística adequada
2. Medir o desempenho de ponta a ponta, incluindo percentis, não só médias
3. Testar cenários de falha: câmera desconectada e rede fora do ar
4. Implementar um watchdog e degradação controlada para operação em produção

Materiais necessários

Item	Detalhes
Arduino UNO Q 4GB	Alimentado por hub USB-C com Power Delivery, acesso por Wi-Fi
Webcam USB (UVC)	Conectada ao hub, em porta que você consiga desconectar. Este laboratório exige a webcam
Computador com Arduino App Lab	Na mesma rede Wi-Fi; dois terminais SSH abertos
App smart-agent	Do Laboratório 14, usado nas Partes C e D

Visão geral do conceito

No Laboratório 13 você mediu o sistema na bancada, com boa luz e tudo conectado. Isso diz pouco sobre o comportamento no mundo real. Benchmarking é uma disciplina, e um benchmark mal feito é pior do que nenhum, porque dá confiança falsa.

Um benchmark rigoroso controla as variáveis: entrada consistente, período de aquecimento para o sistema estabilizar, amostras suficientes para a estatística valer, e monitoramento do ambiente. Ele também reporta percentis. Se a média é 100 ms

mas o percentil 99 é 500 ms, o usuário vai ver travadas de meio segundo que a média esconde.

E ele testa falhas. O que acontece quando a câmera é desconectada? Quando o Wi-Fi cai? Quando o sistema roda por horas? A pergunta deste laboratório não é “o modelo funciona?”, mas “o sistema vai funcionar de forma confiável, dia após dia?”.

Procedimento

Parte A: O benchmark

Passo 1: Crie um app vazio chamado `benchmark`, adicione o Brick **Video Image Classification** e escolha o modelo `person-classification`, como no Passo 1 do Laboratório 14. O sketch pode ficar como veio: o `benchmark` não usa o MCU.

Passo 2: Substitua todo o conteúdo de `python/main.py` por:

```
from arduino.app_utils import App
from arduino.app_bricks.video_imageclassification import VideoImageClassification
import time, statistics

stream = VideoImageClassification(confidence=0.3, debounce_sec=0.0)

# Um benchmark justo descarta o inicio (modelo carregando, caches frios)
# e mede um numero fixo de resultados em regime.
WARMUP = 30          # resultados descartados no inicio
MEASURE = 300        # resultados medidos (cerca de 30 s a 10 resultados/s)

timestamps = []      # instante de chegada de cada resultado medido
count = 0
done = False

def on_results(results: dict):
    global count, done
    if done:
        return
    count += 1
    if count <= WARMUP:
        if count == WARMUP:
            print(f"Aquecimento concluido ({WARMUP} resultados). Medindo...")
            return
    timestamps.append(time.perf_counter())
    if len(timestamps) == MEASURE:
        done = True
        report()

def report():
    # Intervalos entre resultados consecutivos, em ms: uma unica diferenca
    intervals = [(b - a) * 1000 for a, b in zip(timestamps, timestamps[1:])]
    intervals_sorted = sorted(intervals)
    avg = statistics.mean(intervals)
    p95 = intervals_sorted[int(len(intervals_sorted) * 0.95)]
    p99 = intervals_sorted[int(len(intervals_sorted) * 0.99)]
    print("=" * 50)
    print(f"RESULTADOS ({len(intervals)} intervalos)")
```

```

print("=" * 50)
print(f"Intervalo medio: {avg:.1f} ms ({1000/avg:.1f} resultados/s)")
print(f"Desvio padrao: {statistics.stdev(intervals):.1f} ms")
print(f"Minimo / Maximo: {min(intervals):.1f} / {max(intervals):.1f} ms")
print(f"P95: {p95:.1f} ms")
print(f"P99: {p99:.1f} ms")

stream.on_detect_all(on_results)
App.run()

```

O que se mede aqui é o intervalo entre resultados consecutivos, que é o que a aplicação sente. Ele inclui captura, transferência para o runner, inferência e entrega ao callback.

Passo 3: Clique em Run com uma pessoa na cena e anote os resultados. Repita três vezes, parando e iniciando o app, para ver a variação entre execuções.

Dica: os percentis importam mais que a média em sistemas de tempo real. P95 é o valor abaixo do qual ficam 95 % dos intervalos; P99, 99 %. A distância entre a média e o P99 mostra o tamanho das travadas ocasionais.

Exemplo resolvido: com o modelo person-classification, o runner reporta cerca de 27 ms por inferência, mas os resultados chegam ao callback a cerca de 100 ms de intervalo, 10 por segundo. Quem define esse intervalo é a câmera, que o Brick abre a 10 quadros por segundo (Laboratório 8): o modelo fica ocioso boa parte do tempo, esperando o próximo quadro. Medir só a inferência daria 37 resultados por segundo, quase quatro vezes o que a aplicação recebe de fato.

Parte B: Saúde do sistema

Passo 4: No segundo terminal SSH, crie o script de monitoramento:

```

cat > ~/monitor.sh <<'EOF'
#!/bin/bash
# Registra CPU (usuario+sistema), memoria usada e temperatura da CPU a cada 6 ou 7 s
echo "Hora,CPU%,Mem_MB,Temp_C"
while true; do
    CPU=$(top -bn2 -d1 | grep 'Cpu(s)' | tail -1 | awk '{print $2 + $4}')
    MEM=$(free -m | awk '/Mem:/ {print $3}')
    TEMP=$(cat /sys/class/thermal/thermal_zone3/temp 2>/dev/null || echo 0)
    TEMP_C=$(echo "scale=1; $TEMP / 1000" | bc)
    echo "$(date +%H:%M:%S),$CPU,$MEM,$TEMP_C"
    sleep 5
done
EOF
bash ~/monitor.sh | tee ~/system_log.csv

```

A zona térmica 3 é a CPU do UNO Q. O bc já vem instalado. A soma \$2 + \$4 junta o tempo de usuário e o de sistema, porque os containers geram bastante trabalho de sistema. O top -bn2 tira duas amostras e o script usa a segunda, porque a primeira mostra a média desde a partida da placa. O script roda até você apertar Ctrl+C, que também encerra a gravação do arquivo; em repouso ele registra de 1 a 6 % de CPU.

Passo 5: Deixe o monitor rodando durante os benchmarks e durante os testes de falha. O arquivo `system_log.csv` vai para o relatório. Para copiá-lo ao PC:

```
scp arduino@<nome-da-placa>.local:~/system_log.csv .
```

Parte C: Tolerância a falhas

Passo 6: Clique em Stop no app benchmark: só um app usa a câmera por vez. Abra o app smart-agent do Laboratório 14 e clique em Run. Com o LED aceso por uma pessoa na cena, desconecte fisicamente a webcam do hub. Observe e anote:

- O app trava, imprime erro, ou fica em silêncio?
- O LED continua aceso?
- Ao reconectar a câmera, o app volta sozinho?

Aviso: em um sistema de produção, uma câmera desconectada sem tratamento deixa os atuadores no último estado: uma válvula aberta, um alarme tocando. É por isso que o estado seguro importa.

Passo 7: Adicione um watchdog ao app smart-agent. Um watchdog é um vigia: um trecho de código que roda em paralelo e confere, de tempos em tempos, se o resto do sistema continua dando sinal de vida. Clique em Stop. No `python/main.py`, mantenha a classe `DetectionTracker` como está e substitua tudo o que vem depois dela, da linha `tracker = DetectionTracker()` até o fim do arquivo, por:

```
import threading

tracker = DetectionTracker()
led_on = False
events = 0
last_result_time = time.time() # horario do ultimo resultado recebido
WATCHDOG_TIMEOUT = 15.0 # segundos sem resultado para considerar falha
safe_mode = False

def set_led(state: bool):
    global led_on
    if state != led_on:
        led_on = state
        Bridge.call("set_led_state", state)

def on_results(results: dict):
    global events, last_result_time
    last_result_time = time.time() # sinal de vida para o watchdog
    events += 1
    status = tracker.update(results)

    if status == "ALERT_NEW" or status.startswith("ALERT_CLEARED"):
        print(f"EVENTO {time.strftime('%H:%M:%S')}: {status}")

    if status == "ALERT_NEW":
        set_led(True)
        print(f">>> ALERTA #{tracker.total_alerts}: presenca sustentada")
    elif status.startswith("ALERT_CLEARED"):
```

```

        set_led(False)
        print(f"<<< Alerta encerrado. {status}")
    elif status == "ALERT_ACTIVE":
        pass # LED continua aceso
    else:
        set_led(False) # padrao seguro: LED apagado

    if events % 50 == 0:
        print(f"Estado: {tracker.state} | Alertas: {tracker.total_alerts} | "
              f"Historico: {len(tracker.history)} observacoes")

def watchdog_check():
    """Se o modelo parou de responder, leva o sistema para o estado seguro."""
    global safe_mode, led_on
    parado = time.time() - last_result_time > WATCHDOG_TIMEOUT
    if parado and not safe_mode:
        safe_mode = True
        print(f"WATCHDOG {time.strftime('%H:%M:%S')}: sem resultados ha "
              f"{WATCHDOG_TIMEOUT:.0f} s. Entrando em modo seguro.")
        tracker.state = "IDLE" # o agente esquece o alerta em curso
        tracker.presence_start = None # e volta a exigir presenca sustentada
        led_on = False
        try:
            Bridge.call("set_led_state", False)
        except Exception as e:
            print(f"WATCHDOG: falha ao falar com o MCU: {e}")
    elif not parado and safe_mode:
        safe_mode = False
        print(f"WATCHDOG {time.strftime('%H:%M:%S')}: resultados voltaram. Operacao norma
1.")

def watchdog_loop():
    while True: # roda em paralelo ao resto do app
        time.sleep(5)
        watchdog_check()

threading.Thread(target=watchdog_loop, daemon=True).start()

stream.on_detect_all(on_results)
App.run()

```

O watchdog faz três coisas ao entrar em modo seguro: apaga o LED, atualiza a variável `led_on` e devolve o agente ao estado IDLE. As duas últimas importam tanto quanto a primeira. Se o watchdog só apagasse o LED, o agente continuaria achando que está em ALERT com o LED aceso, e quando a câmera voltasse ele não mandaria acender de novo. O estado do software e o estado do mundo físico têm de mudar juntos.

Passo 8: Clique em Run e repita o teste de desconexão. Em até 20 segundos o watchdog deve apagar o LED e registrar a mensagem. Reconecte a câmera e observe se os resultados voltam. Quando voltarem, o LED acende de novo depois de 3 segundos de presença, como em um alerta novo, e o console mostra resultados voltaram. Como o watchdog só confere a cada 5 segundos, essa mensagem pode aparecer depois da linha do alerta novo.

Passo 9: Teste a rede. Com a placa no hub, o App Lab e o SSH chegam a ela pelo Wi-Fi, então ao derrubar a rede você perde o console e os terminais. O que continua visível é o LED. Com o smart-agent rodando, desligue o Wi-Fi do roteador por um minuto e, nesse intervalo, entre e saia da frente da câmera: o LED continua acendendo e apagando? Religue o Wi-Fi, reconecte o SSH e leia o que o app registrou enquanto você estava sem acesso:

```
arduino-app-cli app logs user:smart-agent | grep -a -E "EVENTO|WATCHDOG" | tail -20
```

As linhas EVENTO, com horário, mostram se a detecção local continuou. O log guarda também os eventos de execuções anteriores; use os horários para achar os do seu teste. Se o app usa o Brick Arduino Cloud, veja também se as variáveis pararam de atualizar e se a Cloud retomou sozinha quando a rede voltou.

Dica: essa é a vantagem central da computação de borda. A inferência não depende da rede. A nuvem é útil para relatório, não é requisito para operar.

Parte D: Relatório de referência

Passo 10: Escreva o relatório com cinco seções:

1. **Metodologia:** o que foi medido, quantos resultados, quanto aquecimento, em que condições de luz e cena.
2. **Resultados:** média, desvio padrão, mínimo, máximo, P95 e P99 das três execuções.
3. **Saúde do sistema:** CPU, memória e temperatura ao longo do tempo, a partir do CSV.
4. **Tolerância a falhas:** o que aconteceu com a câmera desconectada, com e sem watchdog, e com a rede fora.
5. **Recomendações:** o sistema está pronto para produção? O que você mudaria?

Perguntas de checkpoint

P1: Por que o período de aquecimento é necessário para um benchmark honesto? O que acontece nos primeiros resultados?

P2: O seu P95 é 110 ms e o P99 é 250 ms. O que pode causar esses intervalos longos ocasionais? Como você investigaria, usando o CSV do monitor?

P3: Um cliente exige 99,9 % de disponibilidade, menos de 9 horas paradas por ano. Qual única mudança no seu sistema teria o maior impacto na confiabilidade?

Principais lições

- Benchmark justo exige aquecimento, condições controladas, repetição e percentis, não só média
- O intervalo entre resultados é a métrica que a aplicação sente; a inferência é só uma parte dele

- Monitorar CPU, memória e temperatura revela problemas que a medição por quadro não mostra
- Câmera desconectada, rede caída e aquecimento são falhas normais; um watchdog com estado seguro evita que elas virem acidentes

O que entregar

1. O código do benchmark e o `main.py` do `smart-agent` com o watchdog.
2. A saída das três execuções do benchmark.
3. O arquivo `system_log.csv`.
4. O relatório com as cinco seções.
5. Respostas escritas para as perguntas de checkpoint.

Solução de problemas

Problema	Solução
Os resultados variam muito entre execuções	Use a mesma iluminação e a mesma posição da câmera. Feche outros apps na placa.
Temperatura acima de 85 °C	Pare o teste. Adicione dissipador. Reduza a resolução ou troque de modelo.
[ERROR] No Camera Device Found ao clicar em Run	A placa não está vendo nenhuma câmera. Se você desconectou a webcam no teste, reconecte antes de iniciar o app.
O watchdog dispara em operação normal	Aumente o <code>WATCHDOG_TIMEOUT</code> para 30 s. Com o modelo <code>person-classification</code> o callback continua sendo chamado mesmo sem ninguém na cena, porque chega a classe <code>non person</code> ; se ele parar de ser chamado, há falha real.
O app não recupera depois de reconectar a câmera	O Brick tenta reconectar sozinho. Se não voltar em um minuto, pare e inicie o app.
<code>monitor.sh</code> mostra temperatura 0	A zona térmica 3 não existe nessa imagem. Liste as zonas com o laço <code>for</code> do Laboratório 13 e use a que se chama <code>cpuss0-thermal</code> .

Notas

Espaço para suas anotações.

Apêndice A: Referência rápida

Comandos do arduino-app-cli

Todos rodam no terminal da placa, por SSH ou ADB.

```
# Apps
arduino-app-cli app list                # Lista apps do usuario e exemplos
arduino-app-cli app new "nome"          # cria /home/arduino/ArduinoApps/nome
arduino-app-cli app new "nome" --from-app examples:inspirational/object-detection # o "
Copy and edit app" pela linha de comando
arduino-app-cli app start user:nome      # inicia app do usuario
arduino-app-cli app start examples:inspirational/blink # inicia um exemplo
arduino-app-cli app stop user:nome
arduino-app-cli app restart user:nome
arduino-app-cli app logs user:nome --all # Log do Python e dos containers, inclu
sive de execucoes anteriores
arduino-app-cli app export user:nome     # exporta para .zip
arduino-app-cli app import arquivo.zip
arduino-app-cli app destroy user:nome    # remove containers, rede e cache do ap
p; os arquivos ficam
arduino-app-cli app clean-cache user:nome # apaga so o cache (o ambiente Python)
do app

# Bricks e modelos
arduino-app-cli brick list
arduino-app-cli brick details arduino:web_ui
arduino-app-cli model list              # modelos embutidos e importados
arduino-app-cli model delete --help    # como remover um modelo importado

# MCU
arduino-app-cli monitor                 # monitor serial do sketch (Monitor.pri
ntln)

# Sistema
arduino-app-cli version
arduino-app-cli system update --only-arduino # atualiza os pacotes Arduino
arduino-app-cli system set-name "novo-nome" # muda o nome da placa (e o .local)
arduino-app-cli system network-mode --help  # modo de rede
arduino-app-cli system cleanup              # remove imagens de container sem uso
arduino-app-cli system keyboard --help     # layout de teclado no modo SBC
```

Os IDs de app têm prefixo: user: para os seus, examples: para os exemplos; arduino-app-cli app list mostra todos. Os arquivos dos exemplos ficam em /var/lib/arduino-app-cli/examples/. Para os exemplos básicos, o ID é o caminho da pasta (examples:core-and-foundational/...). Os da seção Inspirations têm uma pasta a mais no disco: examples:inspirational/blink está em inspirational/common/blink.

Sobre destroy e clean-cache: cada app executado deixa na placa um container parado e uma rede Docker, e o app stop não os remove. Depois de umas trinta execuções de apps diferentes, um app novo pode falhar com could not find an available, non-overlapping IPv4 address pool. O app destroy libera esses recursos e **não apaga os seus arquivos**: a pasta do app continua em ArduinoApps/ e ele volta a funcionar no próximo Run. Não rode clean-cache antes de destroy; nessa ordem o destroy responde que deu

certo, mas deixa o container e a rede para trás. Para apagar um app de verdade, use o menu do app no App Lab ou remova a pasta dele.

Comandos Linux úteis na placa

```
uname -a                # kernel e arquitetura (aarch64)
cat /etc/os-release      # versao do Debian
nproc                   # nucleos de CPU (4)
free -h                 # memoria (3,6 GiB visiveis)
df -h                   # disco: / e /home/arduino
top                      # processos (q para sair)
ip -4 addr show wlan0    # endereco IP da placa no Wi-Fi (linha "inet")
v4l2-ctl --list-devices  # cameras conectadas (ls /dev/video* engana)
lsusb                   # dispositivos USB no hub
cat /sys/class/thermal/thermal_zone3/temp # temperatura da CPU, em milésimos de grau
ls /home/arduino/ArduinoApps/ # seus apps
ls /var/lib/arduino-app-cli/examples/ # exemplos oficiais
docker ps --format '{{.Names}}' # containers dos apps em execucao (<pasta-do-ap
p>-main-1)
nano arquivo.py          # editor de texto: Ctrl+O e Enter salvam, Ctrl+
X sai
```

Transferência de arquivos a partir do PC:

```
scp arquivo arduino@<nome-da-placa>.local:/home/arduino/
scp arduino@<nome-da-placa>.local:/home/arduino/arquivo .
```

Como conectar à placa

Método	Como	Quando usar
App Lab, modo Desktop (USB)	Cabo USB-C ao PC; a placa aparece sozinha	Desenvolvimento sem câmera USB
App Lab, modo Rede (Wi-Fi)	Placa na mesma rede; o App Lab a descobre	Sempre que a porta USB-C estiver ocupada pelo hub
SSH	ssh arduino@<nome-da-placa>.local ou pelo IP	Terminal pela rede, transferência de arquivos
ADB	adb shell com a placa no USB do PC	Terminal sem depender da rede
SBC	Hub USB-C com HDMI, teclado e mouse; monitor ligado à placa	Uso independente, sem PC

A placa tem uma única porta USB-C. Ela alimenta a placa e, no modo Desktop, liga ao PC. Para usar webcam ou outros dispositivos USB, é necessário um hub USB-C com Power Delivery: a fonte entra no hub, a webcam na USB-A do hub, e o acesso à placa passa a ser por Wi-Fi. A placa pode reiniciar na troca do cabo.

Estrutura de um app

```
meu-app/
  app.yaml          # nome, icone, descricao, bricks
  python/main.py    # aplicacao Python (roda em container no Linux)
```

```
python/requirements.txt # opcional: bibliotecas Python extras, uma por linha
sketch/sketch.ino      # código do MCU
sketch/sketch.yaml     # plataforma e bibliotecas (gerado pelo App Lab)
assets/                # index.html e arquivos da Web UI
README.md
```

Três regras que evitam erros comuns:

- A pasta `sketch/` é opcional, mas, se existir, precisa dos dois arquivos. Com um só, o app não inicia: `sketch folder is incomplete: both sketch.ino and sketch.yaml are required.`
- Os exemplos de visão e de Web UI copiados com “Copy and edit app” vêm sem a pasta `sketch/`. Para usar o MCU neles, copie a pasta de um app que a tenha, como o exemplo Blink (Laboratório 9, Passo 6).
- O container dos apps traz NumPy, OpenCV, Pillow e websockets, entre outras. Uma biblioteca que não esteja lá, como a `psutil`, entra pelo `python/requirements.txt` e é instalada na partida do app, com a placa na internet (Laboratório 13, Parte D).

API essencial

```
from arduino.app_utils import App, Bridge
```

```
Bridge.call("funcao", arg1, arg2)      # chama o sketch e espera o retorno (~7 ms)
Bridge.notify("funcao", arg1)          # chama sem esperar
Bridge.provide("nome", callback)       # expoe uma funcao Python ao sketch
App.run(user_loop=funcao)              # inicia os Bricks e repete funcao na thread principal
```

```
#include <Arduino_RouterBridge.h>
void setup() {
    Bridge.begin();
    Bridge.provide("funcao", funcao); // uma linha por funcao exposta
}
void loop() {}                       // pode ficar vazio
// Sketch -> Python: Bridge.notify("callback_python", valor);
// LED_BUILTIN acende em LOW. Modulinos: Modulino.begin(Wire1)
```

Bricks mais usados no manual:

app.yaml	Import em Python
arduino:web_ui	from arduino.app_bricks.web_ui import WebUI
arduino:video_object_detection	from arduino.app_bricks.video_objectdetection import VideoObjectDetection
arduino:video_image_classification	from arduino.app_bricks.video_imageclassification import VideoImageClassification
arduino:object_detection	from arduino.app_bricks.object_detection import ObjectDetection
arduino:arduino_cloud	from arduino.app_bricks.arduino_cloud import ArduinoCloud

Modelos embutidos (escolhidos com `model: no app.yaml`): `yolox-object-detection`, `face-detection`, `mobilenet-image-classification`, `person-classification`, entre outros; a lista completa vem de `arduino-app-cli model list`.

Especificações do UNO Q

Componente	Especificação
MPU	Qualcomm QRB2210: quatro núcleos Cortex-A53 a 2,0 GHz
MCU	STM32U585: Cortex-M33 a 160 MHz, com 786 KB de RAM no chip; para o sketch ficam 768 KB de flash e 256 KB de RAM
GPU	Adreno 702
DSP	Qualcomm Hexagon (não usado pelos Bricks desta versão)
RAM	4 GB LPDDR4 (3,6 GiB visíveis ao Linux)
Armazenamento	eMMC de 32 GB (na variante 4GB)
Wi-Fi	Wi-Fi 5, 2,4 e 5 GHz
Bluetooth	5.1
Câmera	Conector MIPI-CSI e USB via hub
Alimentação	USB-C 5 V / 3 A ou VIN 7-24 V
Sistema	Debian 13 no MPU, Zephyr no MCU
Formato	Compatível com Arduino UNO